3D Collision Avoidance System for Autonomous UAVs in Uncooperative Indoor Environments

DIPLOMA THESIS

Conducted in partial fulfillment of the requirements for the degree of a
Diplom-Ingenieur (Dipl.-Ing.)

supervised by
Ao.Univ.Prof. Dipl.-Ing. Dr.techn. Markus Vincze
and
Dr.techn. Francesco d'Apolito (AIT)

submitted at the
University of Technology Vienna
Faculty of Electrical Engineering and Information Technology Automation and Control Institute

by
Noah Maikisch BSc.
Matr.Nr.: 11779601

Vienna, January 2024

Vision for Robots Group
A-1040 Wien, Gußhausstr. 27–29, Internet:https://www.acin.tuwien.ac.at

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Acknowledgement

I want to express my sincere thanks to everyone who played a role in the successful completion of my master's thesis, both through professional guidance and personal support. First and foremost, I would like to thank my supervisors, Markus Vincze and Francesco d'Apolito, for their support and guidance throughout this thesis. Their expertise and feedback have been crucial in forming this work, and I am genuinely thankful for the opportunity to learn from them. I am equally grateful to Phillipp Fanta-Jende, Felix Bruckmüller, and Christoph Sulzbachner for their insightful comments and constructive ideas, which significantly enrich this work. Special thanks go to my colleagues Wolfgang Bolz, Marco Cella, Francesco Vultaggio, Jules Salzinger, and Pierrick Bournez. The collaborative atmosphere and our regular weekly sessions generated a wealth of ideas. I cannot emphasize enough how their contributions and camaraderie have enhanced this work and my overall experience. Special thanks to the Austrian Institute of Technology for offering me the opportunity to conduct my thesis research. I also want to extend my heartfelt thanks to my family and friends for their constant support and encouragement. A particular mention goes to my girlfriend Anna and my father Andreas, whose unwavering belief in my abilities has been a constant source of strength and inspiration. Lastly, I wish to acknowledge everyone else who directly or indirectly contributed to this thesis, including reviewers and all those who provided technical assistance. Thank you all for making this journey both possible and rewarding.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Abstract

Unmanned Aerial Vehicles (UAV), specifically multicopters, have gained increasing relevance in various application areas in recent years. Since UAV collisions often lead to total crashes and damage, there is an increasing focus on collision avoidance methods, particularly for indoor operations. This work focuses on different approaches for collision avoidance for a custom-built drone operating in unstructured indoor environments. The developed quadcopter platform is equipped with an autopilot system, various sensors, and an integrated onboard computer and is designed to perform fully automated exploratory flights in unknown indoor areas. It is intended to be used in emergencies to assist first responders. Obstacles in the environment are detected using LiDAR Time-of-Flight sensors, and the collision avoidance algorithms then generate new 3D waypoints to circumvent these obstacles. The avoidance calculations are performed on a single-board computer that uses Linux and the Robot Operating System (ROS), passing the calculated positions to the autopilot system. Two different 3D collision avoidance algorithms are implemented and tested: the Artificial Potential Field (APF) and the Dynamic Window Approach (DWA), both extended by the Bug2 algorithm to overcome the problem of local minima. Both algorithms prevent collisions with objects of any size and shape in all three spatial dimensions. The final evaluation shows that the APF approach is superior in calculation demand and performance and is preferred over the DWA method. The validation was carried out in simulation through Hardware-in-the-loop tests under real conditions and real-world experiments.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Kurzfassung

Unbemannte Luftfahrzeuge, im speziellen Multicopter, haben in den letzten Jahren in verschiedenen Anwendungsfeldern immer mehr an Relevanz gewonnen. Insbesondere bei Einsätzen in Innenräumen mit unbekannten Hindernissen wird zunehmend an Algorithmen für die Kollisionsvermeidung gearbeitet, da Drohnen bei einer Kollision meist zumindest vorübergehend flugunfähig werden. Im Fokus dieser Arbeit stehen zwei Methoden zur Kollisionsvermeidung für eine Drohne, die in unstrukturierten und unbekannten Innenbereichen zur Anwendung kommt. Die dafür entwickelte Quadrotor-Plattform ist mit einem Autopilotsystem, diversen Sensoren und einem integrierten Boardcomputer ausgestattet um vollautonome Erkundungsflüge in unbekannten Innenbereichen durchzuführen und soll in Notfallsituationen zur Unterstützung von Ersthelfern eingesetzt werden. Hindernisse in der Umgebung werden mittels LiDAR Time-of-Flight Sensoren erkannt und die Kollisionsvermeidungsalgorithmen generieren daraufhin neue 3D-Wegpunkte zur Ausweichung dieser Hindernisse. Die Berechnungen werden direkt auf einem Einplatinencomputer durchgeführt, der das Robot Operating System (ROS) verwendet um die neu berechneten Wegpunkte an das Autopilotsystem weiterzuleiten. Es wurden zwei unterschiedliche Algorithmen zur Kollisionsvermeidung in 3D implementiert und getestet: das Artificial Potential Field (APF) und der Dynamic Window Approach (DWA), beide erweitert durch den Bug2 Algorithmus, um Probleme mit lokalen Minima zu minimieren. Beide Algorithmen verhindern Kollisionen mit Objekten jeglicher Größe und Form in allen drei räumlichen Dimensionen. In der abschließenden Bewertung zeigt sich, dass die APF-Methode in Bezug auf Geschwindigkeit, Smoothnes, Rechenleistung und Zuverlässigkeit überlegen ist und daher gegenüber dem DWA Verfahren bevorzugt wird. Die Validierung erfolgte sowohl in Simulationen, Hardware-in-the-Loop Tests unter realen Bedingungen und durch Testflüge.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Contents

1. Introduction **6**
- 1.1. Challenge . . . 7
- 1.2. Contribution . . . 7
- 1.3. Results Preview . . . 8
- 1.4. Thesis Outline . . . 9

2. Literature Review **10**
- 2.1. Unmanned Aerial Vehicles . . . 10
  - 2.1.1. UAV Types . . . 10
- 2.2. Perception . . . 12
  - 2.2.1. Ultrasonic Sensors . . . 13
  - 2.2.2. mmWave Radar . . . 13
  - 2.2.3. LiDAR Sensors . . . 14
    - ToF Sensors . . . 14
  - 2.2.4. Vision Based Sensors . . . 15
    - Monocular Cameras . . . 15
    - Stereo Cameras . . . 15
    - RGB-D Cameras . . . 16
  - 2.2.5. Sensors comparison . . . 16
- 2.3. Collision Avoidance Algorithms . . . 17
  - 2.3.1. Global Approaches . . . 18
  - 2.3.2. Reactive Approaches . . . 19
    - Artificial Potential Field . . . 19
    - Virtual Force Field . . . 20
    - Vector Field Histogram . . . 21
    - Optical Flow . . . 22
    - Bug Algorithms . . . 23
    - Dynamic Window Approach . . . 24
    - Other Methods . . . 24

3. UAV Setup **25**
- 3.1. Hardware Equipment . . . 25
  - 3.1.1. Frame . . . 25
  - 3.1.2. IMU . . . 26
  - 3.1.3. Flight Controller . . . 26
  - 3.1.4. Optical Flow . . . 27
  - 3.1.5. Companion Computer . . . 28
  - 3.1.6. Perception Sensors . . . 28
- 3.2. SLAM . . . 29

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.
TU Bibliothek Your knowledge hub WIEN

3.3. Communication . . . 30
3.3.1. MAVLink Message Protocol . . . 31
3.3.2. ROS . . . 31

4. Mathematical Modeling **33**
4.1. Coordinate Frames . . . 33
4.2. Rotations . . . 35
Rotation Matrix . . . 35
Quaternions . . . 36
Transformations Between Quaternions and RPY . . . 36
4.3. UAV Dynamics . . . 37
4.4. Flight Control . . . 39

5. Collision Avoidance **41**
5.1. Obstacle Detection . . . 42
5.2. Avoidance Algorithms . . . 43
5.2.1. Artificial Potential Field . . . 43
5.2.2. Dynamic Window Approach . . . 51
5.2.3. Bug2 . . . 55

6. Experiments & Results **57**
6.1. Simulation Environment . . . 57
6.2. Simulation Results . . . 60
APF+ Method . . . 60
DWA Method . . . 63
Algorithm Comparison . . . 66
6.3. Hardware in the Loop Tests . . . 70
6.4. Experimental Validation . . . 73
Experiment 1 . . . 73
Experiment 2 . . . 75

7. Conclusions and Future Work **77**

A. Appendix **79**
A.1. Smoothness Factors . . . 79
A.2. Simulation Results Destroyed Garage . . . 80
A.2.1. APF+ and DWA Performance Metrics . . . 80
A.2.2. APF+ Trajectories . . . 81
A.2.3. DWA Trajectories . . . 83
A.3. APF Test Flight Trajectories . . . 85

Bibliography **86**

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

List of Abbreviations

A-SLAM Active Simultaneous Location And Mapping

AI Artificial Intelligence

AIT Austrian Institute of Technology

APF Artificial Potential Field

APF+ proposed improved Artificial Potential Field

BFA Bug Family Algorithm

CPP Coverage Path Planning

DBSCAN Density-Based Spatial Clustering of Applications with Noise

DOF Degree Of Freedom

DWA Dynamic Window Approach

EKF Extended Kalman Filter

ENU East-North-Up

ESC Electrical Speed Controller

FOV Field of View

GCS Ground Control Station

GNSS Global Navigation Satellite System

HALE High Altitude Long Endurance

HITL Hardware in the Loop

IMU Inertial Measurement Units

LED Light-Emitting Diode

LiDAR Light Detection And Ranging

MALE Medium Altitude Long Endurance

MAV Micro Unmanned Area Vehicle

MAVLink Micro Air Vehicle Link

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

MEMS Micro Electric Mechanical System

MUAV Mini Unmanned Area Vehicle

NED North-East-Down

PRM Probabilistic Roadmap

PWM Pulse-Width Modulation

Radar Radio Detection And Ranging

RGB-D Color and Depth Camera

RMS Root Mean Square

ROS Robot Operating System

RPY Roll Pitch Yaw

RRT Rapidly exploring Random Tree

RTF Ready To Fly

SITL Software in the Loop

SLAM Simultaneous Location And Mapping

ToF Time of Flight

TUAV Tactical Unmanned Area Vehicle

UAV Unmanned Area Vehicle

VFF Virtual Force Field

VFH Vector Field Histogram

WFM Wall-Following Method

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

List of Tables

2.1. Summarized sensor comparison for obstacle detection. 17

3.1. Utilized UAV hardware equipment 32

6.1. Special configurations trajectory performance metrics 68
6.2. Test environment trajectory performance metrics. 68
6.3. Performance metrics from real flight experiment 1. 73
6.4. Used APF parameter values in simulation and for test flight experiments. 74
6.5. Performance metrics real flight experiment 2. 76

A.1. APF+ performance metrics simulation results Destroyed Garage. . . . 80
A.2. DWA performance metrics simulation results Destroyed Garage. . . . 80

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

List of Figures

1.1. Proposed collision avoidance system . . . 8
1.2. Example of a collision avoidance path in the presence of obstacles . . . 9

2.1. Quadcopter configurations. . . . 11
2.2. Illustrated collision and detection range of a UAV. . . . 12
2.3. Illustration of the APF working principle. . . . 19
2.4. Illustrative Bug2 algorithm avoidance path. . . . 23

3.1. Picture of the constructed quadcopter UAV platform. . . . 25
3.2. ArduPilot position and attitude cascade control . . . 27
3.3. Schematic UAV system overview with communication links. . . . 30

4.1. Relation between coordinate frames I_B and I_W. . . . 33
4.2. Different Coordinate Frames . . . 34
4.3. Parameterization of the rotation matrix using the roll-pitch-yaw angles. 35
4.4. Schematic representation of the quadcopter UAV . . . 37
4.5. Elementary UAV motion maneuvers. . . . 39
4.6. Relation between I_W and I_B for way point navigating. . . . 40

5.1. Overall schematic diagram of the obstacle avoidance system. . . . 42
5.2. Visualizations of the attractive potential function $U_a(\mathbf{p})$ and its gradient. 45
5.3. Visualizations of the repulsive potential function $U_r(\mathbf{p})$ and its gradient. 46
5.4. Visualization of the implemented functions . . . 48
5.5. Typical APF local minima situations. . . . 49
5.6. DWA velocity search space in 3D. . . . 53
5.7. Discrete sample directions in DWA and Bug2 method . . . 56

6.1. Gazebo UAV simulation model. . . . 58
6.2. Gazebo simulation environments from Destroyed Scenarios. . . . 58
6.3. Custom test environment with start and goal positions . . . 59
6.4. Generated APF avoidance paths start → goal . . . 61
6.5. Generated APF avoidance paths goal → start . . . 62
6.6. Generated DWA avoidance paths start → goal . . . 64
6.7. Generated DWA avoidance paths goal → start . . . 65
6.8. Generated trajectories in special obstacle configurations. . . . 67
6.9. Acceleration profiles for large obstacle configuration . . . 69
6.10. Extracted obstacle points from simulated and real sensors . . . 71
6.11. Obstacle extraction in simulation . . . 72
6.12. Experimental obstacle settings for the test flights . . . 73
6.13. APF avoidance path in real flight experiment 1 . . . 74
6.14. Complete APF avoidance path in real flight experiment 2 . . . 75

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

6.15. APF avoidance path in real flight experiment 2 76

A.1. Simulation environment Destroyed Garage 80
A.2. APF+ minimum obstacle distance profile and trajectories position 1 → 1. 81
A.3. APF+ minimum obstacle distance profile and trajectories position 2 → 2. 82
A.4. DWA minimum obstacle distance profile and trajectories 1 position → 1. 83
A.5. DWA minimum obstacle distance profile and trajectories position 2 → 2. 84
A.6. APF test flight minimum distance profile and trajectories. 85

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

1. Introduction

The work in this thesis is part of a larger crisis and disaster management project created in collaboration with the Austrian Institute of Technology (AIT). The overall goal of this project is to develop a fully autonomous indoor Unmanned Area Vehicle (UAV) to improve emergency response by designing a self-operating quadcopter that can assist first responders in critical surroundings. First responders should be provided with a 3D map of the environment to understand the operation area and be prepared for specific challenges. Since the robot is intended to operate in environments like damaged buildings, a ground-based robot would not be adequate for the task. The AIT is responsible for designing and developing the hardware platform and the algorithms enabling the drone to work autonomously. The end objective is to give first responders a reliable autonomous system that can inspect an indoor area and create a 3D map to support rescue and recovery efforts.

Given that the expected area of operation is indoors with no Global Navigation Satellite System (GNSS) available, a Simultaneous Location And Mapping (SLAM) algorithm is used to create a 3D map of the environment and estimate the UAV's position within the map. Based on the created map, an optimization-based path planner calculates further waypoints to maximize the exploration of the environment and increase the map quality. Since the top-mounted Light Detection And Ranging (LiDAR) sensor used for the mapping does not cover the whole surroundings of the UAV - especially the parts below the rotors - additional Time of Flight (ToF) sensors are added to capture the covered areas. The optimal path planner and the SLAM algorithm are calculated on an external ground station where connection losses to the UAV might occur, and they might fail to detect small or dynamic obstacles in the environment. This makes adding a low-level collision avoidance system running directly on the UAV is necessary.

This thesis focuses on designing and implementing a reliable collision avoidance system and the hardware setup of the UAV. It aims to achieve 3D collision avoidance for uncooperative indoor environments. The work does not cover the parts of the global exploration path planner, the SLAM algorithm, or any obstacle classification framework.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

1.1. Challenge

The avoidance system must be able to capture and react to the environment in real-time. Therefore, the algorithm must be lightweight in computational demand and offer a fast execution time. Local path planners make decisions based on the immediate surroundings and sensor inputs, ignoring the global context. This makes them prone to scenarios where they can be trapped in a local minimum situation or generate sub-optimal paths, even though a globally optimal path may exist beyond the local scope.

Since the developed robot operates in three-dimensional space, avoidance in a 2D plane is insufficient, making avoidance movements in 3D necessary. Navigating UAVs indoors presents unique challenges compared to outdoor environments due to the higher density of obstacles and limited space. In such settings, the UAVs operate at lower altitudes, making it necessary to avoid collisions not only with lateral obstacles but also with the floor and ceiling. When flying through narrow spaces, such as doorways, carefully determining the appropriate safety radius around the UAV is crucial. This involves setting velocity and acceleration constraints and selecting an appropriate safety distance to ensure a safe passage without compromising maneuverability.

From a hardware perspective, designing indoor UAVs requires a careful balance between maintaining a compact drone size and ensuring sufficient flight time. The drone must be small enough to navigate tight spaces and maintain adequate flight time. Therefore, the hardware components that support these requirements without adding excessive weight or bulk must be carefully selected.

1.2. Contribution

A hybrid fusion system combining computationally efficient algorithms is utilized to generate paths, thereby reducing the risk of the drone getting trapped in local minima. Two main algorithms, specifically the Artificial Potential Field (APF) and the Dynamic Window Approach (DWA), are implemented in 3D and evaluated against each other. Extending these algorithms to three dimensions significantly enhances the UAV's navigational capabilities, particularly in avoiding obstacles in the vertical axis. This advancement allows the UAV to maneuver over or under obstacles effectively rather than circumvent them laterally. The Bug2 algorithm is incorporated as a fallback to address local minima challenges. In the final implementation, the global path planner is tasked with resolving local minima issues.

To enable effective collision avoidance in confined indoor spaces, the APF method incorporates appropriate potential functions, as well as adaptive influence radius and speed constraints, allowing the UAV to navigate narrow passages without collisions. Path smoothness is achieved through adaptive step size, additional forces, and force limitations. The UAV is designed to account for all present obstacles, facilitating omnidirectional movements of the UAV. In the DWA, the scoring function and weights are carefully tuned, and an additional altitude criterion is introduced to enhance 3D avoidance capabilities.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

LiDAR and ToF sensors were used to provide comprehensive coverage of the UAV's surroundings, enabling effective and reliable obstacle data extraction. The algorithms are designed to run directly on the UAV's companion computer, enabling a reliable real-time collision avoidance system. The algorithms continuously run in the background, activating only if a potential collision is detected. Otherwise, the UAV follows the exploration waypoints set by the global planner. As illustrated in Figure 1.1, the communication between the global exploration path planner and the collision avoidance system is established via custom Robot Operating System (ROS) topics, signaling which planner is currently active and is publishing waypoints to the autopilot flight stack. For the evaluation in this thesis, a start and target position is set, and the reactive collision avoidance algorithm generates the complete path, exemplified in Figure 1.2. The implemented algorithms are thoroughly tested in simulation environments. The APF method is chosen for our project due to its superior performance, confirmed by experimental tests. It effectively integrates with the global planner, using the exploration waypoint as an attraction point and activating collision avoidance when repulsive forces exceed a certain threshold.

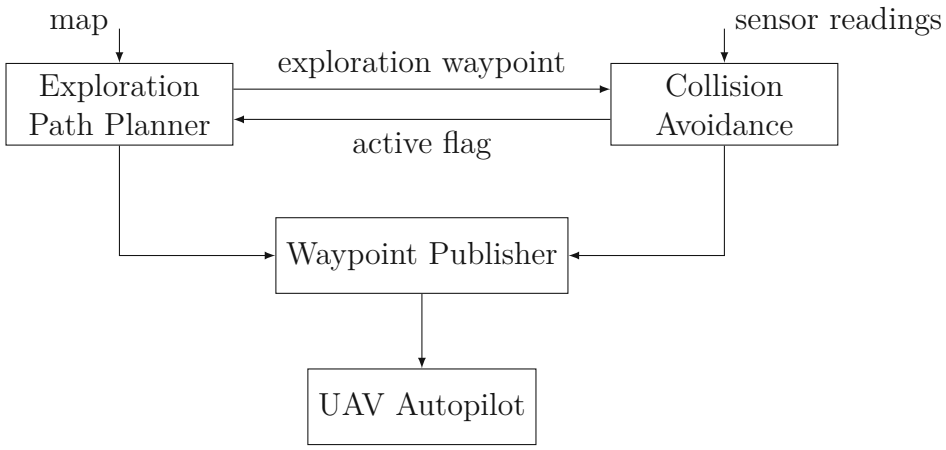


Figure 1.1.: Proposed collision avoidance system interacting with the exploration path planner and sending waypoints to the autopilot flight stack.

1.3. Results Preview

This work presents a platform suitable for autonomous indoor exploration in confined spaces. It is equipped with the minimal required sensors to provide a reliable and detailed map and ensure safe autonomous navigation. The proposed collision avoidance systems can navigate the UAV within confined spaces while ensuring a sufficient safety distance from obstacles by generating smooth avoidance trajectories. The platform can detect obstacles of any size and shape and avoid these in all three dimensions. The DWA and the APF methods have been effectively adapted for three-dimensional navigation. Adaptive parameters are introduced in our proposed improved Artificial Potential Field (APF+) method, allowing the platform to navigate tight spaces and efficiently and smoothly avoid obstacles.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

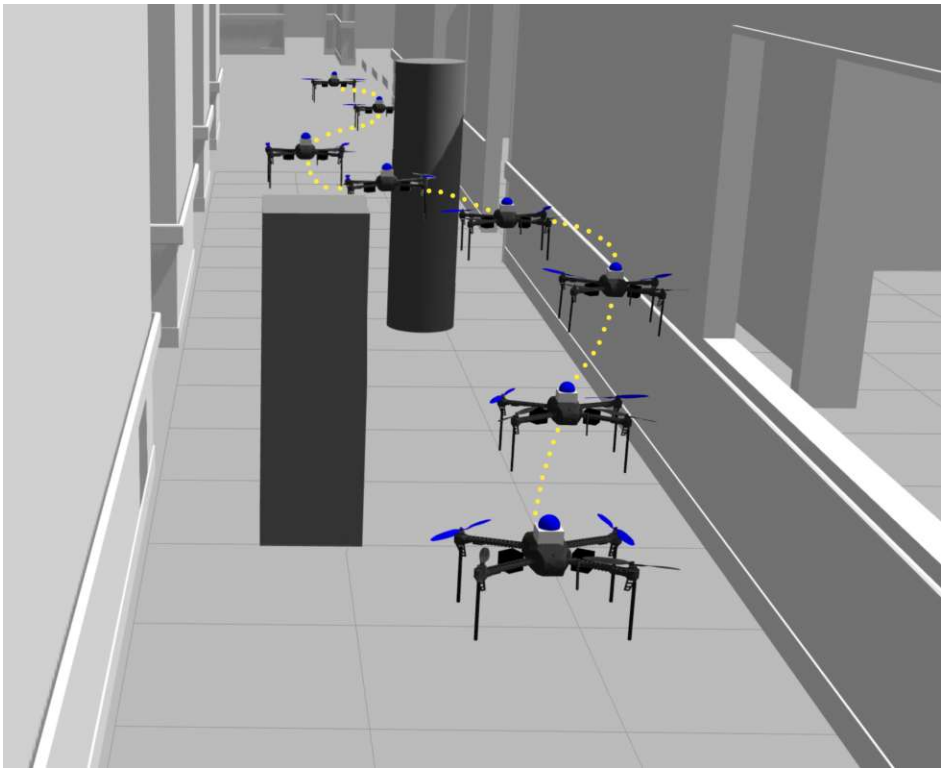


Figure 1.2.: Example of a collision avoidance path in the presence of obstacles, from a start to a target position.

Additionally, the hybrid design of the methods helps avoid situations where the system might get stuck in local minima. While the DWA also can maneuver through narrow passages, the APF+ method demonstrates superior performance in these scenarios, computational demand, and smoothness. The algorithms are extensively tested in simulation and hardware-in-the-loop tests. As implemented, the DWA method cannot operate in real-time on our companion computer. As a result, only the APF+ method is put through experimental tests. The outcomes of these tests align with the results obtained from simulations.

1.4. Thesis Outline

This thesis is organized as follows: Chapter 2 covers a literature review of UAV types, the range of perception sensors used for obstacle detection and mapping, and collision avoidance approaches, both global and reactive. Chapter 3 details the hardware setup of our custom-built UAV platform with all sensors, avionics, and communication links. Chapter 4 covers the mathematical fundamentals of UAV dynamics and flight control. Chapter 5 focuses on our implemented collision avoidance and obstacle detection methods. Finally, Chapter 6 presents simulation and experimental results to validate the effectiveness of the implemented algorithms. Appendix A contains detailed results, including trajectory plots and tables.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

2. Literature Review

Drones equipped with collision avoidance systems have recently gained attention in research. This chapter briefly overviews the broad range of UAV types, operation areas, and designs in section 2.1. Furthermore, in section 2.2, existing and commonly used perception sensors are listed, and conventional obstacle detection and extraction methods are described. Finally, section 2.3 introduces commonly used collision avoidance systems.

2.1. Unmanned Aerial Vehicles

An aircraft is transformed into a UAV, commonly known as a drone, when it acquires the capability for autonomous or remote control flight, operating without an onboard human pilot, and is controlled from a Ground Control Station (GCS). The communication between the GCS and UAVs is facilitated through satellite links, radio frequencies, or other wireless communication technologies [1]. They are equipped with avionics systems that allow them to carry out automated actions and respond to commands from the operator at the GCS. Typically, this is achieved by integrating accelerometer and gyroscope data with information from positioning systems. This enables the flight controller to understand not just its orientation but also its position. Positioning systems include tools like barometers, GNSS, or other systems such as SLAM methods. UAVs cover a broad range of systems, from small commercial drones used for entertainment or photography to large military drones, and the use of UAVs is becoming increasingly popular across various industries, including emergency response, transportation, and cartography [2]. Due to their ability to operate in environments that are not accessible for ground vehicles or might be hazardous to humans, they are also used for detecting and reporting different substances [3].

2.1.1. UAV Types

UAVs come in various types, each designed for specific operational processes, functionalities, and applications. According to Austin [1], the various types of UAVs in decreasing order of size include the following:

1. **High Altitude Long Endurance (HALE)** can fly at very high altitudes of around 15 km and have remarkable endurance, being able to fly for up to 24 hours. They are primarily employed for long-range military operations such as surveillance, reconnaissance, and aerial combat missions across continents.

2. **Medium Altitude Long Endurance (MALE)** operating at altitudes ranging between 5 to 15 km and are similar to HALE UAVs, but with shorter range capabilities. They are also primarily used for military applications.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.
TU Bibliothek Your knowledge hub WIEN

3. **Tactical Unmanned Area Vehicle (TUAV)s** are more simplistic and compact than HALE and MALE UAVs. They are also deployed for reconnaissance purposes with operational ranges between 100 to 300km.

4. **Mini Unmanned Area Vehicle (MUAV)s** weigh less than 20 kg and have a maximum range of 30 kilometers. Their applications are extended to specific civilian purposes.

5. **Micro Unmanned Area Vehicle (MAV)s** are the smallest and lightest UAVs with weights less than 5kg. They are characterized by slow flight speed and precise maneuverability, which make them particularly useful for various civilian applications due to their affordability, manageability, and ability to execute basic tasks. Because of their compact design and maneuverability, MAVs can also operate in indoor environments.

In addition to the classifications based on operational characteristics, UAVs also vary structurally:

1. **Multi-rotor UAVs**:
   - **Quadcopters** are the most common type of commercial drone equipped with four rotors. They come in two main configurations for rotor arrangement: Quad X and Quad +. Quad X resembles an "X" shape from above, while Quad + forms a "+" shape with one arm pointing forward, as shown in Figure 2.1. Quadcopters are widely used for photography, videography, mapping, and surveying.

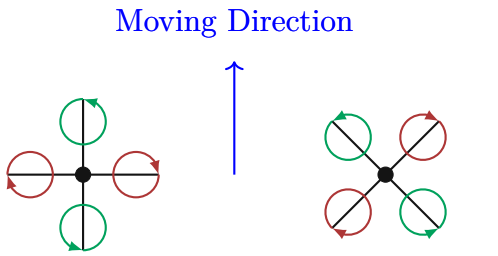


Figure 2.1.: Quadcopter configurations.

   - **Hexacopters** come with six rotors. These drones provide more lift and stability than quadcopters. They are suitable for carrying heavier payloads and high-quality cameras and are generally bigger than quadcopters.
   - **Octocopters** featuring eight rotors offer enhanced lift and stability and are often used for professional cinematography or specialized applications such as industrial inspections. They can carry high payloads and are generally not suited for indoor applications.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

2. **Fixed-wing** UAVs are airplane-like drones that cover long distances and are used for surveying, mapping, and agricultural applications. They have non-holonomic constraints, limited minimum speed, and are unsuited for indoor use.

3. **Single-rotor** resembling traditional helicopters. These UAVs have one rotor and are used for tasks requiring longer flight times and higher efficiency. They are suitable for aerial LiDAR applications, inspections, and agriculture but do not provide a high payload capacity.

Autonomous UAVs have gained attention for their outdoor applications. However, a growing area of interest lies in deploying autonomous indoor UAVs. MAVs (and sometimes MUAVs) multi-rotor UAVs stand out as the primary option for indoor applications due to their compact form factors, vertical takeoff and landing capability and suitability for navigating tight spaces. A Quad-X MAV is employed as the operation platform in the thesis under discussion.

2.2. Perception

To implement an obstacle avoidance algorithm, sensors to capture the environment are required. These sensors collect information about the UAV surroundings in real time and enable the extraction of obstacle data. They provide the distance and the relative position of obstacles, and choosing suitable sensors is the first step in designing an effective collision avoidance system. Obstacles should be detected by the sensors in the area between the collision range $d_{collision}$ and the detection range d_{detect} as illustrated in Figure 2.2. Ideally, d_{detect} should be as large as possible.

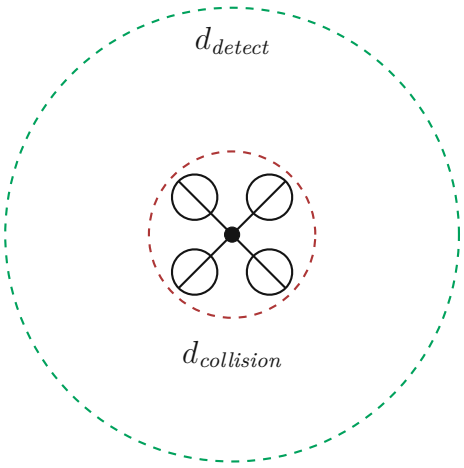


Figure 2.2.: Illustrated collision and detection range of a UAV.

In recent years, sensor technology and computer vision have seen remarkable advancements in performance, device weight, and device size, as well as in availability and affordability, facilitating more options for indoor environmental perception [4]. Indoor UAV platforms are strongly limited in size and payload, requiring small, lightweight, and power-efficient sensors. The following section analyzes and compares commonly used sensor technologies well-suited for environmental perception in indoor settings.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

2.2.1. Ultrasonic Sensors

Ultrasonic sensors are lightweight, compact, inexpensive, and can detect transparent and highly reflective materials. The basic principle of these sensors is a piezoelectric chip that emits ultrasonic waves when it is electrically stimulated. These waves bounce back after hitting an obstacle, and the sensor measures the time it takes to calculate the object's distance. These sensors usually operate within a frequency range of 20-50 kHz and have a measurement range of 20 cm up to 10 m with around 1% resolution in air.

Ultrasonic sensors are not affected by an object's transparency and offer good directional and penetration abilities. However, their measurements can be influenced by environmental conditions like temperature and humidity, and they may provide inaccurate data if the object reflects sound waves away from the sensor or absorbs them. They lack reflection beyond angles greater than $\pm 30°$ and interference or cross-talk with other sensors, and they have a blind spot for objects closer than 20 cm. The distance d_{obst} to a obstacle is calculated as follows:

$$d_{obst} = \frac{v \cdot t_m}{2} \tag{2.1}$$

where $v = 343 m/s$ is the speed of sound and t_m is the measured time of flight the wave. Several works use ultrasonic sensors for indoor UAV collision avoidance [5–8].

2.2.2. mmWave Radar

Radio Detection And Ranging (Radar) have been used in obstacle sensing for years [9]. In UAV applications, millimeter wave (mmWave) Radar is used [10–12]. This technology operates at frequencies between $f = 30 - 300$ GHz, corresponding to $\lambda = 1$–10 mm wavelengths. Short wavelengths enable sharper angular resolution, and the required antennas and hardware can be miniaturized. Radar works by emitting continuous or pulsed electromagnetic waves and then analyzing the reflections from various objects. The time it takes for the signal to return is used to calculate the object's distance. The distance d_{obst} to a obstacle is calculated as follows:

$$d_{obst} = \frac{c \cdot t_m}{2} \tag{2.2}$$

where $c = 299,792,458$ m/s is the speed of light and t_m is the measured time between emitting the wave and receiving an echo. The sensors can detect transparent and highly reflective objects, have a long distance range and a reasonable distance resolution, and can also effectively see through environmental challenges such as fog and smoke, a feature that other sensors often struggle with. On the downside, mmWave Radar does not offer enough data points to detect obstacle boundaries effectively, it suffers from limited vertical resolution, and in confined indoor environments, reflections of the waves (multipathing) can produce false readings. Algorithms exist to filter these multipathing effects, but they mostly make assumptions about flat surfaces and require high computational efforts [13].

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

2.2.3. LiDAR Sensors

LiDAR are light-based sensors and are essential tools for precise distance measurement and environmental mapping and are often used for UAV collision avoidance [14–16]. They use a laser or Light-Emitting Diode (LED) to emit light and a photodetector to capture the returning light. Direct ToF sensors emit light pulses and measure the time of this light to return to calculate d_{obst}:

$$d_{obst} = \frac{c \cdot t_m}{2} \tag{2.3}$$

where c is the speed of light and t_m is the time of flight, whereas indirect ToF sensors emit a constant stream of modulated light and evaluate the phase shift in the returning light to calculate the object's distance d_{obst}:

$$d_{obst} = \frac{c \cdot \varphi}{4\pi \cdot f} \tag{2.4}$$

where φ is the phase shift, and f is the modulation frequency. Laser-based sensors reflect at angles up to $60 - 80°$. This performance is influenced by the surface properties of the object being scanned. The sensors are also relatively robust against crosstalk, interfering only when directly facing another sensor with the same wavelength. A blind spot exists within 10 cm of the sensor, which could be a limiting factor in extremely close-range scenarios, and it falls short of identifying transparent objects like clear glass.

LiDAR is a very commonly used sensor technology for indoor distance measurement and high-precision 3D mapping, and the advancements in recent years have been significant. Modern LiDAR units are increasingly compact, lightweight, and affordable, making them more accessible for various UAV applications. LiDARs provide high-accuracy measurements, long-distance ranging, rapid response times, a wide Field of View (FOV), and can detect small objects due to the use of short-wavelength laser beams. In addition to the powerful pulsed laser emitter and photodetector, a LiDAR system commonly includes a scanning mechanism, often a rotating mirror, that controls the laser across various angles. This enables the system to build a comprehensive and highly detailed 2D or 3D map of the surroundings. There also exist Solid State LiDARs with no rotating parts in the sensor, allowing the sensors to have a more compact design. They use optical components, such as lenses and filters, to help focus the emitted light and filter out noise or unwanted reflections.

ToF Sensors

ToF sensors typically use less powerful light sources like infrared LEDs to emit beams into their FOV and capture an entire frame simultaneously. Due to the lower light intensity, they are less accurate than LiDAR systems, are generally less robust, and offer lower detail and range. On the other hand, they have lower power consumption, are smaller, and are more affordable. They are primarily used for distance ranging and are not designed for mapping purposes.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

2.2.4. Vision Based Sensors

Vision-based sensors are a common choice for indoor collision avoidance due to their versatility and relatively low cost, and there is high research on this topic [17]. Unlike other sensors, they capture the environment through lenses and photoelectric sensors and can acquire information like color, shape, and texture. They can provide high-resolution real-time data and resist external interference, allowing them to operate effectively even in complex, dynamically changing environments. Their passive nature reduces the chances of the sensor system being identified or detected. The limitations of vision-based sensors are computational overhead, especially for real-time applications. Lighting conditions can significantly affect performance, and the sensors struggle with limited range, transparent materials, and highly reflective surfaces.

Monocular Cameras

Their compact design, flexible deployment, lightweight, and low price make them well-suited for UAV applications where payload and budget constraints are critical factors [18–20]. A monocular camera only consists of one camera and is easy to calibrate. It produces a flat, two-dimensional image, which complicates the task of obstacle detection and avoidance since there is no direct depth information. Advanced (artificial intelligence) algorithms can analyze two-dimensional images to extrapolate spatial relationships. Machine learning models can be trained to recognize patterns or features in the images to effectively transform the simple 2D image into a rich source of environmental information.

Fisheye cameras are specialized monocular cameras, offering a broad field of view that makes them particularly useful for complex, constrained environments like narrow corridors or crowded spaces. Their wide-angle lens captures a broader section of the surrounding area in a single frame, providing a more comprehensive view.

Stereo Cameras

Stereo, or binocular, cameras are also very compact, lightweight, and affordable and often used for obstacle detection in indoor UAVs [6, 21, 22]. They emulate the human visual system and are based on the triangulation of rays from multiple viewpoints to acquire depth information directly. With two offset lenses, these cameras capture two slightly different images of a scene, enabling the calculation of object depth and size based on principles of parallax and stereo matching. Once the camera parameters are well-defined, pixels in one image can correspond to pixels in the other image, enabling the computation of the depth at specific points and the reconstruction of the object's 3D position within the environment without the need to classify the obstacle. The correspondence problem, finding the same point on the left and the right image, limits the speed and accuracy. Finding the right balance between them is crucial: higher speed might compromise the depth map accuracy, whereas more accurate calculations could slow down real-time detection. When appropriately optimized, stereo cameras can offer a robust and highly effective option for 3D environmental perception and navigation.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

RGB-D Cameras

Unlike stereo cameras, a Color and Depth Camera (RGB-D) captures depth information directly for each pixel. They combine visible light and infrared sensors to generate a standard image and depth map simultaneously, providing a colored point cloud for obstacle detection. They are also used for indoor UAV applications [23, 24]. There are primarily two techniques for depth acquisition in RGB-D cameras: Structured Light and ToF. Structured light uses projected patterns like Moiré patterns, coded light, or dot patterns and performs well in challenging lighting conditions and complex textures. Due to their complex interference patterns, moiré patterns are helpful for high-precision depth mapping. Coded light improves the speed and robustness of the depth acquisition, while dot patterns offer a more straightforward yet practical approach. Structured light is less sensitive to variations in ambient lighting, making it robust for low-light scenarios. On the other hand, the ToF method is more accurate and suited for environments requiring long-distance measurements but comes with a trade-off of lower image resolution and depth map quality. Compared to stereo cameras, RGB-D cameras have a high power consumption and are generally more expensive.

2.2.5. Sensors comparison

The sensor selection for obstacle detection is a critical design decision for a robust indoor collision avoidance system. As summarized in Table 2.1, each sensor type has unique strengths and weaknesses. The detection range is grouped into Short $\leq$ 10m, Medium $\leq$ 50m, and Long $\geq$ 50m.

Monocular and stereo cameras are lightweight and low in power consumption but require high computational resources for accurate obstacle detection, tend to have limited robustness, and require a sufficient light source. RGB-D cameras offer better accuracy and work well under medium lighting conditions, but their higher power consumption and size and weight are limiting factors. Radar sensors show generally good characteristics but suffer from multipathing and a low angular resolution. Ultrasonic sensors are advantageous due to their low weight, power requirement, and ability to detect transparent objects. However, they have a narrow FOV and short range, making them less reliable for comprehensive obstacle detection. LiDAR shows outstanding accuracy, robustness, and resolution and excellent characteristics in FOV and range but requires more power and is not very lightweight and small. ToF has a narrower FOV and lower range than LiDAR, but the sensors are smaller, more lightweight, and require less power. Often, combinations of different sensor technologies are used to overcome the limitations of single systems [8, 16, 20].

Given these trade-offs, LiDAR and ToF sensors are the choices for the proposed indoor UAV platform where reliability, accuracy, and real-time processing are essential. ToF offers high accuracy and robustness, essential for reliable obstacle detection and collision avoidance in confined spaces. LiDARs' wide FOV and long detection range make it ideal for mapping a large area around the UAV and providing reliable data for the A-SLAM.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

SensorType	Accuracy	Robustness	FOV	Range	Resolution	Weight	Power	Processing Requirement
Mono Camera	Medium	Low	Medium	Medium	Medium	Low	Low	High
Stereo Camera	Medium	Medium	Medium	Medium	Medium	Low	Low	High
RGB-D Cameras	High	Medium	Medium	Medium	High	Medium	Medium	Medium
Radar mmWave	Medium	High	Wide	Long	Medium	Low	Medium	High
Ultrasonic	Low	Medium	Narrow	Short	Low	Low	Low	Low
LiDAR	High	High	Wide	Long	High	High	High	Low
ToF	High	High	Medium	Medium	High	Medium	Medium	Low

Table 2.1.: Summarized sensor comparison for obstacle detection.

2.3. Collision Avoidance Algorithms

Collision avoidance algorithms are implemented on various mobile robot platforms and are a fundamental problem in robot navigation. Initially, avoidance algorithms were developed for robots operating in 2D spaces, but with advancements in robotics, computer technology, and hardware, these algorithms were gradually expanded to 3D environments. Collision avoidance systems are essential for UAVs, not only for fully autonomous but also for semi-autonomous and manually piloted ones, to minimize the risks of accidents that could result from human error or equipment malfunctions. Unlike ground vehicles, UAVs often crash and cannot continue flying after a collision, posing a risk of damage to the platform and the environment. Therefore, implementing effective collision avoidance is, especially indoors, a crucial safety measure for UAV operations, and although the basic research in the field of obstacle avoidance already started in the 1990s, numerous researchers have shown interest in the UAV obstacle avoidance problem in recent years [25]. Still, there is surprisingly low research in 3D implementations since most assume fixed altitudes. Additionally, they are mostly designed for outdoor spaces with sufficient space to maneuver, and many are only tested in simulation.

Collision avoidance algorithms can generally be classified into two main categories: reactive (or local) and deliberative (or global) methods. Reactive algorithms are designed to operate in real-time, relying on actual local sensor data and responding immediately to environmental changes. These algorithms are not designed to provide a full path to a target position but to determine the best action for the current sensor input and not rely on pre-computed maps. They are designed to require low computational overhead, making them suitable for small robotic platforms with limited processing capabilities. Limitations of reactive algorithms include the generation of sub-optimal paths and the tendency to get trapped in local minima. Deliberative methods, on the other hand, are based on pre-computed or dynamically constructed and updated maps of the environment to guide the robot collision-free to the target position, and these approaches can often find near-optimal or even optimal paths and are often related to *pathfinder* instead of classical obstacle avoidance algorithms. Since these methods are based on complete maps, they are computationally intensive, making them less suitable for platforms with restricted computational resources and power, and they may not be well-suited for dynamic environments with rapidly changing obstacles.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

2.3.1. Global Approaches

Global methods provide a complete (sub-)optimal path from the start position $\mathbf{p}_{start}$ to one or more target positions $\mathbf{p}_{goal_i}$ relying on maps with obstacle information. They are based on graph search, representing the environment using rasterization and a search algorithm to create a path that avoids obstacles. Prominent search algorithms for global path planning are the Dijkstra [26], A* [27], D* [28], and Rapidly exploring Random Tree (RRT) [29].

The Dijkstra algorithm, introduced by Edsger Wybe Dijkstra in 1956, is a fundamental search method used to find the shortest path between nodes in a graph efficiently and accurately. The algorithm systematically selects the shortest path from a starting point to all other points in the graph, updating the shortest known distance to each point as it progresses, determining the best path [30].

The A* algorithm is an extension and optimization of the Dijkstra algorithm, designed to find the shortest path in a graph more effectively. A* combines the strengths of Dijkstra's algorithm with a heuristic function to improve performance and accuracy, particularly in complex environments. It calculates the most cost-effective path from a start node to a target node, considering the actual cost to reach node $f(\mathbf{p})$ and an estimated cost from that node to the goal $h(\mathbf{p})$. The D* (Dynamic A*) algorithm addresses the limitations of the traditional A* algorithm in dynamic environments. Unlike A*, D* dynamically updates the path in response to environmental changes. This allows for real-time adaptation to obstacles or terrain changes, enhancing the efficiency and safety of navigation in unpredictable settings. A* and its adaptions are the most widely used path-planning algorithms [11, 17, 31] for UAVs.

The RRT algorithm, proposed by LaValle in 1998, is designed to efficiently navigate through high-dimensional dynamic spaces, operating by incrementally constructing a tree, starting from a root node and expanding towards unexplored areas of the search space. It does this by randomly sampling points in the space and adding them to the tree to rapidly cover the search area. The RRT approach is often employed for autonomous path planning with obstacle avoidance in UAVs [32, 33].

Many other approaches, such as Probabilistic Roadmap (PRM) and algorithms based on machine learning, including neural networks, reinforcement learning, and deep reinforcement learning, are utilized in UAV collision avoidance and path planning. In [34], a comprehensive review of global approaches and in [33] implemented approaches in UAVs are provided in detail.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

2.3.2. Reactive Approaches

Global approaches use a complete map to find the best path toward a goal. In contrast, reactive approaches are designed to avoid dynamic obstacles from sensor readings in unknown environments. Many different approaches for reactive collision avoidance that process obstacle information from onboard sensors and subsequently generate new waypoints in real time have been proposed. Since this work focuses on reactive obstacle avoidance in unknown environments, a selection of the most common approaches is described in more detail:

Artificial Potential Field

The Artificial Potential Field is a robust and well-established method for robot navigation and obstacle avoidance in unknown environments and was initially proposed by Khatib [35] in the 1980s as a real-time obstacle avoidance approach and has become one of the fundamental techniques in the field of robotics and autonomous systems [36]. It is important to mention that the APF can also be used as a global pathfinder if all obstacles are known beforehand. The APF method has been deployed for various purposes ranging from indoor robots and autonomous vehicles to drones and space robotics [37]. It is classified under the category of force field methods. The fundamental principle behind this is the representation of the robot and the obstacles in its environment as charged particles. The goal is considered to be a positive charge, while the robot and the obstacles are considered to be negative charges. The robot then is influenced by the virtual forces exerted by these charges, where the goal is considered to exert an attractive force while the obstacles exert repulsive forces, and the superposition of these forces then defines the motion of the robot, as illustrated in Figure 2.3.

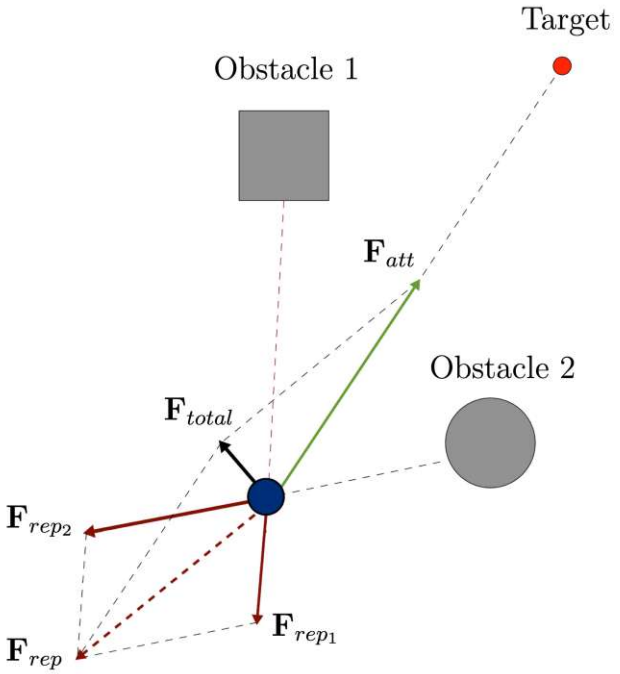


Figure 2.3.: Illustration of the APF working principle.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

The advantages of the APF method are its effectiveness and computational efficiency, making it particularly suitable for real-time applications. However, the method has some well-known limitations and drawbacks. For instance, the robot may be trapped in local minima in specific configurations, rendering it unable to reach the target [38]. Additionally, if an obstacle is close to the destination, the repelling force exerted by the obstacle may prevent reaching the final destination. Another issue is the choice of appropriate potential functions and parameters, which is often non-trivial and requires careful tuning [39]. Researchers proposed several extensions to the plain APF method. The APF method is combined with other simple algorithms in some approaches. Wang et al. [40] switch between APF and the Wall-Following Method (WFM) to create a computational efficiency, real-time collision avoidance method. Lin et al. [41] presented a method for ground vehicles combined with Bug2 proposed for dynamic obstacle avoidance, and Zhao et al. [42] proposed an improved method and solved the problems of path oscillations and local minimums. Another extension is the Harmonic Potential Field method, which combines the plain APF with harmonic functions to reduce the probability of getting trapped in local minima [43]. Mac et al. [44] implemented the APF for indoor environments operating in 2D, and Lindqvist et al. [45] presented an adaptive APF for safe and stable local navigation in critical scenarios using an adaptive influence radius of the obstacles. Some other approaches involve integrating the APF method with Artificial Intelligence (AI) techniques, such as fuzzy logic or genetic algorithms, to optimize the potential fields and robot's path [46]. Hu et.al [47] combined the APF with the DWA to resolve the local minima problem. Recently, interest has grown in combining APF methods with machine learning techniques, especially deep learning. For example, deep reinforcement learning has been used to learn the parameters and functions of the potential field in complex environments [48, 49]. This work proposes a reliable 3D implementation of the APF, addressing these limitations with various modifications and adaptions.

Virtual Force Field

The Virtual Force Field (VFF) method [50] builds upon the potential field method by incorporating a Certainty Grid to better handle the uncertainty in sensor data. The algorithm addresses the challenges of handling sensor uncertainties and modeling errors, a common issue in robotic applications. This approach is particularly well-suited for dealing with inaccurate sensor data (e.g., sonar) and allowing the fusion of multiple sensor readings. It is designed to enable a robot to navigate through an environment with obstacles using a combination of attractive and repulsive forces, as in the APF method. The Certainty Grid records the likelihood of obstacles in each grid cell overlaying the robot's environment. When the robot's sensors detect something, the corresponding cell value increases. Only when a cell's value exceeds a certain threshold does the system consider an obstacle present, which helps filter out false positives from sensor noise. The sum of these forces results in a resultant vector that influences the robot's movement, steering it toward the target while avoiding collisions with obstacles. This method allows for a more robust and reliable navigation than the basic potential field method. Note that the use of a highly accurate and reliable LiDAR sensor does not necessarily require this handling of uncertainties.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Vector Field Histogram

Based on the idea of the VFF approach, the Vector Field Histogram (VFH) was proposed by Borenstein and Koren in 1991 [38], which further refines obstacle avoidance by creating a polar histogram of obstacles around the robot. The VFH uses this histogram to choose the best direction to move towards the target while avoiding obstacles, thus enabling the robot to navigate in tighter spaces and with faster response times. The algorithm offers computational efficiency, robustness, and insensitivity to misreadings, making it particularly effective in cluttered obstacle courses.

The Cartesian histogram grid is the core element of VFH. It is a statistical representation of the robot's environment using its range sensors to create a real-time grid of the surrounding area. This grid maps the environment around the robot, providing a clear and detailed representation of potential obstacles in its path. Derived from the Cartesian grid, the polar histogram is a one-dimensional representation focusing on the robot's immediate surroundings. It simplifies the data from the Cartesian grid into a more manageable format for real-time processing. This histogram is crucial for understanding the spatial distribution of obstacles around the robot. From this histogram, *candidate valleys* can be identified. These are sectors within the polar histogram where the obstacle density falls below a predefined threshold, with the lower density in these sectors indicating that obstacles are located further away. These low-density areas are potential paths for the robot to travel through safely.

Over the years, the VFH algorithm has evolved. In 1998, the VFH+ [51] and in 2000, the VFH* [52] were introduced by Ulrich and Borenstein. The VFH+ improved the original implementation by accounting for the robot's size, using a binary polar histogram for more stable steering decisions, and considering the robot's motion model to avoid collisions. The improved method also includes guidelines for navigating valleys using a heuristic cost function to determine the optimal direction. This has increased trajectory smoothness and better handling of different robot sizes without manual parameter adjustments. The VFH* stands out for using the A* search algorithm, adding a layer of predictive capability to the steering commands. This has effectively addressed the limitations of local path planning algorithms, especially in scenarios where a global optimum is challenging to achieve. A drawback of the VFH is that it fails to consider vehicle motion dynamics, posing collision risks for non-holonomic robots. VFH+ improves this slightly but still assumes a constant steering radius, which is unsuitable for all systems like differential drives. In [53], the authors propose 3DVFH+, a novel approach that extends the 2D VFH+ obstacle avoidance algorithm to 3D. Their method utilizes the octomap framework to represent the 3D environment accurately, and from this, a 2D Polar Histogram is generated. In 2018, Baumann [54] combined the 3DVFH+ with the VFH* on a quadcopter UAV platform, avoiding obstacles in all three dimensions.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Optical Flow

Optical flow is currently one of the primary areas of research in UAV collision avoidance [55–57]. This technique is based on how certain biological systems, like birds and insects, navigate their environment and avoid collisions by perceiving changes in the visual patterns caused by motion.

In robotics, a simple low-resolution camera is used to trace pixel-wise motion in relation to the robot's movement. This is achieved by observing the changes in brightness and darkness of objects, surfaces, and edges in a visual scene between sequential image frames. The fundamental principle behind optical flow in UAVs is that the motion of the mounted camera on the UAV generates changes in the captured images. These changes, often represented as velocity vectors of moving pixels, are analyzed to discern the UAV's own motion and the relative position of objects in their environment. When normalized by the UAV's speed, the magnitude of these pixel velocities is inversely proportional to the distance of objects. This means that objects closer to the UAV will appear to move faster in the visual field, providing distance information for collision avoidance.

UAVs equipped with optical flow-based systems typically use algorithms to steer away from areas with higher optical flow values, indicating closer obstacles [55, 56, 58]. However, implementing optical flow for collision avoidance presents a challenge in ensuring the reliability of the optical flow data. To accurately estimate pixel velocities, the UAV must move perpendicular to the camera's orientation. This requirement can be limiting when the UAV navigates in constrained or cluttered spaces. Additionally, optical flow does not work in dark environments or conditions with low light and may not perform well in low-contrast environments. Despite these challenges, the use of optical flow in UAV collision avoidance has shown considerable promise, offering a relatively low-cost and efficient approach for autonomous navigation in complex environments.

Optical flow algorithms have undergone significant advancements, ranging from classical methodologies to modern deep learning-based techniques [59, 60]. Conventional approaches like Lucas-Kanade [61] and Horn-Schunck [62] rely on the theory that the intensity levels remain constant between serial frames. These methods often involve manually-engineered features and may struggle with complex situations. In contrast, modern techniques use the power of neural networks to learn relevant features for motion tracking automatically. State-of-the-art solutions such as FlowNet [63] and PWC-Net [64] have shown significant performance gains, particularly in complicated or unclear environments. In [65] a comprehensive study of optical flow implemented in UAVs can be found.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Bug Algorithms

The Bug algorithms are a set of four algorithms (Bug0, Bug1, Bug2, Tangent-Bug) used in robotics for collision avoidance. The Bug2 algorithm is one of the Bug Family Algorithm (BFA) and was first presented by Lumelsky and Skewis in 1989 [66]. It is a straightforward and effective method for robotic path planning in environments with unknown obstacles and an extension of Bug0 and Bug1. The algorithm directs the UAV to move directly in a straight line ($\mathbf{m}_{\text{line}}$) toward its destination until it encounters an obstacle. At this *hit point*, the UAV circumnavigates the obstacle until it intersects the m-line again. This point is known as the *exit point*. The UAV then leaves the obstacle and resumes its destination-directed motion along the $\mathbf{m}_{\text{line}}$.

The Bug algorithms represent a trade-off between computational complexity and path efficiency. Although they do not always yield the shortest possible path, they are computationally less intensive than other more sophisticated path-planning algorithms, making them a viable option for real-time applications. Research on the BFA has led to the development of many variations such as the I-Bug Algorithm [67], Point Bug Algorithm [68], Insert Bug Algorithm [69], Vis Bug Algorithm [70], and Dist Bug Algorithm [71]. Also, the WFM [72] has been developed based on the ideas of the BFA, with the consideration of proper constraints such as the size of the robot, and has become popular in many robotics applications [73, 74]. The primary limitations of the algorithms are that they can lead in endless loops or repetitive trajectories in environments with closely spaced or complex obstacles, particularly if the robot always follows the right or left direction of the wall tangent when an obstacle is detected. Figure 2.4 demonstrates the $\mathbf{m}_{\text{line}}$ and the possible avoidance paths from $\mathbf{q}_{start}$ to $\mathbf{q}_{goal}$.

The Bug2 algorithm has proven effective for obstacle avoidance in an environment with relatively simple obstacles. However, it may not yield the most efficient path in a complex environment with many obstacles or maze-like configurations.

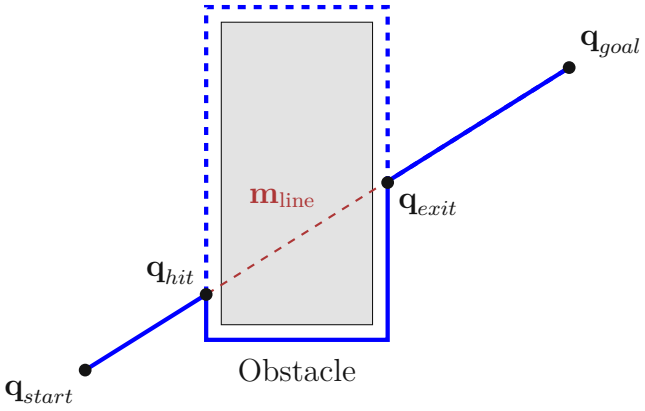


Figure 2.4.: Illustrative Bug2 algorithm avoidance path.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Dynamic Window Approach

The DWA is another prominent method for local obstacle avoidance and was introduced by Fox et al. [75] in 1997. It is designed for ground robots operating in 2D space, and applying the method to UAVs increases the complexity due to more DOF. The fundamental principle of the DWA algorithm includes sampling a variety of control parameters and predicting their effect on the robot's current movement in simulation, assuming these controls remain unchanged for a short period. It considers the constraints of the robot's speed and acceleration while only considering values that ensure safe positions at each sample. Then, the most suitable trajectory is chosen based on a defined scoring function. The scoring function usually considers the goal, speed, and obstacle factors. These simulated trajectories are then measured against the scoring function, and the controls corresponding to the best trajectory are used to guide the robot's next movement. This entire cycle is continually carried out while the robot is moving.

The DWA has seen significant success and is implemented as a standard local planning algorithm used in the highly utilized ROS 2D navigation package *dwa_local_planner* [76]. However, the algorithm has several shortcomings, including challenges in designing the scoring function, an over-cautious speed window, and a tendency for the UAV to get stuck in a local optimum [77, 78]. In 1999, Brock and Khatib [79] improved the method by using A* instead of the heading function to solve local minima problems. Moon et al. [80] invented a hybrid DWA combining holonomic and non-holonomic dynamics while considering the turn dynamics of a fixed-wing UAV in 2D. Bai et al. [31] present an algorithm that combines enhanced A* and DWA methods to optimize global paths for UAVs, addressing both safety and speed constraints. Experimental results show that the algorithm not only shortens path length and planning time but also improves path smoothness and obstacle avoidance safety. Feng et al. [81] presents a fusion algorithm that integrates 3D VFH+ and an improved DWA, incorporating a new rotation cost evaluation function to enhance cooperative obstacle avoidance for multi-UAV formation flight in unknown settings. Simulation outcomes show that this advanced algorithm significantly optimizes obstacle avoidance, including rotation costs, compared to traditional DWA methods. Tusseyeva [82] adapts the DWA to guide non-holonomic autonomous unmanned underwater vehicles in 3D marine environments. Finally, Xu et al. [83] presented a 3D obstacle avoidance algorithm based on the DWA, including fuzzy logic for adaptive weights to escape local minima situations.

Other Methods

A range of other reactive collision avoidance algorithms exists, including approaches based on machine learning. A detailed review of collision avoidance approaches implemented in UAVs can be found in [84] and [17].
This work mainly focuses on implementing and deploying the APF, DWA, and Bug2. These algorithms were chosen for their relative computational simplicity, making them suitable for real-time operations on a companion computer. Moreover, they have been proven effective in unknown and dynamically changing environments, thus aligning with the operational requirements of our project.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

3. UAV Setup

This chapter describes the hardware setup, communication links, and avionics of the custom indoor UAV platform in detail. In Figure 3.1, the constructed platform with all its equipped components is pictured.



Figure 3.1.: Picture of the constructed quadcopter UAV platform.

3.1. Hardware Equipment

For UAVs operating outdoors, GNSS is commonly used for localization. However, accurate positioning and safe navigation become problematic when UAVs are operated in constrained indoor spaces or caves. This increases the interest in equipping the UAV with sensors to effectively detect and map the surroundings to enable secure autonomous indoor navigation capabilities [85].

3.1.1. Frame

For this project, a development frame is set up. A range of Ready To Fly (RTF) platforms exists, but a custom-built platform is required for our use case. RTF platforms often already provide the integration of various sensors but only spare options to equip the platform with additional sensors. The employed Hexsoon EDU 450 [1] development

[1] https://ardupilot.org/copter/docs/reference-frames-hexsoon-edu450.html

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.
TU Bibliothek Your knowledge hub WIEN

frame includes the mounting platform with arms and stands as well as motors, ESCs and blades which seamlessly integrate and are perfectly compatible with each other. The platform is small enough for indoor use in confined spaces and provides a sufficient maximal payload to integrate all the needed sensors and avionics. There is a direct trade-off between platform size, flight time, and payload. In Table 3.1, the selected equipment with its power consumption and weight is listed. The specified payload of the platform is 960g and exactly the sum of all the used equipment, enabling the UAV to fly for approximately 5-10 minutes.

3.1.2. IMU

Inertial Measurement Units (IMU) are essential for quadcopters' stable flight and position control. A standard Micro Electric Mechanical System (MEMS) IMU is equipped with a three-axis accelerometer, a three-axis gyroscope, and often a three-axis magnetometer [86]. These components measure linear acceleration, angular velocity, and magnetic fields, providing the drone's 3D orientation and relative movement information. Given the inherent limitations like the gyroscope's tendency for long-term drift and the accelerometer's sensitivity to noise, sensor fusion algorithms such as Extended Kalman Filter (EKF) are employed. These algorithms effectively combine the data from these diverse sensors to offer a more accurate and reliable estimation of the quadcopter's pose in 3D space. MEMS gyroscopes operate based on the Coriolis effect. According to this principle, a mass m moving at a velocity $\mathbf{v}$ in a frame rotating with an angular velocity $\boldsymbol{\omega}$ is subject to a force $\mathbf{F}$ described by $\mathbf{F} = -2m(\boldsymbol{\omega} \times \mathbf{v})$. Vibrating masses are used in MEMS gyroscopes, which move along a particular axis. When subjected to rotation, an additional perpendicular vibration is induced, displacing the mass from its initial position. This displacement is typically measured through capacitance variations.

3.1.3. Flight Controller

Flight controllers are microcontroller-based boards equipped with IMUs, providing interfaces for connecting external devices and motor control. Prominent controllers are Pixhawk [87], CubePilot [2] and DJI Naza [3]. UAVs naturally struggle with aerodynamic stability and require an electronic flight controller for proper aerodynamic balance. The software embedded in a flight controller typically compromises advanced algorithms, including PID control loops for 3D attitude adjustment, velocity control, and EKF for integrating sensor data. For custom build drones, there mainly exist two autopilot platforms, namely ArduPilot [4] and PX4 [88]. Both platforms have an academic background: ArduPilot was invented as a student project in 2007 at the University of California, and PX4 was initiated by Lorenz Meier in 2011 at ETH Zurich's Computer Vision and Geometry Group. They are open-source and can usually be installed directly onto most flight controller boards.

[2] https://docs.cubepilot.org
[3] https://www.dji.com
[4] https://ardupilot.org

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

In this work, the Cube Pilot Orange+ [5] flight controller and the ArduPilot platform is utilized. The flight controller offers additional layers of reliability by featuring triple-redundant IMUs. This IMUs encompass accelerometers and gyroscopes and are mechanically isolated to reduce vibration-induced errors. Additionally, they are temperature-controlled using onboard heating elements to maintain their optimal working conditions. This advanced flight controller also incorporates dual barometers and a single magnetometer for enhanced redundancy. In Figure 3.2, the cascaded copter control architecture of the ArduPilot autopilot is shown. It consists of a position and attitude control containing square-root controller (P), PID, and Feed-Forward (FF) control. The ArduPilot software supports manual flight via remote and an autonomous *GUIDED* mode. In *GUIDED* mode, the UAV autonomously navigates to set positions using the coordinates $\mathbf{p}_x$, $\mathbf{p}_y$, $\mathbf{p}_z$, and heading ψ (refer to Equation 4.28).

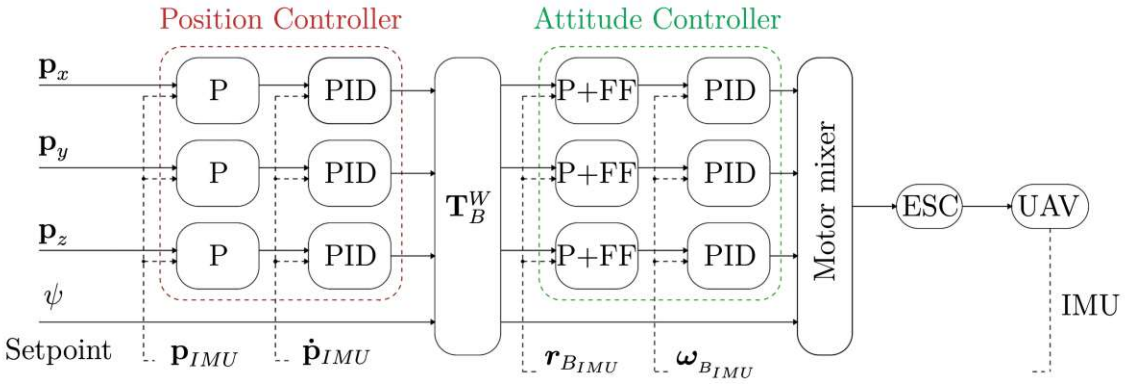


Figure 3.2.: Schematic overview of the position and attitude cascade control in the ArduPilot flight controller.

3.1.4. Optical Flow

An optical flow sensor is a simple low-resolution camera that traces pixel-wise motion in relation to a vehicle's movement as detailed in 2.3.2. Optical flow is used for collision avoidance and tracking the motion of a UAV. The general procedure involves two core steps: capturing a scaled-down image and analyzing consecutive frames to estimate the speed of pixel movement. This data is subsequently transformed to estimate the UAVs velocity and orientation. In our platform, a HereFlow [6] optical flow sensor is integrated into the UAV's flight control system to achieve enhanced navigational capabilities through data fusion. The sensor plays a dual role in estimating the UAV's velocity and maintaining stable altitude during hover conditions. The optical flow data are combined with inputs from the flight controller to generate a more robust and accurate model of the UAV's state, thereby ensuring more stable and reliable performance while hovering and moving.

[5] https://docs.cubepilot.org/user-guides/autopilot/the-cube-module-overview
[6] https://www.cubepilot.com/hereflow

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

3.1.5. Companion Computer

The complexity of autonomous UAVs requires robust and intelligent onboard computing beyond traditional flight controllers' capacity. Companion computers achieve these goals, offering real-time data processing from sensors and other peripherals and enabling the implementation of advanced algorithms such as computer vision and path planning. Companion computers also provide a platform for advanced sensor integration, communication, and integration between different hardware components and software applications.

In this work, the Raspberry Pi 4 Model B [7] is utilized as the companion computer for the UAV platform. It operates Linux, specifically Ubuntu 20.04, offering a stable and high-performance environment that supports ROS and MAVROS, enabling seamless sensor integration and communication to the flight controller. The Raspberry Pi 4 has several serial ports, four USB ports, and an Ethernet port. The serial ports are utilized for the CygLiDARs, allowing for streamlined data acquisition from these sensors and the MAVROS communication with the flight controller, and the Ethernet port is employed to connect with the Livox sensor. The Raspberry Pi 4's hardware, with 8GB SDRAM and up to 2 TB microSD support, ensures sufficient memory and storage capacity to handle data-intensive tasks. Energy efficiency and its lightweight design are significant considerations.

3.1.6. Perception Sensors

An intricate sensor suite is employed to capture and interpret the UAV's surroundings to facilitate autonomous navigation within indoor environments. The sensor technologies are comprehensively described in section 2.2.

Enabling SLAM, a sensor that offers both a wide FOV and high resolution, is essential. Additional considerations are the weight and power consumption of the sensor to ensure optimal flight performance. To this end, a Livox MID 360 LiDAR [8] is mounted at the top of the UAV. This sensor provides a FOV of 360° × 59°, affording extensive environmental coverage. Furthermore, it is lightweight compared to other LiDAR systems, consumes relatively low power, and is financially affordable, making it a highly suitable choice for our UAV application.

For obstacle avoidance, a sensor array consisting of three 2D/3D Cygbot CygLiDAR D1 [9] ToF LiDAR sensors is implemented. These are strategically positioned at intervals of 120° around the UAV. Each CygLiDAR sensor provides a laser scan covering a FOV of 120° and generates a 3D point cloud within the dimensions of 120° × 65°. A complete 360° coverage is achieved by utilizing three such sensors, compensating for the blind spots of the top-mounted Livox LiDAR.

To further refine the UAV's perception, two Benewake TFmini S single-point laser

[7] https://www.raspberrypi.com/products/raspberry-pi-4-model-b
[8] https://www.livoxtech.com/mid-360
[9] https://www.cygbot.com/2d-3d-dual-solid-state-tof-lidar

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

rangefinders [10] are installed — one each at the top and bottom of the UAV. These sensors provide precise distance measurements to the floor and ceiling, facilitating obstacle avoidance in the vertical dimension and ensuring a safe flight altitude. This multi-sensor configuration enriches the spatial awareness of the UAV and enhances its ability to navigate safely and efficiently indoors.

3.2. SLAM

Indoor environments are challenging in localization due to the unavailability of GNSS signals. Consequently, indoor UAV rely on alternative technologies for localization. SLAM [89] enables robots to estimate their position and orientation to navigate autonomously in an unknown environment while simultaneously constructing and updating a map. So, the SLAM problem is generally separated into two major tasks: localization and mapping. Localization is focused on estimating the UAV's pose relative to a constructed or given map. At the same time, mapping is concerned with reconstructing the environment based on sensors such as cameras or LiDARs. Consider a UAV with its pose $\boldsymbol{q} \in \mathbb{R}^n$, describing its position $\boldsymbol{p}$ and orientation $\boldsymbol{r}$. The goal of the SLAM problem is to find the optimal state vector $\boldsymbol{q}^*$ that minimizes the measurement error $\boldsymbol{e}_i(\boldsymbol{q})$, weighted by a covariance matrix $\boldsymbol{S}_i \in \mathbb{R}^{n \times n}$, which encapsulates the uncertainty in the pose measurements. The objective function is formulated as follows:

$$\boldsymbol{q}^* = \arg\min_{\boldsymbol{q}} \sum_i \boldsymbol{e}_i^T(\boldsymbol{q}) \boldsymbol{S}_i \boldsymbol{e}_i(\boldsymbol{q}) \tag{3.1}$$

Traditional SLAM algorithms often function in a passive mode, wherein the robot is either manually controlled or moves towards pre-defined waypoints. The navigation or path-planning algorithm does not actively participate in determining the robot's motion. In autonomous exploration Coverage Path Planning (CPP) focuses on creating a path that efficiently explores an area, ensuring it covers every part. Combining CPP and SLAM and introducing an active loop closure mechanism enables Active Simultaneous Location And Mapping (A-SLAM) [90], which aims to tackle the optimal exploration problem by proposing navigation strategies that generate future goals or target positions. This reduces the uncertainty in the map and the robot's pose, enabling a fully autonomous navigation and mapping system. A detailed review of the A-SLAM research over the past decade can be found in [91]. Using A-SLAM is highly advantageous for our specific indoor UAV application. Not only does it dynamically generate a map of the environment, but it also simultaneously offers real-time localization of the UAV. Additionally, A-SLAM significantly enhances exploration capabilities by optimizing flight paths and dynamically updating them based on newly acquired data. This results in a more efficient and thorough coverage of the indoor space, effectively meeting our objectives for navigation and data acquisition. In [92], the performance of state-of-the-art SLAM algorithms based on the Livox MID 360 LiDAR sensor is tested in simulation and proves the sensor to work well for indoor UAV A-SLAM applications. Additionally, the Livox MID360 comes with an integrated IMU, increasing the robustness of the navigation and mapping capabilities of the quadcopter.

[10] https://www.benewake.com/TFminiS

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

3.3. Communication

Since the developed platform operates autonomously and parts of the algorithms are not running directly on the UAV, a reliable communication link is required to the GCS and the onboard equipment. ArduPilot supports the MAVLink protocol, enabling communication and control capabilities across various devices and platforms. In Figure 3.3, a schematic system overview of the platform with its communication links is given. The UAV needs waypoint commands from a control station for autonomous exploration. The A-SLAM algorithm with the optimal path planner is not executed directly on the UAV. Instead, these computations are offloaded to a GCS. The UAV and the GCS communicate wirelessly over a WiFi network using an external antenna for long-range and high data throughput. In our setup, the drone is equipped with an external WiFi antenna [11] and the GCS is a computer equipped with a powerful industrial router [12]. The GCS is running ROS, publishing the exploration waypoint and estimated position topic to the companion computer, and receiving point cloud and IMUdata from the UAV. On the UAV, the companion board computer is also running ROS. It provides the ROS-Master to communicate with the GCS, external ToF, and LiDAR sensors and establishes a MAVLink communication to the flight controller. The MAVLink communication transfers sensor data from the flight controller to the companion computer and the position setpoints from the companion computer to the controller. The flight controller is additionally equipped with a radio module to manually control the UAV with remote control.

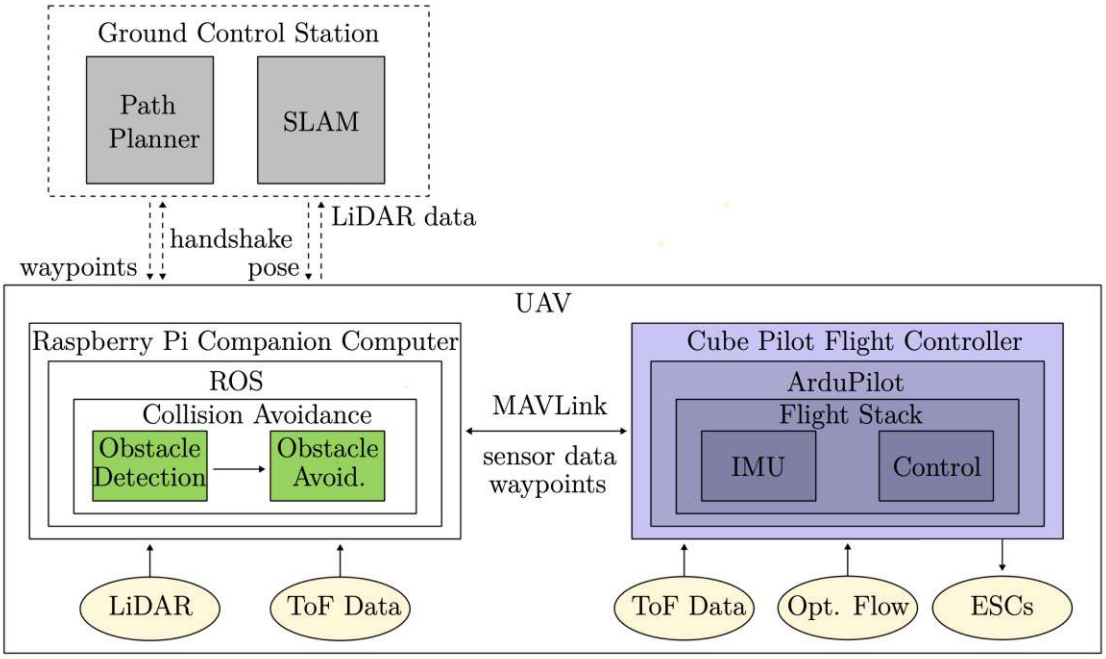


Figure 3.3.: Schematic UAV system overview with communication links.

[11] https://alfa-network.eu/awus036acm
[12] https://teltonika-networks.com/products/routers/rutx11

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

3.3.1. MAVLink Message Protocol

Micro Air Vehicle Link (MAVLink) is a lightweight, header-only message library for micro air vehicles, developed by Lorenz Meier in 2009 [93]. It sends instructions to the UAV onboard systems and receives status updates, telemetry, and other information from those systems. It allows real-time communication between the UAV and the ground control station. The MAVLink protocol is a crucial communication protocol used in the drone UAV industry, particularly in combination with the Ardupilot platform. MAVLink serves as the foundation for communication between various components of a UAV system, including the ground control station, onboard computers, and flight controllers. Once a connection is established, each device sends the *HEARTBEAT* message (e.g., 1Hz). MAVLink facilitates the uploading, downloading, and modification of missions in real time. In this context, a mission refers to a predefined set of actions the drone should execute, like waypoints or other commands. Local setpoints for the UAV's position, velocity, and orientation are transmitted using the `/mavros/setpoint_raw/local` topic. The UAV's actual position and orientation are acquired from the `mavros/global_position/local` and the IMUdata from the `mavros/imu_data` topic. To facilitate communication between the exploration path planner and the collision avoidance system, two custom message topics are employed:

- `/collision_avoidance/flag` indicates whether the collision avoidance system is active or if no collision has been detected.
- `/collision_avoidance/waypoint` allows the path planner to send calculated waypoints to the collision avoidance system.

3.3.2. ROS

The open-source ROS [76] is not an operating system in the traditional sense but a middleware designed for robot software development. It encapsulates libraries and tools, facilitating seamless integration and collaboration and linking various robotic hardware and software components. It provides hardware abstraction, device drivers, visualizers (RViz), and a structured framework where developers can create individual robotic functionalities as *nodes*. These nodes communicate with each other over a topic-based messaging system. Additionally, ROS's integration with simulation environments such as Gazebo offers a safe and efficient playground for testing algorithms, eliminating the risks associated with real-world robot testing.

The official bridge between MAVLink and ROS is MAVROS [13]. MAVROS translates messages from a ROS topic to MAVLink messages, allowing high-level commands from a ROS node to be interpreted by flight controllers. Conversely, it converts MAVLink messages from flight controllers to ROS messages, making data from the drone available to ROS nodes. In this work, we use ROS Noetic Ninjemys (ROS 1) [94] with the open-source Linux distribution Ubuntu 20.04 (Focal Fossa) [95].

[13] http://wiki.ros.org/mavros

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Object	Name	Specification	Weight [g]	Power [W]
Frame	Hexsoon EDU 450 Development Frame	360x360x260mm Payload 960g	530	–
Motor	T-Motor 2216 880KV	KV: 880, 16A max.	4x 65	4x max. 260
Rotor	T-Motor T1045	⌀ 25.5 cm	4x < 5	–
ESC	Hobby Wing XRotor 20A	20A max. 600Hz	4x 15.5	n.s.
Flight Controller	CubePilot Cube Orange+	STM32H753VI Processor 3x IMU, 2x Barometer 1x Compass	73	14
Battery	LEMONRC LiPo	3300mAh, 48.84 Wh Voltage: 14.8V	290	–
Sensor	Cygbot CygLiDAR D1 Solid State ToF LiDAR	2D/3D ToF $\lambda = 808nm$, 15 Hz Range 2D/3D: 0.2 - 8.0m ±1% 0.05 - 2.0m ±1% FOV: 120° horizontal 65° (3D) vertical Resolution: 2D: 1° (Angle) 3D: 160x60 (Pixel)	3x 28	3x 2.5
Sensor	Livox MID360	LiDAR with IMU $\lambda = 905nm$, 10 Hz Range: >40m ±0.01% FOV: 360° horizontal -7° - 52° vertical Resolution: ≤ 2cm, < 0.15°	265	6.5
Sensor	Hex HereFlow	Optical Flow: FOV: 42° Max. V.: $7.4rad/s$ LiDAR: $\lambda = 840nm$, up to 50 Hz Range: 2.0m ±3% FOV: 27°	5	0.1
Sensor	Benewake TFmini-S	Single-Point LiDAR $\lambda = 850nm$, 100 Hz Range: 12m ±1% Resolution: 1cm	2x5	2x0.1
Companion Computer	Raspberry PI 4 Model B	Quad-core ARMv8 8GB RAM 64-bit, 1.5GHz	46	<15
WiFi Antenna	ALFA AWUS036ACM	MIMO 2.4GHz 300Mbps 5GHz 867Mbps Power: 23dBm	58	n.s.
Regulator	Matek UBEC DUO	Buck regulator Two Channels	32	n.s.
Extras	Cables, Custom 3D Prints Connectors, Mounts	–	< 100	–
		Total	<963	<43.3

Table 3.1.: Utilized UAV hardware equipment. n.s = not specified

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

4. Mathematical Modeling

This section presents the mathematical basics of a quadcopter UAV. The configuration of the UAV is described by its position $\mathbf{p}$ and a rotation $\mathbf{R}_B^W$, where $\mathbf{p} = [x, y, z]^T$ is the position of the center of mass in the body frame I_B, and $\mathbf{R}_B^W$ is the rotation matrix from the world frame I_W to the body frame I_B as shown in Figure 4.1.

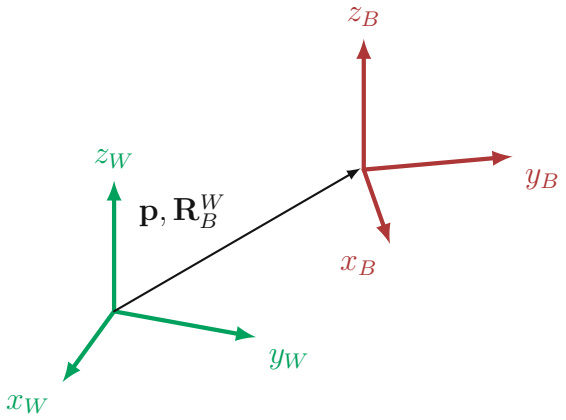


Figure 4.1.: Relation between coordinate frames I_B and I_W.

4.1. Coordinate Frames

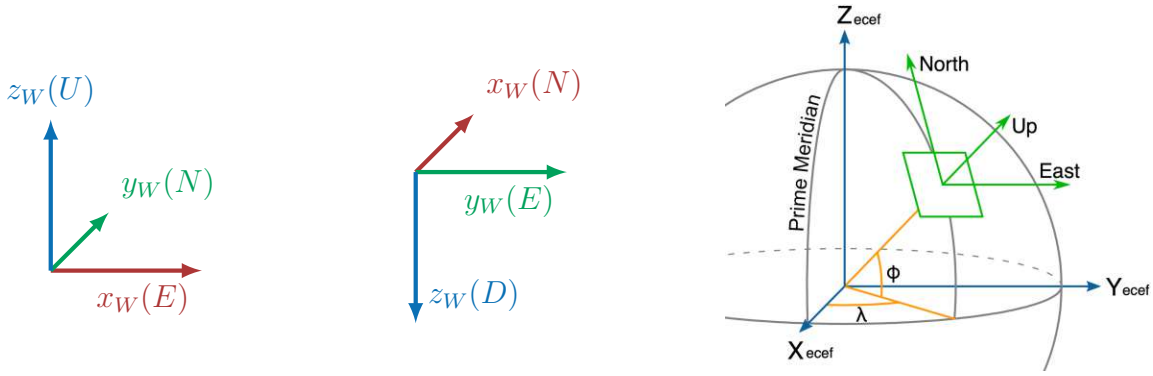
In the context of UAVs, there are commonly three coordinate frames used, namely the *Body Frame*, *World Frame*, and *Local Frame*. The Body Frame is attached to the UAV, with axes that move along with the vehicle, and the forward direction typically aligns with the x-axis and the upward direction with the z-axis. The World Frame is the fixed frame that provides an absolute reference against which the UAV's motion can be described. The Local Frame is defined as a reference frame for various computational tasks relative to the UAV's take-off position. A brief overview of commonly used conventions is given since there are different conventions for the alignment of coordinate frames. ArduPilot, Gazebo, and MAVLink use different conventions.

ArduPilot uses the North-East-Down (NED) coordinate system where the x-axis points north, the y-axis points east, and the z-axis points down toward the earth's surface. In Gazebo, the default coordinate frame used for defining the positions and orientations of models within the simulation environment is East-North-Up (ENU), where the x-axis points east, the y-axis points north, and the z-axis points up, away from the earth's surface. For the local navigation frame, MAVLink uses ENU as the default.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

For the global frame, the geographic Latitude (ϕ), Longitude (λ), and Altitude (z) (LLA) coordinate system and the Earth-Centered, Earth-Fixed (ECEF) XYZ-frame are commonly used.

The conventions of the different three-dimensional frames are illustrated in Figure 4.2. Figure 4.2a illustrates the ENU convention, Figure 4.2b the NED convention, and in Figure 4.2c the global frame LLA with its geographic latitude ϕ and longitude λ as well as the ECEF coordinate system is shown. It also illustrates the reference of the local ENU frame within the global frame.



(a) ENU (b) NED (c) Global Frame [96]

Figure 4.2.: Different Coordinate Frames

The transformation between the ENU and NED coordinate frames can be achieved using a coordinate rotation or change-of-basis matrix. The transformation matrix T from ENU to NED is defined as:

$$T = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{pmatrix} \tag{4.1}$$

A point p in ENU can then be transformed to NED using $p_{\text{NED}} = T \cdot p_{\text{ENU}}$.

Note that in the following sections, only the body frame I_B and the world frame I_W are used since no absolute GNSS position is available, and therefore, the world frame is referenced to the local frame $I_L = I_W$.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

4.2. Rotations

Rotation matrices or quaternions typically represent orientations in the three-dimensional space and the rotation of coordinate frames. Orientations and transformations between frames are crucial for sensor fusion, control algorithms, and path planning.

Rotation Matrix

Rotation matrices are used to describe rotations between two coordinate frames. Roll Pitch Yaw (RPY) angles are commonly used to describe rotations in three-dimensional space. They are easy to visualize and understand, as they directly correspond to rotations about specific cardinal axes. Consider the position of a sensor is described by $\mathbf{p}_B = [x, y, z]^T$ and $\mathbf{R}_B^W$, where $\mathbf{R}_W^B(\psi, \theta, \phi) \in SO3$ is the rotation matrix from I_B to I_W:

$$\begin{aligned}\mathbf{R}_B^W = \mathbf{R}_{z,\psi}\mathbf{R}_{y,\theta}\mathbf{R}_{x,\phi} &= \begin{bmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi & c_\phi \end{bmatrix} \\ &= \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}\end{aligned} \tag{4.2}$$

with the corresponding roll angle ϕ, pitch angle θ and yaw angle ψ, often denoted as *heading* as illustrated in Figure 4.3 and where $s.$ and $c.$ are the compact forms of $sin(\cdot)$ and $cos(\cdot)$. The transformation from I_W to I_B is given by $\mathbf{R}_W^B = (\mathbf{R}_B^W)^{-1} = (\mathbf{R}_B^W)^T$. Limitations of the concept are the Gimbal Lock and numerical instability.

With the known translation vector $\mathbf{r}_B^W$, the vector $\mathbf{p}_B$ in the coordinate frame I_B can then be transformed to the coordinate frame I_W as follows:

$$\mathbf{p}_W = \mathbf{R}_B^W \mathbf{p}_B + \mathbf{r}_B^W \tag{4.3}$$

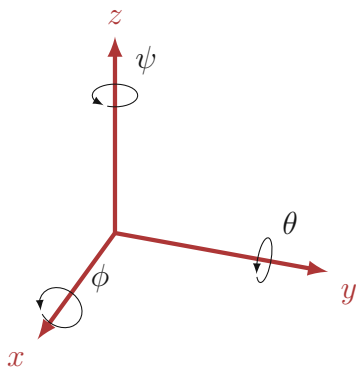


Figure 4.3.: Parameterization of the rotation matrix using the roll-pitch-yaw angles.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Quaternions

In flight controllers and for transformations in ROS, quaternions are widely used to describe rotations in three-dimensional space. Quaternions are computationally efficient and compact. A quaternion q is a mathematical object that extends complex numbers and is defined as:

$$q = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k} \tag{4.4}$$
$$q^{-1} = w - x\mathbf{i} - y\mathbf{j} - z\mathbf{k} \tag{4.5}$$

where $w, x, y, z \in \mathbb{R}$, and $\mathbf{i}$, $\mathbf{j}$, $\mathbf{k}$ are the quaternion units and q^{-1} is the conjugate of q. Then, a three-dimensional vector $\mathbf{v}$ can be rotated by a quaternion q through the operation:

$$\mathbf{v}' = q\mathbf{v}q^{-1} \tag{4.6}$$

where $\mathbf{v}'$ is the rotated vector.

Transformations Between Quaternions and RPY

Since RPY angles are more convenient and meaningful, they are the preferred method for setting and displaying orientations with user interactions. However, to guarantee performance in real-time, it is beneficial to use quaternions. Quaternions often result in faster calculations for 3D rotations because they require fewer trigonometric operations and do not suffer from gimbal lock. Therefore, converting between RPY angles and quaternion representations is necessary. The transformations are as follows:

Quaternion to RPY

$$\phi = \operatorname{arctan2}\left(2 \cdot (wx + yz), 1 - 2 \cdot (x^2 + y^2)\right) \tag{4.7}$$
$$\theta = \arcsin\left(2 \cdot (wy - zx)\right) \tag{4.8}$$
$$\psi = \operatorname{arctan2}\left(2 \cdot (wz + xy), 1 - 2 \cdot (y^2 + z^2)\right) \tag{4.9}$$

RPY to Quaternion

$$w = \cos\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) + \sin\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) \tag{4.10}$$
$$x = \sin\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) - \cos\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) \tag{4.11}$$
$$y = \cos\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) + \sin\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) \tag{4.12}$$
$$z = \cos\left(\frac{\phi}{2}\right)\cos\left(\frac{\theta}{2}\right)\sin\left(\frac{\psi}{2}\right) - \sin\left(\frac{\phi}{2}\right)\sin\left(\frac{\theta}{2}\right)\cos\left(\frac{\psi}{2}\right) \tag{4.13}$$

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

4.3. UAV Dynamics

This section introduces the dynamics and kinematics of a quadcopter UAV. A detailed derivation of a quadcopter's differential equations and dynamics can be found in [97] and the general theoretical derivations in [98]. In Figure 4.4, a model of a simplified quadcopter platform with its body coordinate system I_B and the world coordinate system I_W is presented. The UAV's position is considered as a central mass in the origin of the body frame I_B, denoted as $\mathbf{p}$.

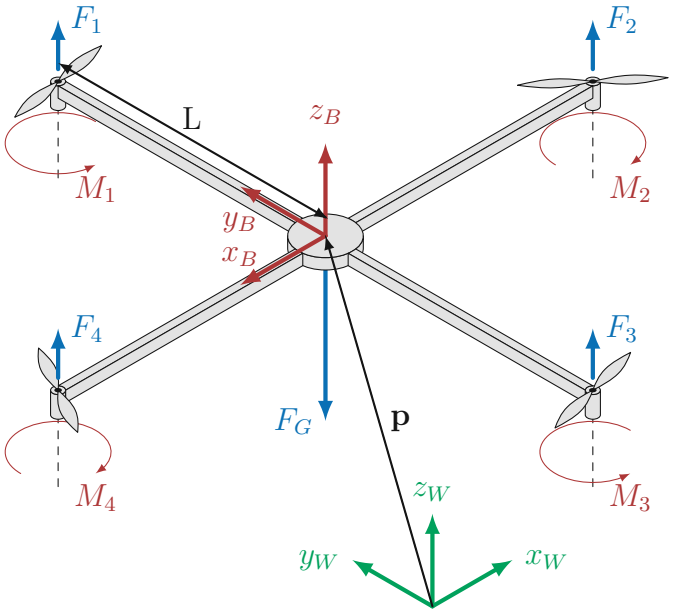


Figure 4.4.: Schematic representation of the quadcopter UAV and the relation between body coordinate frame I_B with the acting moments and forces and the world coordinate frame I_W.

Every motor $i \in \{1, 2, 3, 4\}$ rotates with an angular rate ω_i which generates an opposing rotating spin, the moment M_i. A rotor spinning in a clockwise direction generates a torque that rotates the drone's body in the counter-clockwise direction and vice versa. Additionally, a rotor i exerts an upward thrust, denoted as force F_i, in the positive z_B direction. The resulting aerodynamic moment M and aerodynamic force F are given by:

$$M = \sum_i M_i = \sum_i k_M \omega_i^2 \tag{4.14}$$

$$F = \sum_i F_i = \sum_i k_F \omega_i^2 \tag{4.15}$$

with constant drag parameter $k_M > 0$ and lift parameter $k_F > 0$. Since there is normally no measurement for the motor speed, it is estimated by the Pulse-Width Modulation (PWM) value sent to the motor controller.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

By deriving the vertical forces acting on the vehicle within the world frame I_W and formulating the motion equations with Newton's second law, we can express the acceleration of the center of mass:

$$m \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ F_G \end{bmatrix} + \mathbf{R}_B^W \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix} \tag{4.16}$$

where m denotes the quadrotor's mass, $F_G = -mg$ is the force exerted by the gravitational acceleration g, $\mathbf{R}_B^W$ denotes the transformation matrix (Equation 4.2) from I_B to I_W, and F represents the force exerted by the rotors, detailed in formula Equation 4.15.

While it is beneficial to express the linear equations of motion in the world frame I_W, using the body frame I_B for rotational motion is advantageous. This allows to describe rotations around the UAV's center rather than around a fixed inertial center. Its IMU is utilized to acquire the quadcopter's angular accelerations. The outputs from the IMU are represented as $\boldsymbol{\omega_B} = [p, q, r]^T$. To calculate the angular accelerations, Euler's method is applied, defined by:

$$\mathbf{I}\dot{\boldsymbol{\omega}}_B + \boldsymbol{\omega}_B \times [\mathbf{I}\boldsymbol{\omega}_B] = \boldsymbol{\tau} - \boldsymbol{\tau}_G \tag{4.17}$$

where $\mathbf{I} = diag(I_x, I_y, I_z)$ denotes the inertia matrix of the quadcopter with $I_x = I_y$ since the quadcopter is built symmetrically to the body frame I_B, and $\boldsymbol{\tau}$ denotes the external torques

$$\boldsymbol{\tau} = \begin{bmatrix} L(F_3 - F_1) \\ L(F_4 - F_2) \\ M_1 - M_2 + M_3 - M_4 \end{bmatrix} \tag{4.18}$$

where L is the distance from each rotor to the vehicle's center of gravity as shown in Figure 4.4 and $\boldsymbol{\tau}_G$ stands for the gyroscopic moment generated by the rotor's inertia. However, the gyroscopic moment and aerodynamic torques are neglected because the inertia generated by the platform itself is significantly greater than the inertia produced by the rotors, and they tend to neutralize each other during flight due to the motor's clockwise and counter-clockwise rotation.

To convert the the angular velocities $\boldsymbol{\omega_W} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]^T$ in the world frame I_W to angular velocities $\boldsymbol{\omega_B} = [p, q, r]^T$ from the body frame I_B we apply the transformation:

$$\begin{aligned} \boldsymbol{\omega_B} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} &= \begin{bmatrix} \dot{\phi} \\ 0 \\ 0 \end{bmatrix} + \mathbf{R}_{x,\psi} \begin{bmatrix} 0 \\ \dot{\theta} \\ 0 \end{bmatrix} + \mathbf{R}_{x,\psi}\mathbf{R}_{y,\theta} \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix} \\ &= \mathbf{T}\boldsymbol{\omega_W} = \begin{bmatrix} 1 & 0 & -\sin\theta \\ 0 & \cos\phi & \sin\phi\cos\theta \\ 0 & -\sin\phi & \cos\phi\cos\theta \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \end{aligned} \tag{4.19}$$

and with the inverted matrix the conversion from $\boldsymbol{\omega_B}$ to $\boldsymbol{\omega_W}$ is calculated:

$$\boldsymbol{\omega_W} = \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \mathbf{T}^{-1}\boldsymbol{\omega_B} = \begin{bmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi\cos\theta^{-1} & \cos\phi\cos\theta^{-1} \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \tag{4.20}$$

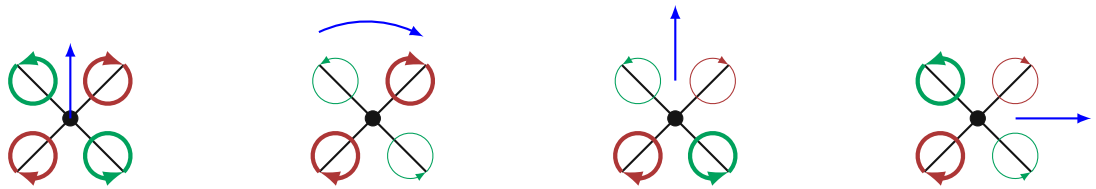
Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

4.4. Flight Control

The collective thrust generated by all four motors $i \in \{1, 2, 3, 4\}$ controls the altitude and make the quadcopter fly. To maintain stability and prevent axial rotation, the rotors must rotate in opposite directions $(\omega_1 + \omega_3) - (\omega_2 + \omega_4) = 0$, as shown in Figure 4.4. To ensure that the UAV hovers steadily at a fixed position, the following conditions must be met:

$$F_i = \frac{mg}{4}, \quad \sum_{i=1}^{4} M_i = 0, \quad \dot{\phi} = \dot{\theta} = \dot{\psi} = 0 \tag{4.21}$$

For a movement in a specific direction or a yaw movement, the motors must be controlled with varying power levels as illustrated in Figure 4.5. This task is managed by the flight controller, in this case, by the ArduPilot Cube Orange+ and the Electrical Speed Controller (ESC). In Figure 3.2, the position and attitude controller unit implemented in the ArduPilot flight controller is depicted.



Altitude Move Up | Yaw Rotate Right | Pitch Move Forwards | Roll Move Right

Figure 4.5.: Elementary UAV motion maneuvers.

For a quadcopter, the behavior in 3D space is described by a 12-state model

$$\mathbf{x}^T = \left[x, y, z, \dot{x}, \dot{y}, \dot{z}, \phi, \theta, \psi, \dot{\phi}, \dot{\theta}, \dot{\psi}\right] \tag{4.22}$$

where the first set $[x, y, z, \dot{x}, \dot{y}, \dot{z}]$ is related to its $\mathbb{R}^3$ position $\boldsymbol{p}$ [m] and velocity $\boldsymbol{v}$ [m/s] in the world frame I_W and the second set $[\phi, \theta, \psi, \dot{\phi}, \dot{\theta}, \dot{\psi}]$ to its $\mathbb{R}^3$ orientation $\boldsymbol{r}$ [rad] and angular velocity $\boldsymbol{\omega}_B$ [rad/s] in the body frame I_B. The system has six Degree Of Freedom (DOF) $(\phi, \theta, \psi, x, y, z)$, and since a quadcopter only has four control inputs, which are the thrusts generated by the quadcopter's four motors, the system is underactuated. The rotational subsystem is fully actuated since the four motors can generate torques that allow precise control over these three angles. In contrast, the translational subsystem is underactuated, and two of the six DOFs in the quadcopter system are coupled, namely the x and y axes in the translational motion.

The control inputs $\mathbf{u}$ for the quadcopter can be defined with the forces F_1, F_2, F_3, F_4 and moments M_1, M_2, M_3, M_4 generated by its four rotors as:

$$\boldsymbol{u} = \begin{bmatrix} u_z \\ u_x \\ u_y \\ u_\psi \end{bmatrix} = \begin{bmatrix} F_1 + F_2 + F_3 + F_4 \\ L(F_2 - F_4) \\ L(F_3 - F_1) \\ M_1 - M_2 + M_3 - M_4 \end{bmatrix} \tag{4.23}$$

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

The motor dynamics are generally much faster than the rigid-body dynamics and aerodynamic effects of the quadcopter [99]. Therefore, when needed, the desired rotor speeds ω_i can be achieved instantaneously. With the control inputs 4.23, combining with 4.2, 4.16 and 4.17 the state space representation is given as:

$$\dot{\mathbf{p}} = \mathbf{v} \tag{4.24}$$

$$\dot{\mathbf{v}} = \ddot{\mathbf{p}} = -\begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} + \frac{u_z}{m}\begin{bmatrix} c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi s_\theta c_\phi - c_\psi s_\phi \\ c_\theta c_\phi \end{bmatrix} \tag{4.25}$$

$$\boldsymbol{\omega_W} = \mathbf{T}^{-1}\boldsymbol{\omega_B} \tag{4.26}$$

$$\dot{\boldsymbol{\omega}}_{\boldsymbol{B}} = \mathbf{I}^{-1}\left[\begin{bmatrix} u_x \\ u_y \\ u_\psi \end{bmatrix} - \boldsymbol{\omega_B} \times [\mathbf{I}\boldsymbol{\omega_B}]\right] \tag{4.27}$$

Since the quadcopter represents a differential flat system [100, 101], path planning and waypoint navigation only rely on the translation and the heading between the UAV's body frame I_B and the world frame I_W and is fully described by $\mathbf{p} = [x, y, z]$ and ψ leading to a simpler model given in Equation 4.28. The z_W axis is equivalent to the z_B axis. This is because the body frame I_B is formed by performing only a yaw rotation ψ around the z_W axis as shown in Figure 4.6.

$$\begin{cases} \dot{p}_x = v_x \cos(\psi) - v_y \sin(\psi), \\ \dot{p}_y = v_x \sin(\psi) + v_y \cos(\psi), \\ \dot{p}_z = v_z, \\ \dot{\psi} = \omega_z, \end{cases} \tag{4.28}$$

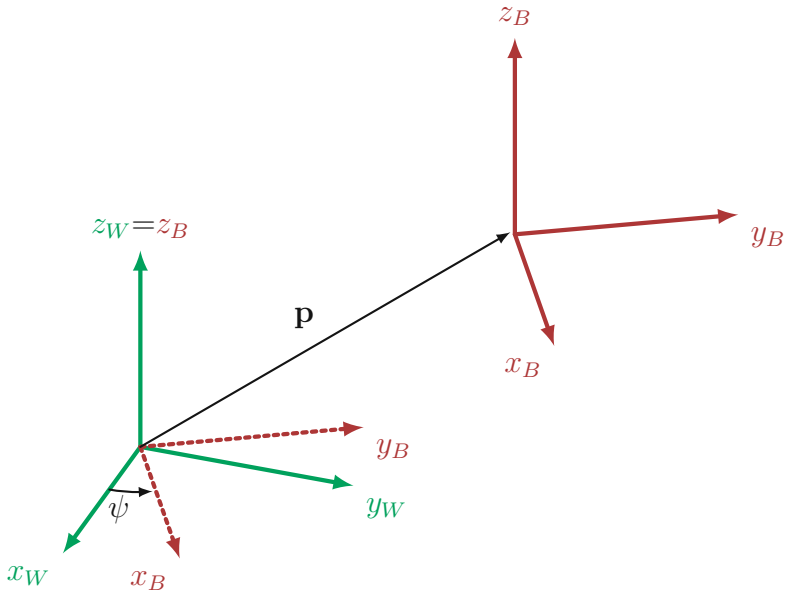


Figure 4.6.: Relation between I_W and I_B for way point navigating.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

5. Collision Avoidance

In the context of autonomously navigating and operating the UAV within uncooperative indoor environments, it is crucial to have an onboard, low-level collision avoidance system in place. While the autonomous path planner [102] using the exploration logic from the GLocal framework [103] is designed to provide a collision-free path, it is not always fail-proof, mainly when there are obstacles out of top-mounted LiDARs FOV or small obstacles are present. A further limitation of the global planner is that it primarily accounts for a static environment and is not designed to respond to dynamic changes or movements within the space. The precision of both SLAM and path planning can be less reliable, especially in confined indoor settings such as navigating through doors. There also might be instances where signals from the ground station - responsible for SLAM and path planning calculations - are lost, delayed, or interrupted. This means the avoidance algorithm needs to run locally on the UAV. Consequently, the algorithm must be lightweight regarding computational demands to ensure efficient and responsive navigation in such environments.

Figure 5.1 provides an overview of the proposed avoidance system, including obstacle detection and collision avoidance. The 360° LIVOX LiDAR provides the point cloud and IMU data for computing the A-SLAM and estimating the position within the created map. The autonomous path planner uses the map and position to calculate the further exportation path. At the same time, collision avoidance checks the calculated waypoints and executes an avoidance movement if necessary. A handshake procedure between the path planner and the avoidance system takes place to communicate which planner is active. Collision avoidance includes detecting obstacles, processing this information, and making real-time decisions to recalculate the trajectory accordingly. The collision avoidance algorithm is executed at an update rate of $f = 5$ Hz, achieving a compromise between reactivity and computational costs. This rate is sufficiently fast to reliably respond to dynamic changes in the environment while being slow enough to minimize computational overhead. Considering that the sensors employed for obstacle detection have an update rate of up to 15 Hz, this setup ensures that the collision avoidance algorithm consistently utilizes the most recent obstacle data available. The resulting avoidance movement only considers the UAVs current position, target waypoint from the exploration planner, and sensor inputs. The following sections address the object detection employed in this thesis, the chosen collision avoidance methods, and their practical application to prevent collisions.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

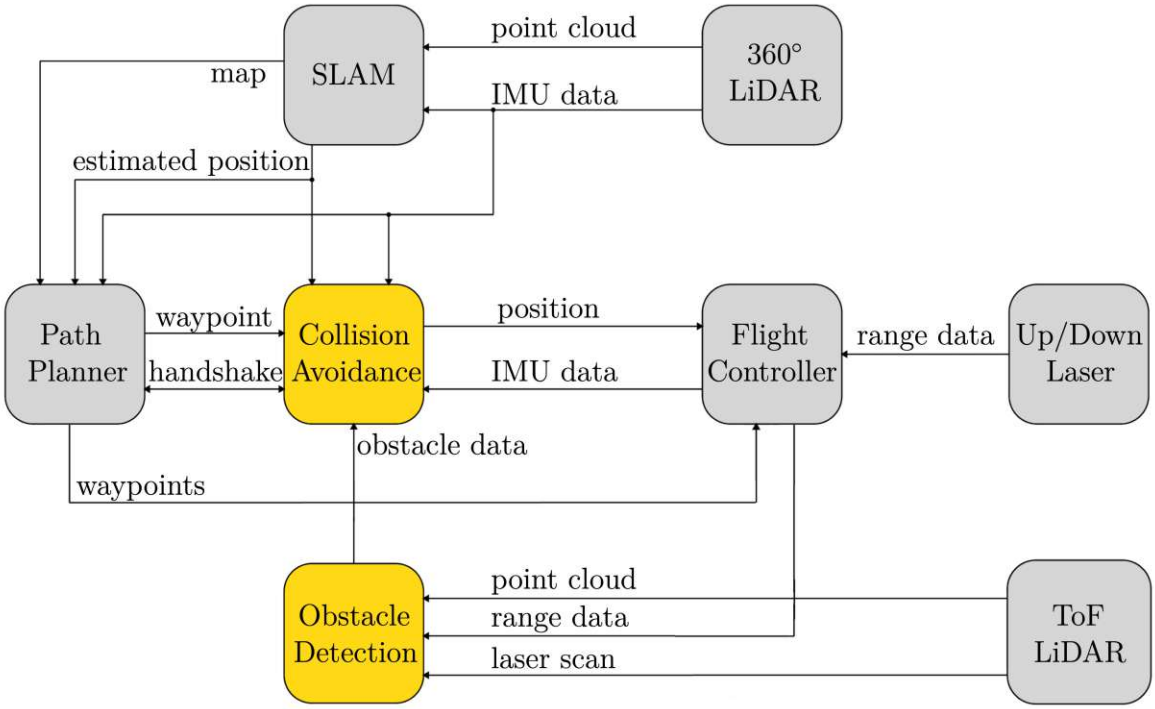


Figure 5.1.: Overall schematic diagram of the obstacle avoidance system.

5.1. Obstacle Detection

Reliable, accurate, fast obstacle detection is critical for real-time obstacle avoidance systems. Obstacle detection involves preprocessing the sensor data and obstacle feature extraction. Based on the sensor type, different approaches for effectively localizing objects in the UAV's surrounding environment are used [4].

The 2D/3D Cyglidar and TFmini 1D ToF LiDAR sensors generate point clouds and laser scans, which include depth data that can be easily processed for obstacle detection. The laser scan ROS topics generated by each sensor are combined into one single topic using the *IRA-Laser Toolkit* [1] developed by Cornell University. Similarly, the point clouds of the three sensors are merged using the *pointcloud-concatenate* [2] point cloud merging tool, resulting in a single merged point cloud. These tools have proven robust and efficient, making them the choice for this critical task. Merging all the captured data into a single laser scan or point cloud topic makes it easier to manage and process the latest obstacle data in one timestamp.

A clustering algorithm is applied to identify individual obstacles on the merged point cloud. Clustering algorithms have limitations and advantages and have been compared on toy datasets to analyze their efficacy and suitability for different types of data in [104]. Based on these results, the Density-Based Spatial Clustering of Applications with Noise (DBSCAN) [105] clustering is used to extract single obstacle information.

[1] https://github.com/iralabdisco/ira-laser-tools
[2] https://github.com/aseligmann/pointcloud-concatenate

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Unlike other clustering algorithms like K-means and others, DBSCAN can identify clusters of any shape and size. The number of clusters does not need to be known in advance, which is a crucial advantage when considering complex obstacles in the real world. Furthermore, DBSCAN filters out noise and outliers, enabling a more accurate and reliable obstacle detection and representation of noisy sensor data. The *scikit-learn* [3] library provides the used DBSCAN clustering implementation.

Another important aspect of this thesis involves minimizing the number of data points used to describe each detected obstacle. Reducing these points without compromising on the obstacle's spatial fidelity is essential for real-time processing by the avoidance algorithms. The aim is to offer a computational advantage while maintaining high accuracy and reliability. The *open3d.geometry.voxel_down_sample* [4] implementation shows the best performance regarding coverage and number of necessary points. Various experimental setups have been established to validate the detection, including simulated and real-world scenarios, detailed in section 6.3.

5.2. Avoidance Algorithms

This section presents the extended versions of the Artificial Potential Field (APF+) and the 3D Dynamic Window Approach algorithm, which incorporate the Bug2 algorithm to overcome local minima. A detailed discussion and the evolution and general behavior of these algorithms is provided in section 2.3. The configuration vector of the UAV is denoted as $\mathbf{x} = [\mathbf{p}^T \psi]^T \in \mathbb{R}^4$ with the position of the UAV $\mathbf{p} = [x, y, z]^T \in \mathbb{R}^3$ and the yaw rotation angle or *heading* $\psi \in [-\pi, \pi)$.

5.2.1. Artificial Potential Field

The implementation of the APF+ method in this work is based on multiple approaches [40, 45, 106, 107], with extensions to three-dimensional space and additional features. The artificial potential field within the robot operates is defined by a scalar function $U(\mathbf{p}) : \mathbb{R}^3 \rightarrow \mathbb{R}$ as the sum of attractive and repulsive potentials:

$$U(\mathbf{p}) = U_a(\mathbf{p}) + U_r(\mathbf{p}) \tag{5.1}$$

The repulsive potential is the sum of distinct repulsive potentials generated by various obstacles:

$$U_r(\mathbf{p}) = \sum_i U_{r_i}(\mathbf{p}) \tag{5.2}$$

where $U_{r_i}(\mathbf{p})$ denotes the repulsive potential caused from obstacle i. If we assume that the artificial potential field $U(\mathbf{p})$ is differentiable, the gradient of the field can be denoted as:

$$\nabla U(\mathbf{p}) = [\frac{\partial U(\mathbf{p})}{\partial x}, \frac{\partial U(\mathbf{p})}{\partial y}, \frac{\partial U(\mathbf{p})}{\partial z}]^T \tag{5.3}$$

[3] https://github.com/scikit-learn
[4] http://www.open3d.org

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

which serves as a vector that points in the direction where the function $U(\mathbf{p})$ experiences the most rapid local increase. The resulting artificial force of the UAV from the negative gradient of the artificial potential is given by:

$$\mathbf{F}(\mathbf{p}) = \mathbf{F}_a(\mathbf{p}) + \mathbf{F}_r(\mathbf{p}) = -\nabla U(\mathbf{p}) = -\nabla U_a(\mathbf{p}) - \nabla U_r(\mathbf{p}) \tag{5.4}$$

This force is a vector facing in the direction that causes a local maximal decline in the potential $U(\mathbf{p})$. To calculate the new position of the UAV the method of gradient descent [108] with the step size $\alpha > 0$ is applied:

$$\mathbf{p}_{t+1} = \mathbf{p}_t - \alpha \nabla U(\mathbf{p}_t) = \mathbf{p}_t + \alpha \mathbf{F}(\mathbf{p}_t) \tag{5.5}$$

until the condition

$$\delta_{goal}(\mathbf{p}) \leq \delta_{threshold} \tag{5.6}$$

is met, where $\delta_{goal}(\mathbf{p}) = \|\mathbf{p} - \mathbf{p}_{goal}\|$ is the Euclidean distance between the UAV's position $\mathbf{p}$ and the destination $\mathbf{p}_{goal}$ and $\delta_{threshold}$ is a threshold radius.

Attractive Potential

Khatib's attractive potential [35] is engineered to be zero at the destination, and increases as the UAV diverges from this point. It is designed to be a quadratic function concerning the distance from the goal within the threshold distance ρ and then increases linearly with the target distance:

$$U_a(\mathbf{p}) = \begin{cases} \frac{1}{2} k_a \delta_{goal}^2(\mathbf{p}) & \text{if } \delta_{goal}(\mathbf{p}) \leq \rho \\ \rho k_a \delta_{goal}(\mathbf{p}) - \frac{1}{2} k_a \rho^2 & \text{if } \delta_{goal}(\mathbf{p}) > \rho \end{cases} \tag{5.7}$$

where $k_a > 0$ is a scaling constant that characterizes the intensity of the potential field, as illustrated in Figure 5.2a. The parameter $\rho > 0$ functions as the switching boundary coefficient. A quadratic function is used to ensure that only positive values are permitted. The term $\frac{1}{2} k_a \rho^2$ is introduced to guarantee the continuity condition. The gradient of the attractive potential can be computed as follows:

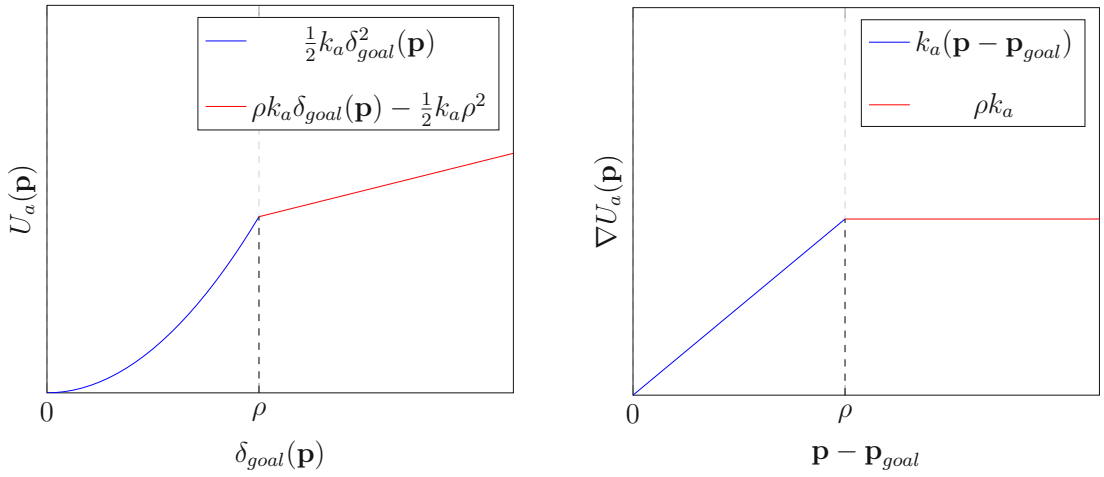
$$\nabla U_a(\mathbf{p}) = \begin{cases} k_a \delta_{goal}(\mathbf{p}) \nabla \delta_{goal}(\mathbf{p}) & \text{if } \delta_{goal}(\mathbf{p}) \leq \rho \\ \rho k_a \nabla \delta_{goal}(\mathbf{p}) & \text{if } \delta_{goal}(\mathbf{p}) > \rho \end{cases} \tag{5.8}$$

and with $\nabla \delta_{goal}(\mathbf{p}) = \frac{\mathbf{p} - \mathbf{p}_{goal}}{\|\mathbf{p} - \mathbf{p}_{goal}\|}$ the result of the attractive force is:

$$\mathbf{F}_a(\mathbf{p}) = -\nabla U_a(\mathbf{p}) = \begin{cases} -k_a(\mathbf{p} - \mathbf{p}_{goal}) & \text{if } \delta_{goal}(p) \leq \rho \\ -\rho k_a \frac{\mathbf{p} - \mathbf{p}_{goal}}{\delta_{goal}(\mathbf{p})} & \text{if } \delta_{goal}(p) > \rho \end{cases} \tag{5.9}$$

Below the switching boundary ρ, the gradient of the attractive potential increases linearly with the target distance $\delta_{goal}(\mathbf{p})$, as illustrated in Figure 5.2b. It then flattens to a constant value, given that $\frac{\mathbf{p} - \mathbf{p}_{goal}}{\delta_{goal}(\mathbf{p})} = 1$. The two key influencing parameters are the gain k_a and the switching boundary ρ. The function should align with the described characteristics to prevent excessively high values for ∇U_a.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.



(a) Attractive potential function $U_a(\mathbf{p})$ (b) Gradient $\nabla U_a(\mathbf{p})$

Figure 5.2.: Visualizations of the attractive potential function $U_a(\mathbf{p})$ and its gradient.

Repulsive Potential

The repulsive potential prevents the UAV from colliding with obstacles. Each obstacle has a high repulsive potential when the UAV is close to it and weakens as the distance from the obstacle increases. The repulsive potential is the summation of the effects of all obstacles:

$$U_r(\mathbf{p}) = \sum_i U_{r_i}(\mathbf{p}) \tag{5.10}$$

There are many approaches to repulsive potential functions like exponential potential functions, smooth attractive-repulsive functions [109], or functions considering the distance to the target position [110]. The most commonly and also in this work used repulsive potential function is Khatib's [35] original barrier function where each singular repulsive potential $U_{r_i}(\mathbf{p})$ is expressed as:

$$U_{r_i}(\mathbf{p}) = \begin{cases} \frac{1}{2}k_r\left(\frac{1}{\delta_{obs_i}(\mathbf{p})} - \frac{1}{d_0}\right)^2 & \text{if } \delta_{obs_i}(\mathbf{p}) < d_0 \\ 0 & \text{if } \delta_{obs_i}(\mathbf{p}) \geq d_0 \end{cases} \tag{5.11}$$

where $\delta_{obs_i}(\mathbf{p}) = \|\mathbf{p}_{obs_i}\|$ is the Euclidean distance to obstacle i, $k_r > 0$ is a scaling factor, d_0 is the influence range of the obstacle, and the term $-\frac{1}{d_0}$ is to guarantee the continuity condition. The gradient of the repulsive potential $U_{r_i}(\mathbf{p})$ results in:

$$\nabla U_{r_i}(\mathbf{p}) = \begin{cases} k_r\left(\frac{1}{\delta_{obs_i}(\mathbf{p})} - \frac{1}{d_0}\right)\frac{-\nabla\delta_{obs_i}(\mathbf{p})}{\delta^2_{obs_i}(\mathbf{p})} & \text{if } \delta_{obs_i}(\mathbf{p}) < d_0 \\ 0 & \text{if } \delta_{obs_i}(\mathbf{p}) \geq d_0 \end{cases} \tag{5.12}$$

The repulsive force is the negative gradient of the potential $-\nabla U_{r_i}(\mathbf{p})$:

$$\mathbf{F}_{r_i}(\mathbf{p}) = \begin{cases} k_r\left(\frac{1}{\delta_{obs_i}(\mathbf{p})} - \frac{1}{d_0}\right)\frac{\mathbf{p}_{obs_i}}{\delta^3_{obs_i}(\mathbf{p})} & \text{if } \delta_{obs_i}(\mathbf{p}) > d_0 \\ 0 & \text{if } \delta_{obs_i}(\mathbf{p}) \geq d_0 \end{cases} \tag{5.13}$$

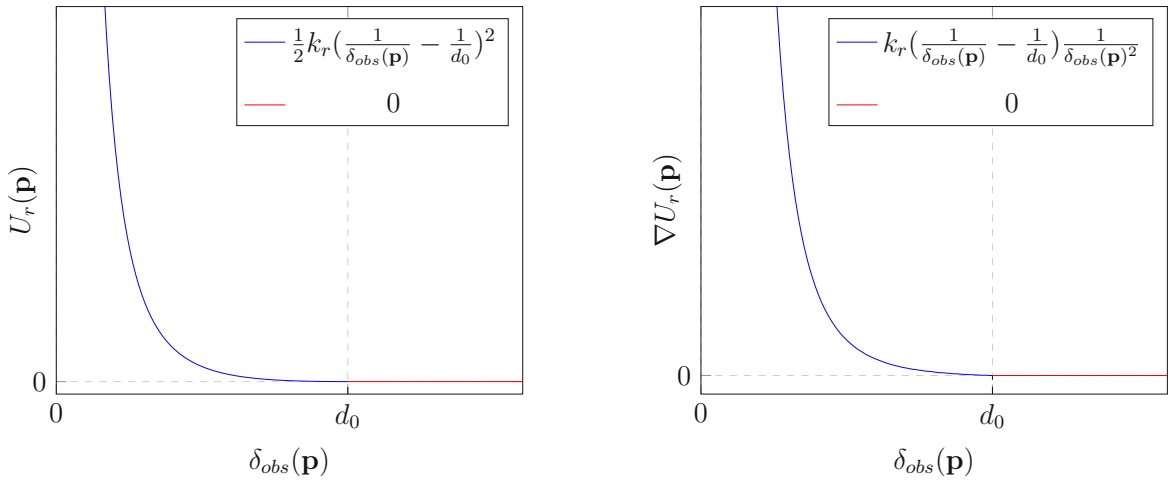
Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

In indoor navigation, it is crucial to account for not only walls and objects as potential obstacles but also the floor and ceiling. A straightforward quadratic potential function is utilized to ensure that the UAV maintains an optimal altitude — avoiding flight too close to either the ground or the ceiling. This function serves to keep the UAV away from these vertical boundaries:

$$U_{r_i,z}(\mathbf{p}) = \begin{cases} \frac{1}{2} k_{g,c} \delta_{g,c}^2(\mathbf{p}) & \text{if } \delta_{g,c}(\mathbf{p}) \leq z_{th} \\ 0 & \text{if } \delta_{g,c}(\mathbf{p}) > z_{th} \end{cases} \tag{5.14}$$

while $U_{r_i,x}(\mathbf{p}) = U_{r_i,y}(\mathbf{p}) = 0$ and where $\delta_{g,c}(\mathbf{p})$ represents the distance of the UAV to the ground or ceiling, $k_{g,c} > 0$ serves as a scaling factor, which adjusts the strength of the repulsion and z_{th} is used as a threshold distance. This quadratic potential formula generates a linear repulsive force, effectively guiding the UAV to maintain a safe distance from both the ground and ceiling:

$$\mathbf{F}_{r_i,z}(\mathbf{p}) = -\nabla U_{r_i,z}(\mathbf{p}) = \begin{cases} -k_{g,c} \delta_{g,c}(\mathbf{p}) & \text{if } \delta_{g,c}(\mathbf{p}) \leq z_{th} \\ 0 & \text{if } \delta_{g,c}(\mathbf{p}) > z_{th} \end{cases} \tag{5.15}$$



(a) Repulsive potential function $U_r(\mathbf{p})$ (b) Gradient $\nabla U_r(\mathbf{p})$

Figure 5.3.: Visualizations of the repulsive potential function $U_r(\mathbf{p})$ and its gradient.

Additional logic is integrated to ensure smooth movement across all three dimensions while improving speed and reliability in restricted areas. The scaling constants $\mathbf{k_a} > \mathbf{0}$ and $\mathbf{k_r} > \mathbf{0}$ are considered as a vector of constant values for each dimension:

$$\mathbf{k_a}^T = [k_{a_x}, k_{a_y}, k_{a_z}]$$

$$\mathbf{k_r}^T = [k_{r_x}, k_{r_y}, k_{r_z}]$$

This results in being more independent in the avoidance maneuvers in the z-axis compared to the avoidance movements in the horizontal plane. To make the resulting avoidance movement smoother, an additional force is added. Lindqvist et al. [45] proposed to add a weak "rotational" component to move smoother around obstacles

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

and reduce repeating bouncing movements. Based on this approach an additional force $\tilde{F}_r(\mathbf{p})$ is implemented:

$$\tilde{\mathbf{F}}_{r,x}(\mathbf{p}) = \tilde{k}\left(\text{sign}(\mathbf{F}_{r,x})\|\mathbf{F}_{r,y}\| + \mathbf{F}_{r_\epsilon}\right) \tag{5.16}$$

$$\tilde{\mathbf{F}}_{r,y}(\mathbf{p}) = \tilde{k}\left(\text{sign}(\mathbf{F}_{r,y})\|\mathbf{F}_{r,x}\| + \mathbf{F}_{r_\epsilon}\right) \tag{5.17}$$

$$\tilde{\mathbf{F}}_{r,z}(\mathbf{p}) = 0 \tag{5.18}$$

$$\tilde{\mathbf{F}}_{r}(\mathbf{p}) = [\tilde{\mathbf{F}}_{r,x}(\mathbf{p}), \tilde{\mathbf{F}}_{r,y}(\mathbf{p}), \tilde{\mathbf{F}}_{r,z}(\mathbf{p})]^T \tag{5.19}$$

where $\tilde{k} > 0$ serves as scaling factor and $\mathbf{F}_{r,x}, \mathbf{F}_{r,y}$ are the cumulative repulsive forces in x and y direction, respectively and $\mathbf{F}_{r_\epsilon} > 0$ is a additional rotational force to cover the case $sign(0) = 0$. In this work, we consider the scaling factor $\tilde{k}$ is not a constant value but dependent on the nearest obstacle distance:

$$\tilde{k} = k_{\tilde{k}} \cdot \frac{1}{1 + e^{-\sigma\left(\delta_{obs_{min}} - \tilde{\delta_{th}}\right)}} \tag{5.20}$$

where $\delta_{obs_{min}}$ is the minimal obstacle distance, $\tilde{\delta_{th}}$ is the threshold obstacle distance, and $k_{\tilde{k}} > 0$ and $\sigma > 0$ are constant factors, resulting in smoother avoidance movements in surroundings with enough space and preventing impairing rotational forces in confined spaces such as narrow passages like doorways.

Finally, the resulting repelling force $\mathbf{F}_r(\mathbf{p})$ is the sum of all components:

$$\mathbf{F}_r(\mathbf{p}) = \sum_i \mathbf{F}_{r_i}(\mathbf{p}) + \sum_i F_{r_i,z}(\mathbf{p}) + \tilde{\mathbf{F}}_r(\mathbf{p}) \tag{5.21}$$

APF methods struggle with effectively navigating through close obstacles. It is generally better to assign a larger radius of influence d_0 to give more leeway and time for the avoidance maneuver to begin and move around obstacles without high repulsive gains. Nevertheless, assigning a larger radius becomes problematic, especially in indoor environments when navigating through confined spaces like narrow corridors, passages, or doors. To address this problem, an adaptive approach for determining the radius of influence d_0 has been proposed in [45]. In this work, we consider the influence radius dependent on the currently estimated velocity of the UAV v, as presented in Equation 5.22, where $d_{0_{min}}$ is the minimal safety radius, $d_{0_{max}}$ is the maximum radius, $\tau > 0$ a scaling constant and $\|v\|$ represent the norm of the velocity of the UAV.

$$d_0 = d_{0_{max}} \cdot e^{-\tau(v_{th} - \|v\|)} \in [d_{0_{min}}, d_{0_{max}}] \tag{5.22}$$

The goal is to find a trade-off between having a suitably large radius of influence for early detection and avoidance of obstacles and maintaining the capacity to navigate more confined areas. As long as no obstacles are detected within this defined distance, the avoidance algorithm is inactive, and the UAV is moving straight to the calculated waypoint from the global planner.

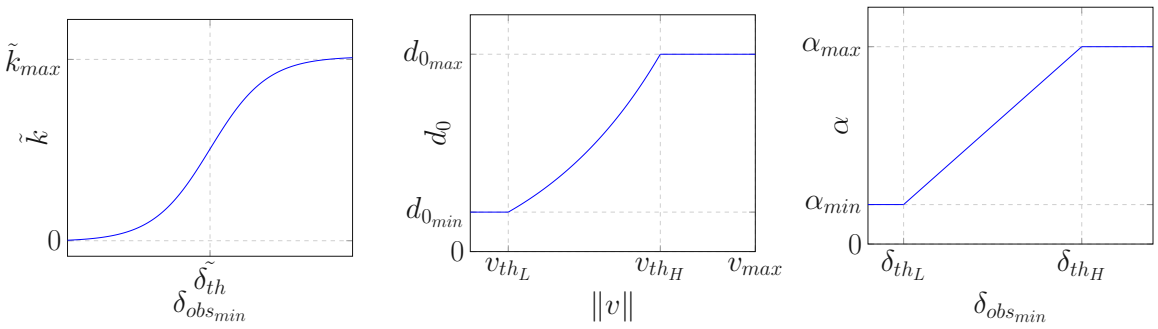
Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

The step size α in Equation 5.5 determines the distance step in the direction of the next calculated position and implicitly the speed of the UAV. There is a trade-off between UAV speed and safe operation step size and overshoot since a high value leads to a significant change in position. To overcome this trade-off, the step size is set variable to the minimum obstacle distance $\delta_{obs_{min}}$:

$$\alpha = \begin{cases} k_\alpha \delta_{obs_{min}} & \text{if } \delta_{th_H} < \delta_{obs_{min}} < \delta_{th_L} \\ \alpha_{min} & \text{if } \delta_{obs_{min}} < \delta_{th_L} \\ \alpha_{max} & \text{if } \delta_{obs_{min}} > \delta_{th_H} \end{cases} \tag{5.23}$$

with the minimum and maximum step size $\alpha_{min} > 0$, $\alpha_{max} > 0$, lower and higher distance threshold δ_{th_L}, δ_{th_H} and scaling factor $k_\alpha \in [0, 1]$. The implemented functions of the rotational factor $\tilde{k}$, influence radius d_0, and step size α are visualized in Figure 5.4.

Additionally, logic has been implemented to handle cases with large values in the resulting forces and reduce overshooting and oscillation. When the target is at a significant distance, the attractive force becomes exceedingly potent, and the repulsive force increases with the number of obstacles and decreasing distance to obstacles. To manage this, a threshold F_{max} has been set, and a scaled normalization with the scaling factor $\hat{\rho} \in [0, 1]$ is applied for smooth flight behavior. To prevent oscillations and to overshoot near the target position, the rotational scaling factor $k_{\tilde{k}}$, influence radius d_0, and step-size α are reduced with decreasing distance to the goal. The complete algorithm is presented in Algorithm 1.



(a) Rotational factor $\tilde{k}$ (b) Influence Radius d_0 (c) Step size α

Figure 5.4.: Visualization of the rotational factor, influence radius, and step size.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

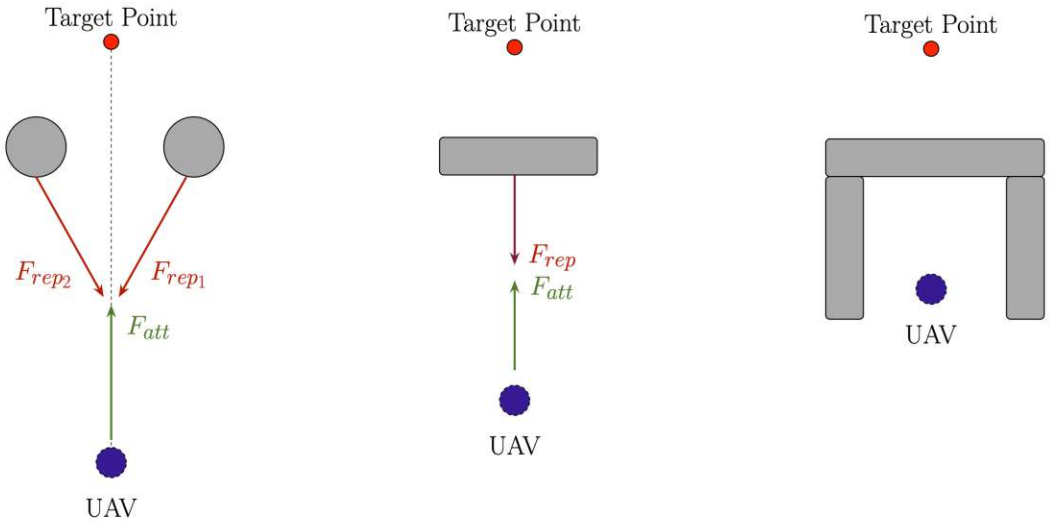
Local Minima

A significant limitation of the APF is the possibility of the UAV getting trapped in local minima. Typical local minima situations are illustrated in Figure 5.5. In such scenarios, the components of the resulting forces from the APF cancel each other out, leaving the UAV unable to progress toward its goal. This poses a significant challenge in highly symmetrical or complex obstacle configurations. To overcome the failure scenario 5.5a the adaptive influence radius (Equation 5.22) is implemented, for 5.5b the rotational component (Equation 5.19) offers a remedy, but in 5.5c the algorithm is trapped in any event. Several authors proposed that features from other approaches can be utilized together to escape this local minima situation. For example, the concept of combining the WFM and APF was introduced by Yun and Tan in 1997 [72]. This combination ensures that when one method fails due to its limitations, the other can take over the path-planning task. This compensation mechanism between the two methods leads to better paths than using each method individually. Researchers have found that this combined method, equipped with appropriate situation awareness, can offer superior path planning outcomes [40, 111, 112]. This work utilizes the Bug2 algorithm, the same method proposed in [41] for ground vehicles. To detect if the UAV is trapped in a local minimum, the following two conditions are checked:

$$\|\mathbf{F}_{att} + \mathbf{F}_r\| < F_{th} \tag{5.24}$$

$$\forall i \in \{1, ..., k\} : \mathbf{p}_i < \delta_{th} \tag{5.25}$$

where $\mathbf{F}_{att} + \mathbf{F}_r$ is the cumulative sum of the artificial potential force, $F_{th} > 0$ is a threshold parameter, $\mathbf{p}_i$ is the UAV's position at the time i, $k > 0$ is the number of considered positions and $\delta_{th} > 0$ is a threshold distance. If one of these conditions is met, the UAV is likely trapped, and the algorithm switch is performed.



(a) Symmetric obstacles (b) Balance of forces (c) U-shaped obstacle

Figure 5.5.: Typical APF local minima situations.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Algorithm 1 APF+ Algorithm

procedure APF+($\mathbf{p}$, $\mathbf{p}_{goal}$)
  $\mathbf{F}_r \leftarrow 0$
  $\mathbf{F}_a \leftarrow 0$
  $\tilde{\mathbf{F}}_r \leftarrow 0$
  while $\|\mathbf{p} - \mathbf{p}_{goal}\| \geq \delta_{threshold}$ **do**
    $d_0 \leftarrow$ (*Equation* 5.22)
    $\mathbf{F}_a \leftarrow$ (*Equation* 5.9)
    for all obs_i **do**
      if $\delta_{obs_i}(\mathbf{p}) < d_0$ **then**
        $\mathbf{F}_r \leftarrow \mathbf{F}_r + \mathbf{F}_{r_i}$ by (Equation 5.13)
      end if
    end for
    $\mathbf{F}_r \leftarrow \mathbf{F}_r + \mathbf{F}_{r_z}$ by (Equation 5.15)
    $\tilde{k} \leftarrow$ (*Equation* 5.20)
    $\tilde{\mathbf{F}}_r \leftarrow$ (*Equation* 5.19)
    $\mathbf{F}_r \leftarrow \mathbf{F}_r + \tilde{\mathbf{F}}_r$
    if $\|\mathbf{F}_r\| \geq F_{max}$ **then**
      $\mathbf{F}_r \leftarrow sign(\mathbf{F}_r) F_{max}$
    end if
    $\mathbf{F} \leftarrow \mathbf{F}_a + \mathbf{F}_r$
    if Equation 5.24 **or** Equation 5.25 **then** ▷ Local Minima
      switch algorithm
    end if
    if $\|\mathbf{F}\| > \hat{\rho}$ **then**
      $\mathbf{F} \leftarrow \hat{\rho} \frac{\mathbf{F}}{\|\mathbf{F}\|}$
    end if
    $\alpha \leftarrow$ (*Equation* 5.23)
    $\mathbf{p} \leftarrow \mathbf{p} + \alpha \mathbf{F}$
  end while
end procedure

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

5.2.2. Dynamic Window Approach

This section presents the implementation and mathematical formulation of the Dynamic Window Approach. The proposed implementation is based on the principle of the original work [75] with adaptions to 3D and various modifications and considering the quadcopter UAV as a holonomic robot. Calculating a new collision-free trajectory includes predicting all reachable positions within the following time step and evaluating the best one based on several criterion factors.

Motion Equations of the UAV

The holonomic property of a quadcopter enables immediate movement in every direction in space within one time step Δt. The approximated discrete positions x_{k+1}, y_{k+1} and z_{k+1} are given as:

$$\begin{aligned} x_{k+1} &= x_k + v \cdot \Delta t \cdot \cos(\psi) \\ y_{k+1} &= y_k + v \cdot \Delta t \cdot \sin(\psi) \\ z_{k+1} &= z_k + v_z \cdot \Delta t \end{aligned} \tag{5.26}$$

where v is the norm of the velocity in the xy plane, v_z the velocity in z direction and $\psi \in [-\pi, \pi]$ the yaw angle.

Dynamic Window Search Space

In the velocity space exists an endless variety of velocity pairs (v, v_z), representing linear velocities. The search space, a subset of the velocity space, reduces these possibilities based on the UAV's moving capabilities and the restrictions of the environment. The constant constraints of the UAV are given by its linear velocity limits v_{min}, v_{max} and $v_{z_{min}}$, $v_{z_{max}}$ define the overall search space:

$$V_\mathrm{s} = \{(v, v_z) \,|\, v \in [v_\mathrm{min}, v_\mathrm{max}],\ v_z \in [v_{z_\mathrm{min}}, v_{z_\mathrm{max}}]\}\,. \tag{5.27}$$

The limitations of the UAV's motor torque impact the maximum and minimum speeds it can achieve in a specific time frame. These constraints further restrict the dynamic window. With the given UAV's current velocities v and v_z, the dynamic window for the next period is defined as:

$$V_a = \{(v, v_z) \,|v \in [v - \dot{v}_b \Delta t, v + \dot{v}_a \Delta t]\,,\ v_z \in [v_z - \dot{v}_{z_b} \Delta t, v_z + \dot{v}_{z_a} \Delta t]\} \tag{5.28}$$

with the values $\dot{v}_a, \dot{v}_{z_a}$ for accelerations, the breaking accelerations $\dot{v}_b, \dot{v}_{z_b}$ and the sampling time Δt. Finally, the pair of values representing the speeds at which the vehicle can stop without hitting any obstacles are defined in Equation 5.29, where δ_{min} denotes the minimum distance between the UAV and the closest obstacle along the path. These are called obstacle velocities V_o and form a dynamic window centered around the vehicle's current speed.

$$V_o = \left\{(v, v_z) \,|v \leq \sqrt{2\delta_{min}\dot{v}_b},\ v_z \leq \sqrt{2\delta_{min}\dot{v}_{z_b}}\right\} \tag{5.29}$$

The resulting search space, the dynamic window V_d, is represented as intersection of Equation 5.27, Equation 5.28 and Equation 5.29 as follows:

$$V_d = V_s \bigcap V_a \bigcap V_o. \tag{5.30}$$

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Scoring Function
As evaluation criteria of the trajectories, the scoring function is defined as:

$$J(v, v_z, \psi) = \lambda \left[\alpha \cdot goal(v, v_z, \psi) + \beta \cdot dist(v, v_z, \psi) + \gamma \cdot vel(v, v_z) + \sigma \cdot alt(v_z)\right] \tag{5.31}$$

This scoring function considers the $goal(v, v_z, \psi)$ criteria, indicating the progress of arriving at the target, the distance $dist(v, v_z, \psi)$ representing the clearance criteria related to the proximity of obstacles interfering with the path and the velocity criteria, represented by the $vel(v, v_z)$ function, is used to rate the robot's speed along the specified trajectory. The altitude criteria $alt(v_z)$ is added to consider flight altitude and obstacles in the z-direction. The parameters α, β, γ and $\sigma > 0$ are weighting the criteria functions, and λ serves as a parameter for normalization.

The *goal* criteria is defined as:

$$goal(v, v_z, \psi) = \|\mathbf{p}_{sim}(v, v_z, \psi) - \mathbf{p}_{\text{goal}}\| \tag{5.32}$$

where $\mathbf{p}_{sim}(v, v_z, \psi) \in \mathbb{R}^3$ represents the simulated position with respect of (v, v_z, ψ) and $\mathbf{p}_{goal} \in \mathbb{R}^3$ is the target position. To consider the obstacles, the dist criteria function Equation 5.33 is implemented. To ensure collision with any obstacle, the function value is set to infinity if the simulated obstacle distance $\delta(v, v_z, \psi)$ is below a certain threshold distance δ_{th} or exponential increasing with the obstacle distance and $k > 0$ when within the influence radius d_0.

$$dist(v, v_z, \psi) = \begin{cases} \infty & \text{if } \delta_{obs}(v, v_z, \psi) < \delta_{th} \\ k \exp\left(\delta_{obs}(v, v_z, \psi)\right) & \text{if } \delta_{obs}(v, v_z, \psi) < d_0 \\ 0 & \text{else} \end{cases} \tag{5.33}$$

The velocity is scored by the *vel* criterion Equation 5.34, considering the simulated velocities v, v_z and corresponding maximal velocities $v_{max}, v_{z_{max}}$.

$$vel(v, v_z) = \|v - v_{max}\| + \|v_z - v_{z_{max}}\| \tag{5.34}$$

Finally, analog to Equation 5.33, the *alt* criterion Equation 5.35 is implemented to ensure constant flight altitude and to avoid detected obstacles from the up- and down-facing sensors.

$$alt(v_z) = \begin{cases} \infty & \text{if } \delta_{obs}(v_z) < \delta_{z_{th}} \\ k_z(\delta_{obs}(v_z))^2 & \text{if } \delta_{obs}(v_z) < d_0 \\ 0 & \text{else} \end{cases} \tag{5.35}$$

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Adding the altitude as an extra dimension significantly increases the complexity and computational demands of the algorithm. To keep the computational demand as low as possible, only eight discrete directions in equal 45° intervals are considered for four velocities in the xy plane, shown in Figure 5.7a. This still enables smooth movements in all directions while reducing calculation overhead. Since the UAV should generally maintain a constant altitude, the v_z velocity window is also considered with just three velocity samples, enabling sufficient avoidance in the vertical direction while keeping the search space as low as possible. The resulting omnidirectional velocity search space with its $8 \cdot 4 \cdot 3 = 96$ possible velocities and directions is illustrated in Figure 5.6, which is further restricted if close obstacles are present. To achieve movements through confined spaces, the influence radius d_0 and threshold radius d_{th} for obstacles, the scaling parameter k, and the criteria weights parameter α, β, γ and $\sigma > 0$ need to be set very carefully.

The DWA generally avoids local minima more effectively than the APF method. However, in certain specific configurations, the DWA can also fail to reach the target, such as when navigating into a large or U-shaped obstacle as depicted in Figure 5.5c. A transition to the Bug2 algorithm is initiated to address this, analogous to the approach taken with APF+. The mathematical representation of this scenario is:

$$\forall i \in 1, ..., k : \mathbf{p}_i < \delta_{th} \tag{5.36}$$

just like in Equation 5.25.

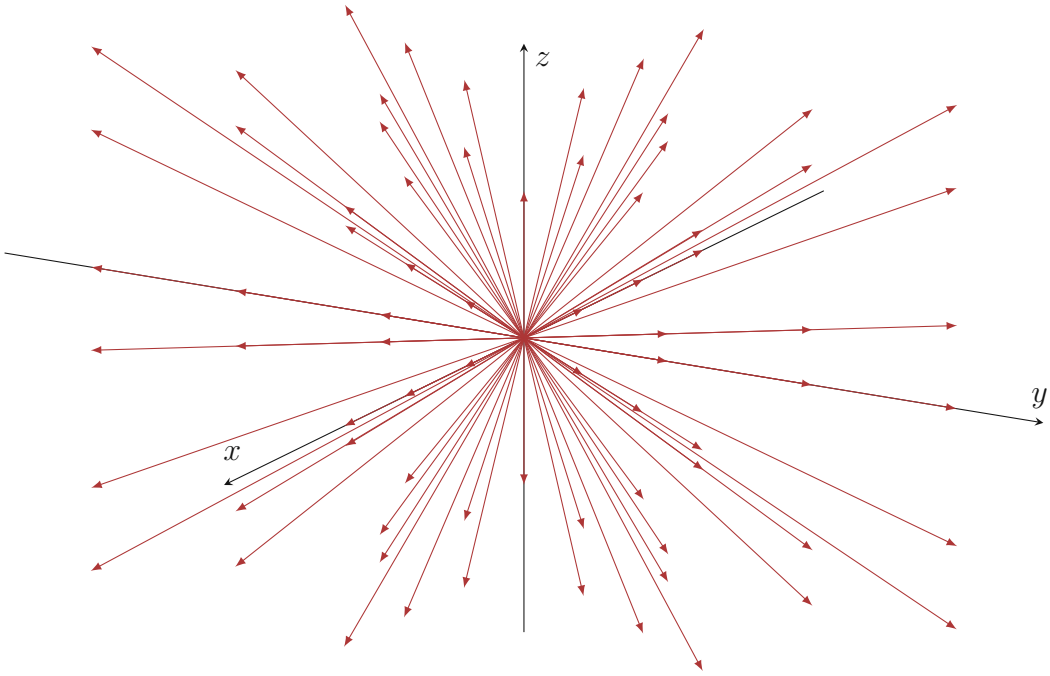


Figure 5.6.: DWA velocity search space in 3D.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Algorithm 2 Proposed Dynamic Window Approach

procedure DWA($\mathbf{p}$, $\mathbf{p}_{goal}$)
  $\mathbf{V}_s \leftarrow$ Equation 5.27
  $\mathbf{V}_a \leftarrow$ Equation 5.28
  $\mathbf{V}_o \leftarrow$ Equation 5.29
  $\mathbf{V}_d \leftarrow \mathbf{V}_s \cap \mathbf{V}_a \cap \mathbf{V}_o$
  Initialize scoring function J_{best}
  Initialize scoring parameters $\lambda, \alpha, \beta, \gamma, \sigma$
  while $||\mathbf{p} - \mathbf{p}_{goal}|| \geq \delta_{threshold}$ **do**
    for all $(v, v_z) \in \mathbf{V}_d$ **do**
      for all ψ_i **do**
        $\mathbf{p}_{sim} \leftarrow$ Simulate Position (Equation 5.26)
        for all *obsi* **do**
          $dist(v, v_z, \psi_i) \leftarrow$ (*Equation* 5.33)
        end for
        $goal(v, v_z, \psi_i) \leftarrow$ (*Equation* 5.32)
        $vel(v, v_z) \leftarrow$ (*Equation* 5.34)
        $alt(v_z) \leftarrow$ (*Equation* 5.35)
        $J \leftarrow$ (Equation 5.31)
        if $J \leq J_{best}$ **then**
          $J \leftarrow J_{best}$
          $\mathbf{v}_{best}, \mathbf{v}_{z_{best}}, \psi_{best} \leftarrow v, v_z, \psi_i$
        end if
      end for
    end for
    if Equation 5.36 **then** ▷ Local Minima
      switch to Bug2 algorithm
    end if
    $\mathbf{p} \leftarrow$ Calculate Position ($\mathbf{v}_{best}, \mathbf{v}_{z_{best}}, \psi_{best}$) (Equation 5.26)
  end while
end procedure

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

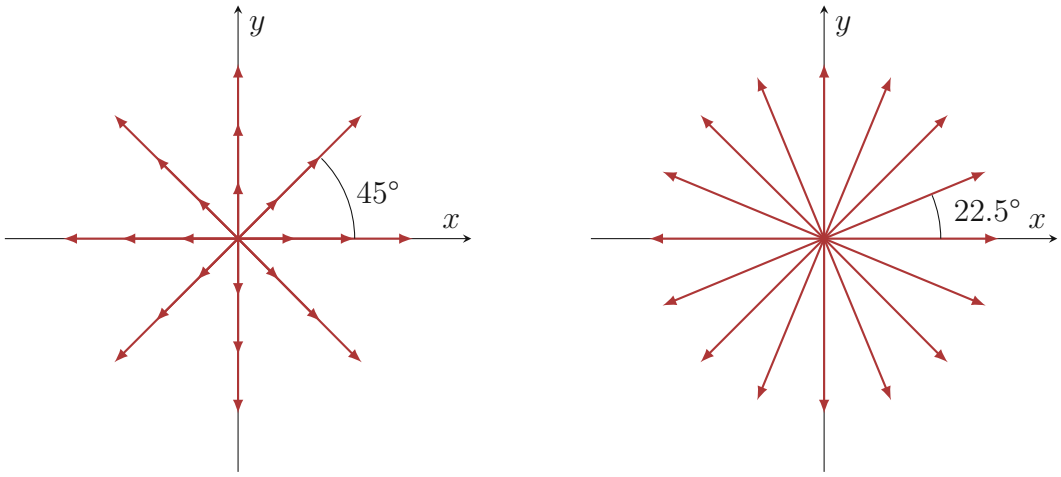
5.2.3. Bug2

The Bug2 algorithm is not implemented for standard path planning or comprehensive obstacle avoidance in this work. Instead, it serves as a backup algorithm to overcome local minima issues encountered in both the APF+ method and DWA, as depicted in Figure 5.5c and Figure 5.5b. The algorithm directs the UAV along the $\mathbf{m}_{\text{line}}$ until an obstacle is encountered. In our fallback mechanism, the full $\mathbf{m}_{\text{line}}$-following procedure often is not executed due to the proximity of obstacles when the UAV becomes trapped. The algorithm then determines the direction of the obstacle and initiates a circumnavigation in a predefined direction while maintaining a constant distance from the obstacle. It reverts to the APF or DWA method upon re-encountering the $\mathbf{m}_{\text{line}}$. Switching back to APF or DWA based on a decreased target distance, as proposed by [40], often led to an endless loop of entrapment and escape attempts in many scenarios.

It is important to note that this algorithm is included primarily to address local minima situations. However, it is not employed in the final UAV's implementation with the global planner, which already serves as a mechanism to resolve such issues. We experimented with altering the wall-following direction based on the goal's location, but tests show that a fixed-direction approach is more effective. To ensure safe navigation around obstacles, we maintain the minimal obstacle distance within the thresholds $\delta_{threshold_-} \leq \delta_{obs_{min}} \leq \delta_{threshold_+}$ during avoidance maneuvers. In our implementation, the Bug algorithm simplifies the environmental analysis by considering only 16 distinct positions, as illustrated in Figure 5.7b, categorizing detected obstacle directions into these positions.

The Bug2 algorithm is triggered under specific conditions using either the APF or DWA methods. For the APF method, Bug2 is activated when the condition specified in Equation 5.24 or Equation 5.25 is met. In the context of the DWA method, Bug2 takes control if the system meets the criteria outlined in Equation 5.36. The implemented pseudo-code of the Bug2 algorithm is provided in Algorithm 3.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.



(a) DWA velocity search space in 2D (b) Bug2 discrete directions

Figure 5.7.: Discrete sample directions in DWA and Bug2 method

Algorithm 3 Bug2 Algorithm

procedure BUG2($\mathbf{p}_{start}$, $\mathbf{p}_{goal}$)
  compute $\mathbf{m}_{\text{line}}$ from $\mathbf{p}_{start}$ to $\mathbf{p}_{goal}$
  move towards $\mathbf{p}_{goal}$ following $\mathbf{m}_{\text{line}}$ with step size α
  while $\|\mathbf{p} - \mathbf{p}_{goal}\| \geq \delta_{threshold}$ **do**
    if obstacle encountered $\delta_{obs_{min}} \leq \delta_{threshold}$ **then**:
      $\mathbf{p}_{hit} \leftarrow \mathbf{p}$
      while $\|\mathbf{p} - \mathbf{m}_{\text{line}}\| \geq \delta_{threshold}$ **do**
        Calculate $\delta_{obs_{min}}$ and its direction
        Circumnavigate obstacle maintaining $\delta_{threshold_-} \leq \delta_{obs_{min}} \leq \delta_{threshold_+}$
      end while
      switch algorithm ▷ Local Minima Resolved
      break
    end if
    move towards $\mathbf{p}_{goal}$ following $\mathbf{m}_{\text{line}}$ with step size α
  end while
end procedure

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

6. Experiments & Results

In this section, the results of the proposed implementations are presented, analyzed, and compared. A simulation environment is set up, and all the implemented logic is extensively tested in simulation. Therefore, all the hardware, sensors, and other components must be designed in simulation software to test the system properly. A complete simulation setup includes test environments to run and test the algorithms. Finally, Hardware in the Loop (HITL) and real-world experiments tests are performed to validate the simulation results.

6.1. Simulation Environment

Since simulation is an essential step in the process of module testing before and during the actual development of a collision avoidance algorithm, Gazebo 11 [113], an open-source physics engine simulator, is set up using the ArduPilot Software in the Loop (SITL) simulation, ROS, and the MAVROS package. Simulation allows detailed testing in a virtual environment without needing a working real platform and provides debugging data. Gazebo facilitates using the same ArduPilot firmware and MAVLink in the simulation as it is deployed in the actual platform. In Gazebo, a variety of ROS plugins exists for models and sensors, including plugins to precisely model the used LiDAR and ToF sensors, IMU data, and other avionics. Gazebo supports using custom environments, which is crucial for testing scenarios that are hard to replicate in the real world or require specific conditions. The test environments are modeled in Blender [114], and all the simulation tests are performed on a workstation with a 4.2GHz Intel Core i7 processor, 16GB RAM, and a Radeon Pro 570 simulating with a Real-Time Factor of 0.7-0.9.

The open-source *3DR Iris-base* [1] UAV is utilized as a simulation model of the quadcopter, shown in Figure 6.1. It shows approximately the same properties in size and shape as the actual platform and is commonly used by quadcopter developers. The flight dynamics and characteristics, namely the lift, drag, and moment, can be adapted by changing motor coefficients and the properties of the propellers using the *libLiftDragPlugin.so* library. The Cyglidar D1 sensors and the up-and down-facing Benewake TFmini sensors are modeled with the *libgazebo-ros-velodyne-laser.so* and *libgazebo-ros-laser.so* libraries. The open-source autopilot software suite *libArduPilotPlugin.so* provides the plugin for ArduPilot. This plugin allows the simulation of the drone in the Gazebo environment and communicates with the ArduPilot software running on a simulated flight controller.

[1] https://www.arducopter.co.uk/iris-quadcopter-uav.html

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.
TU Bibliothek Your knowledge hub WIEN

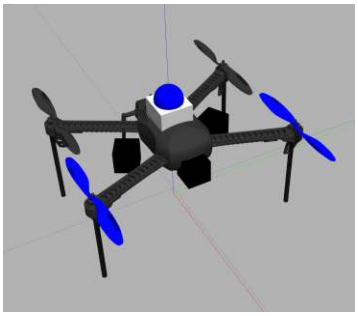
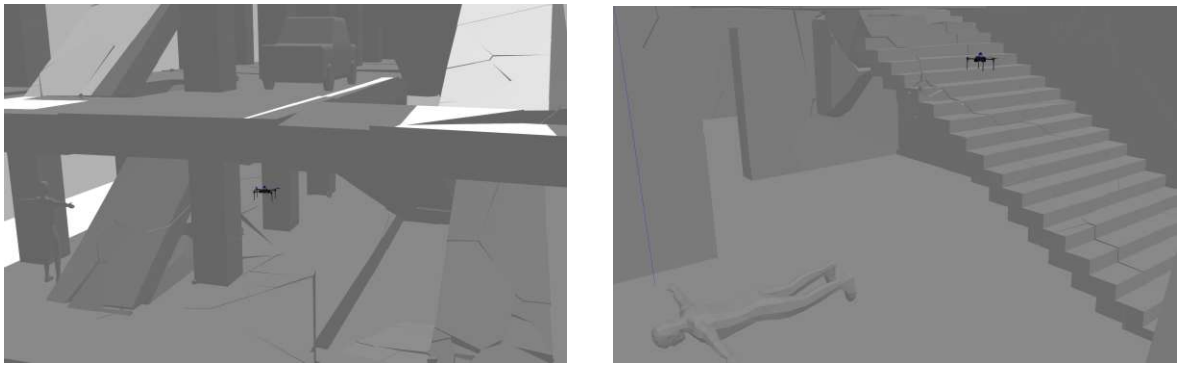


Figure 6.1.: Gazebo UAV simulation model.

The algorithms are tested in a custom simulation environment illustrated in Figure 6.3 and in environments provided by the *Destruction Scenario Dataset* [2]. The destroyed environments, showcased in Figure 6.2, are particularly relevant as they accurately represent the scenarios the algorithms are intended for. The custom test environment covers all possible obstacle configurations the UAV should successfully avoid. Figure 6.3b displays the environment in the x-y plane. Here, L represents an obstacle of low height, M medium height, while H signifies elevated obstacles or obstacles that are not grounded.

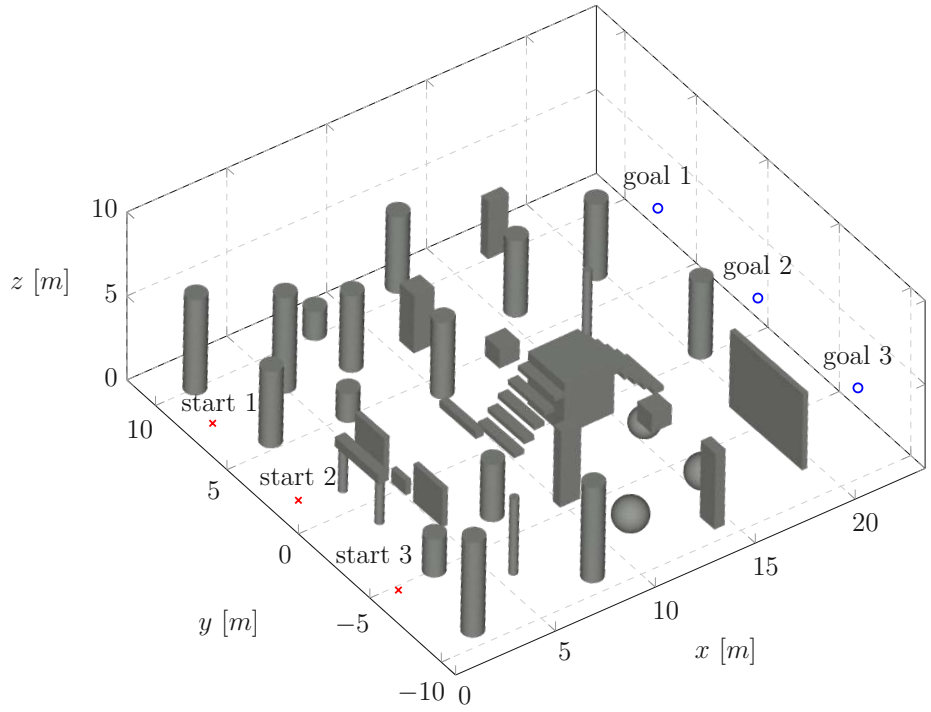


(a) Destroyed Garage (b) Destroyed House

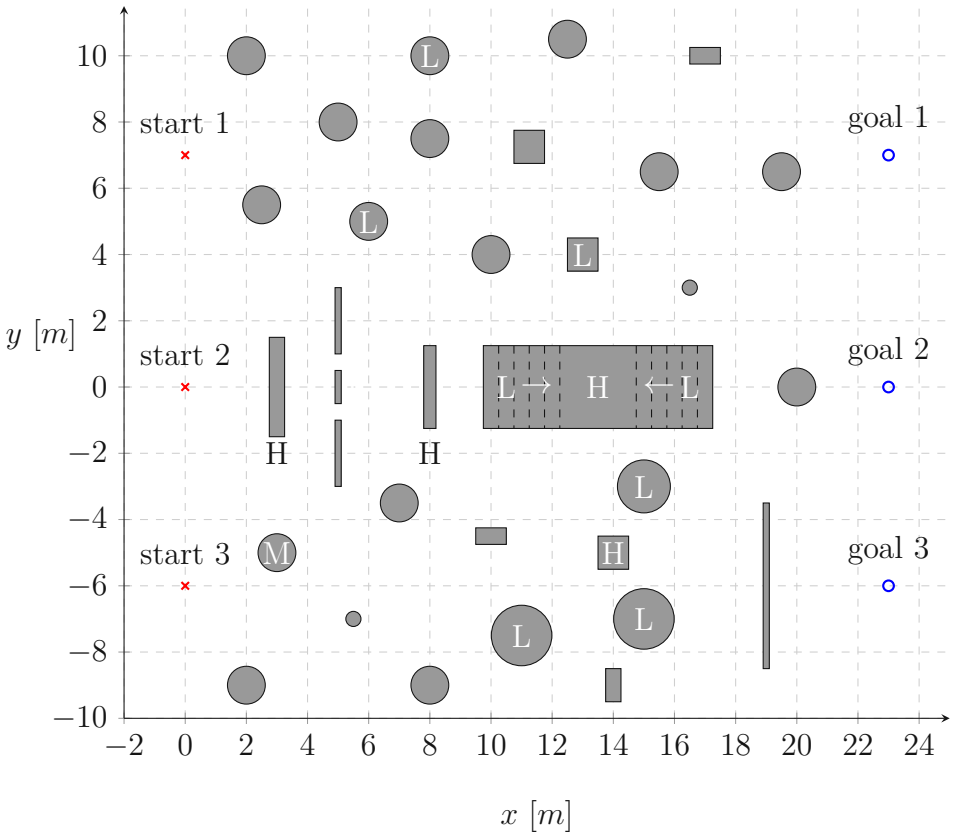
Figure 6.2.: Gazebo simulation environments from Destroyed Scenarios.

[2] https://github.com/roboptics/destruction_scenarios

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.



(a) Test environment in 3D



(b) Test environment in x-y Plane

Figure 6.3.: Custom test environment with start and goal positions. L indicates low-height, M medium-height, and H elevated/non-grounded obstacles.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

6.2. Simulation Results

In this section, the simulation results of the implemented algorithms are presented. Comparative avoidance paths are illustrated to evaluate the differences between the approaches. Both algorithms successfully avoid obstacles in all three dimensions and generate complete paths toward the desired target position without getting stuck in local minimum situations. In the following, the results of the methods are discussed in detail.

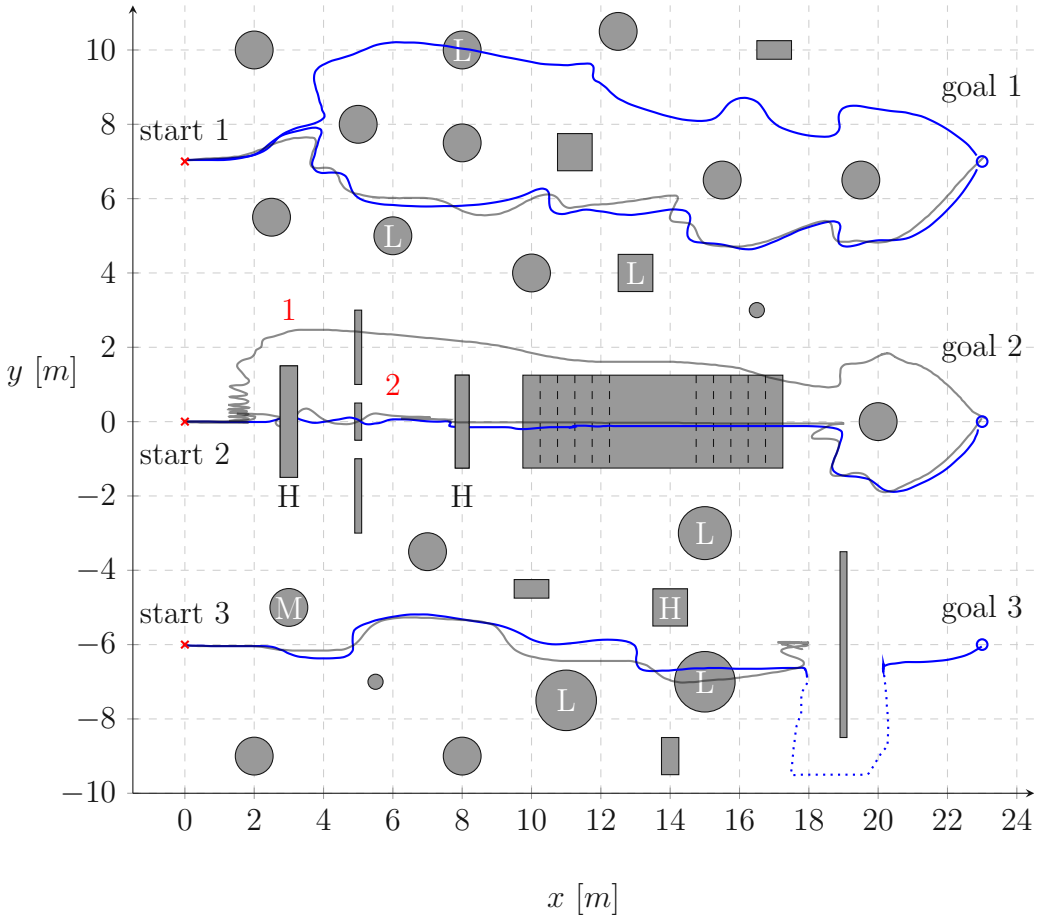
APF+ Method

The proposed APF+ method is compared against the plain implementation to evaluate our implementation. Here, 'plain' refers to the APF method expanded to three dimensions but without the adaptive behavior and rotational component, which are key features of our advanced implementation. Our method is designed to increase the smoothness, in other words, to decrease the number of repelling movements when encountering an obstacle, decrease the overshoot when close to the target position, and enable the UAV to pass narrow passages. Figure 6.4 illustrates the avoidance paths from the start positions $\mathbf{p}_{start_i}$ to the goal positions $\mathbf{p}_{goal_i}$, $i \in \{1, 2, 3\}$. Specifically, Figure 6.5a presents these trajectories in the x-y plane, while Figure 6.5b presents them in the x-z plane. Figure 6.5 shows the return paths from the goal positions $\mathbf{p}_{goal_i}$ back to the start positions $\mathbf{p}_{start_i}$.

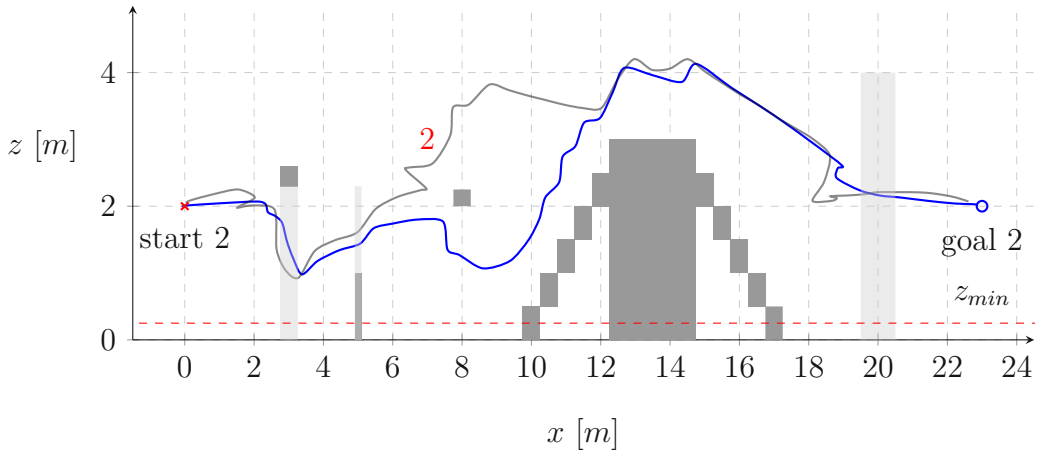
The plain APF method often encounters issues with repelling movements, leading to abrupt changes in direction to avoid obstacles. This reactive behavior slows down the UAV and also increases energy consumption due to generating additional accelerations. A particular challenge arises when the target is located directly behind a large obstacle, potentially causing the UAV to become trapped. Adjusting the influence radius of the obstacles presents a trade-off: a larger radius can hinder the UAV's ability to navigate through narrow passages, as illustrated in Path Figure 6.4 1. Conversely, as seen in Path Figure 6.4 2, a smaller radius allows the UAV to pass through tighter spaces. However, operating the UAV at high speeds with a small influence radius can delay obstacle detection and reaction, leading to potential collisions.

With our implementation, the UAV navigates around all obstacles more smoothly and can maneuver through narrow spaces and climb stairs. It adjusts its altitude to avoid aerial obstacles and smoothly circumnavigates them. It maintains a safe distance from obstacles while ensuring a reliable and steady speed throughout its flight, adjusting its linear speed depending on the minimum detected obstacle distance $\delta_{obs_{min}}$. This ensures safe maneuvering in environments with close obstacles while allowing the UAV to move fast in more open spaces where obstacles are farther away. The implementation of the additional virtual rotational force facilitates smoother navigation around obstacles. This helps avoid abrupt repelling movements and sudden changes in acceleration, which can be energy-intensive and slow down the UAV. Our approach enables the UAV to reach its destination more efficiently, and overshoot is minimized.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.



(a) Avoidance in x-y plane

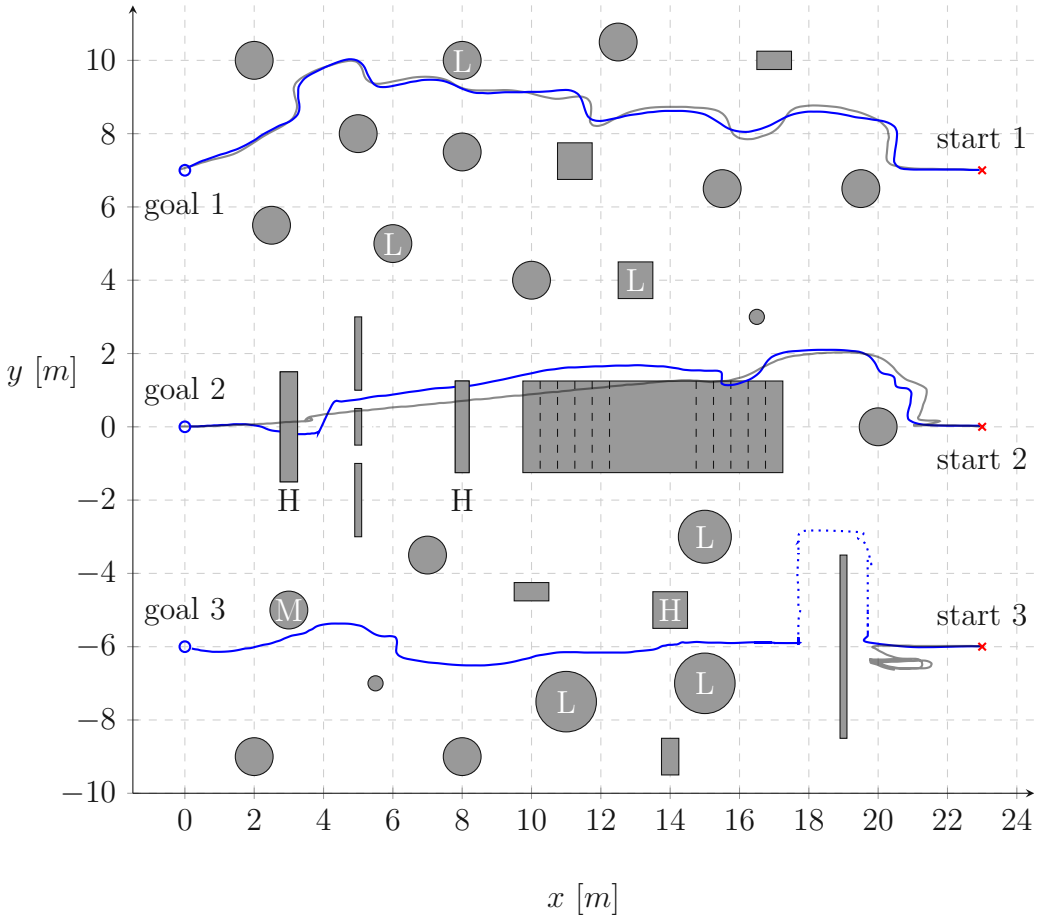


(b) Avoidance in x-z plane

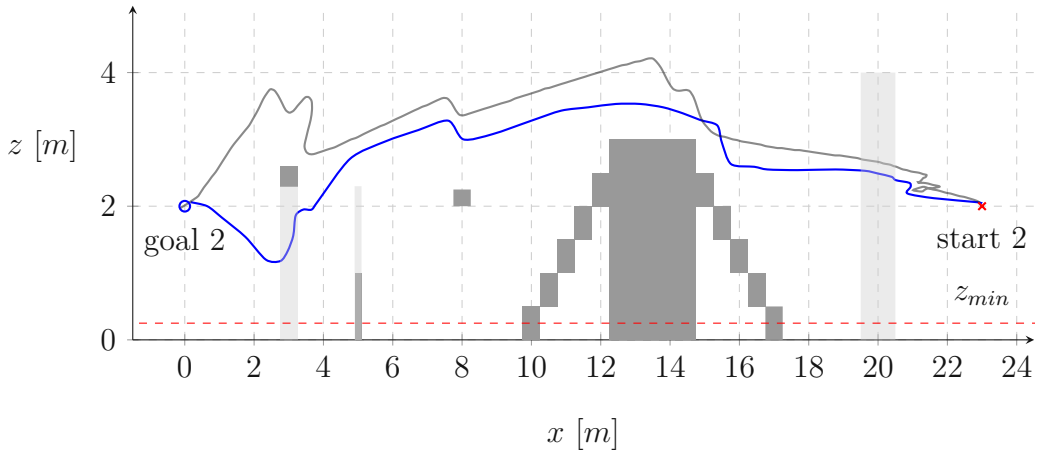
APF+ Plain APF Bug2

Figure 6.4.: Generated APF avoidance paths start → goal

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.



(a) Avoidance in x-y plane



(b) Avoidance in x-z plane

APF+ Plain APF Bug2

Figure 6.5.: Generated APF avoidance paths goal → start

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

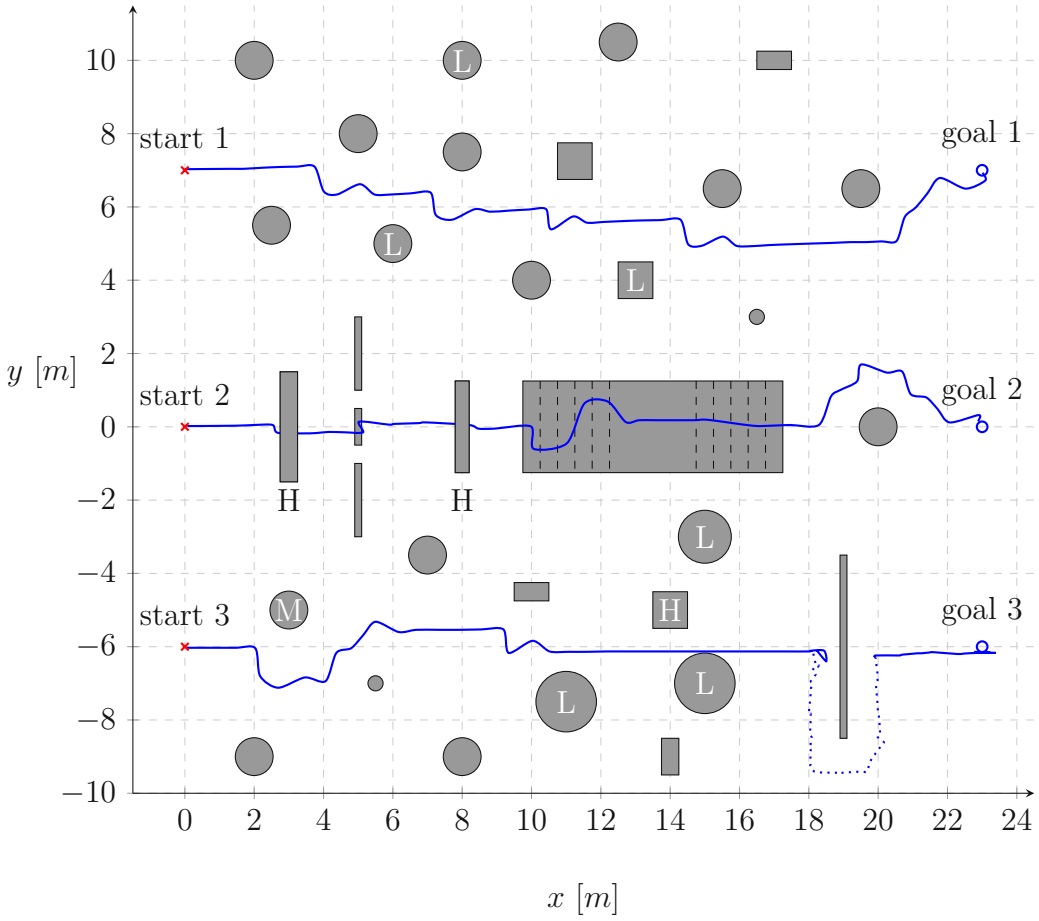
DWA Method

The DWA is also effectively implemented in three dimensions, ensuring the avoidance of all obstacles along its trajectory. Like in the previous section, Figure 6.6 displays the avoidance paths from the starting points $\mathbf{p}_{start_i}$ to the goals $\mathbf{p}_{goal_i}$, plotted in both the xy and xz planes. Additionally, the return paths are illustrated in Figure 6.7.

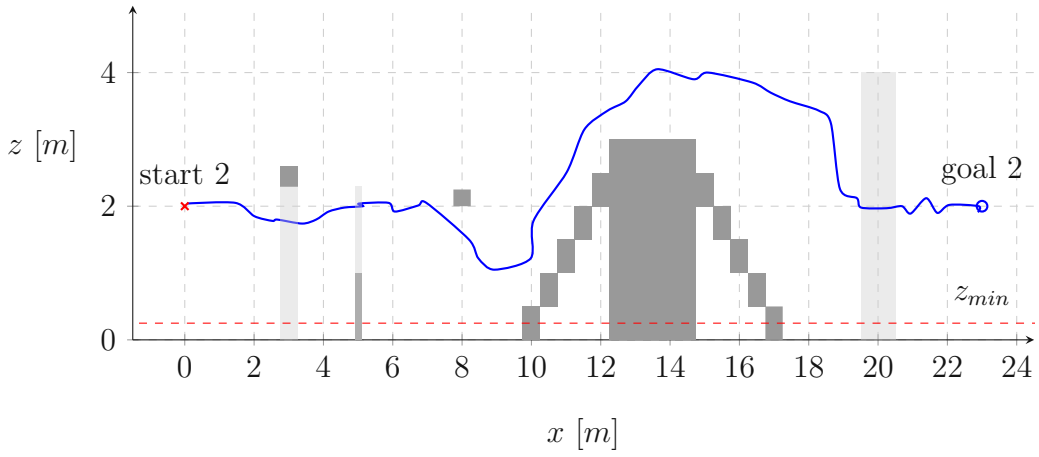
Since only some discrete positions are considered in the proposed implementation, the avoidance paths are not as smooth as in the APF+ method. However, this characteristic is also due to the overall approach of the method. By appropriately adjusting the weighting factors in the scoring function, the UAV's behavior is fine-tuned to effectively navigate through narrow passages and avoid obstacles in the vertical direction. Local minima are successfully escaped by switching to the Bug2 method, as already demonstrated in the APF+ approach. However, the transition to this algorithm switch in the DWA is less seamless than in the APF+ method. In the DWA, the UAV must remain within the vicinity of a position until the switch is executed, while in APF+, the switch is often triggered by the balancing of the attractive and repulsive forces.

Despite these differences, the DWA effectively avoids all obstacles while maintaining a safe distance from them. The system efficiently prevents collisions with very close obstacles, especially in narrow passages, by setting the obstacle scoring function (mentioned in Equation 5.33) to a value ∞ whenever an obstacle is identified within a defined distance threshold, enabling the UAV to allow still movements in narrow spaces but not into the direction of obstacles. Its holonomic consideration allows the UAV to instantly change its flight direction in any direction without first altering its heading. This capability leads to faster responses to dynamic obstacles and safer avoidance maneuvers. However, the DWA sometimes does not reach its target position precisely or is overshooting. An attempt was made to minimize this by reducing the velocity window near the target. However, this adjustment also affected the UAV's performance in following waypoints along a trajectory and was reversed. The DWA sometimes faces difficulty maintaining a steady altitude. Although this issue is not readily observable during simulations, the trajectory shows the fluctuations in altitude, clearly depicted in Figure 6.7b.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.



(a) Avoidance in x-y plane

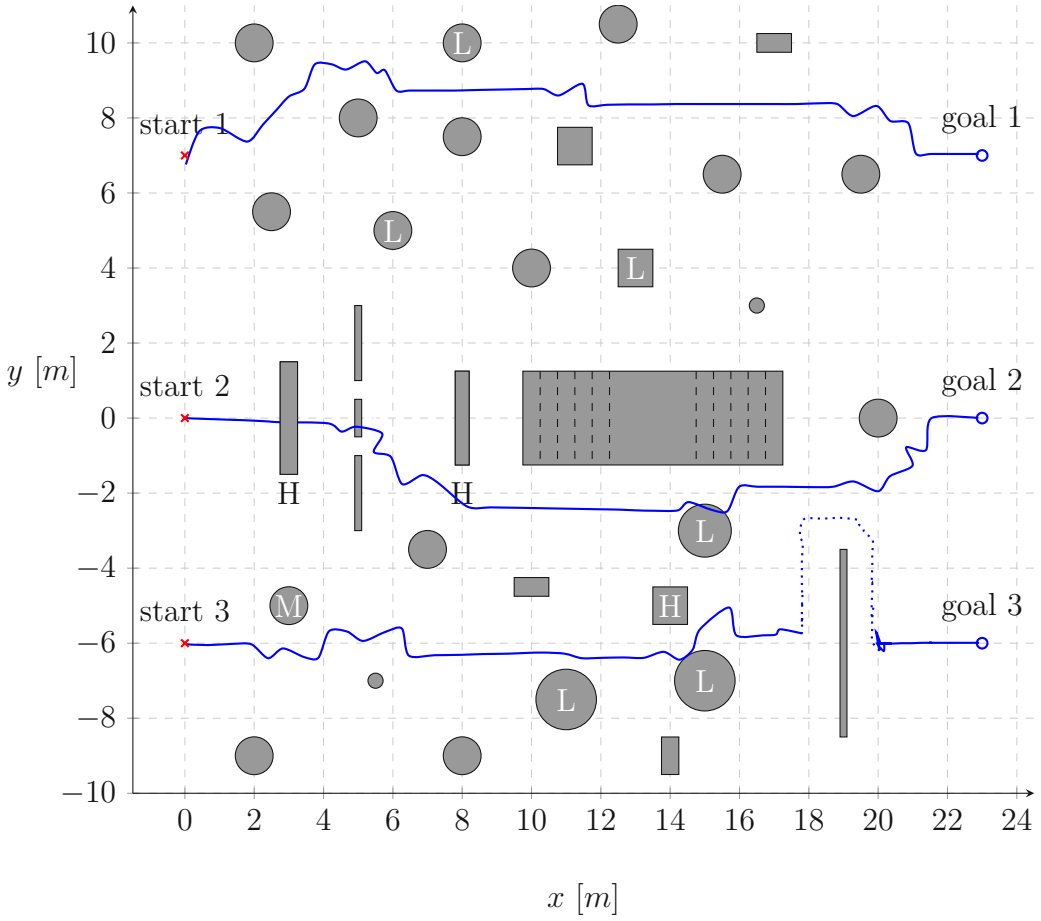


(b) Avoidance in x-z Plane

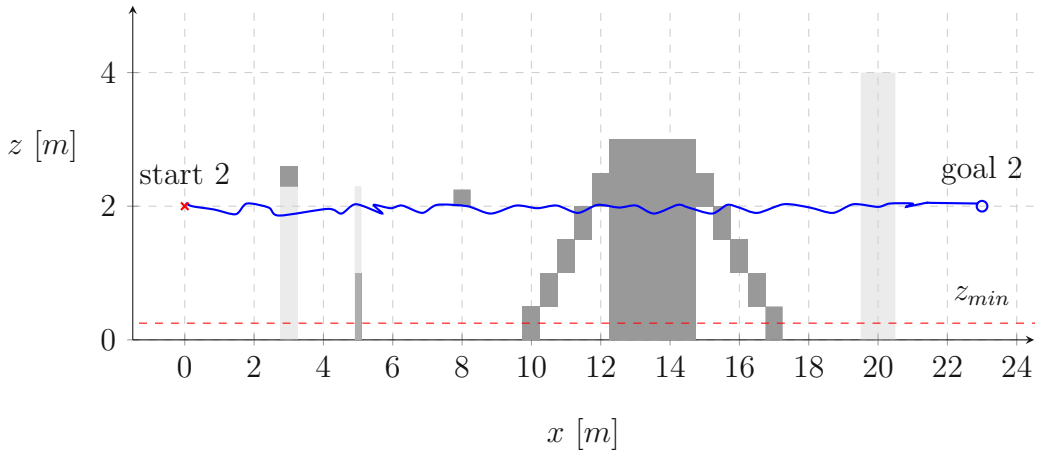
DWA Bug2

Figure 6.6.: Generated DWA avoidance paths start → goal

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.



(a) Avoidance in x-y plane



(b) Avoidance in x-z Plane

DWA Bug2

Figure 6.7.: Generated DWA avoidance paths goal → start

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Algorithm Comparison

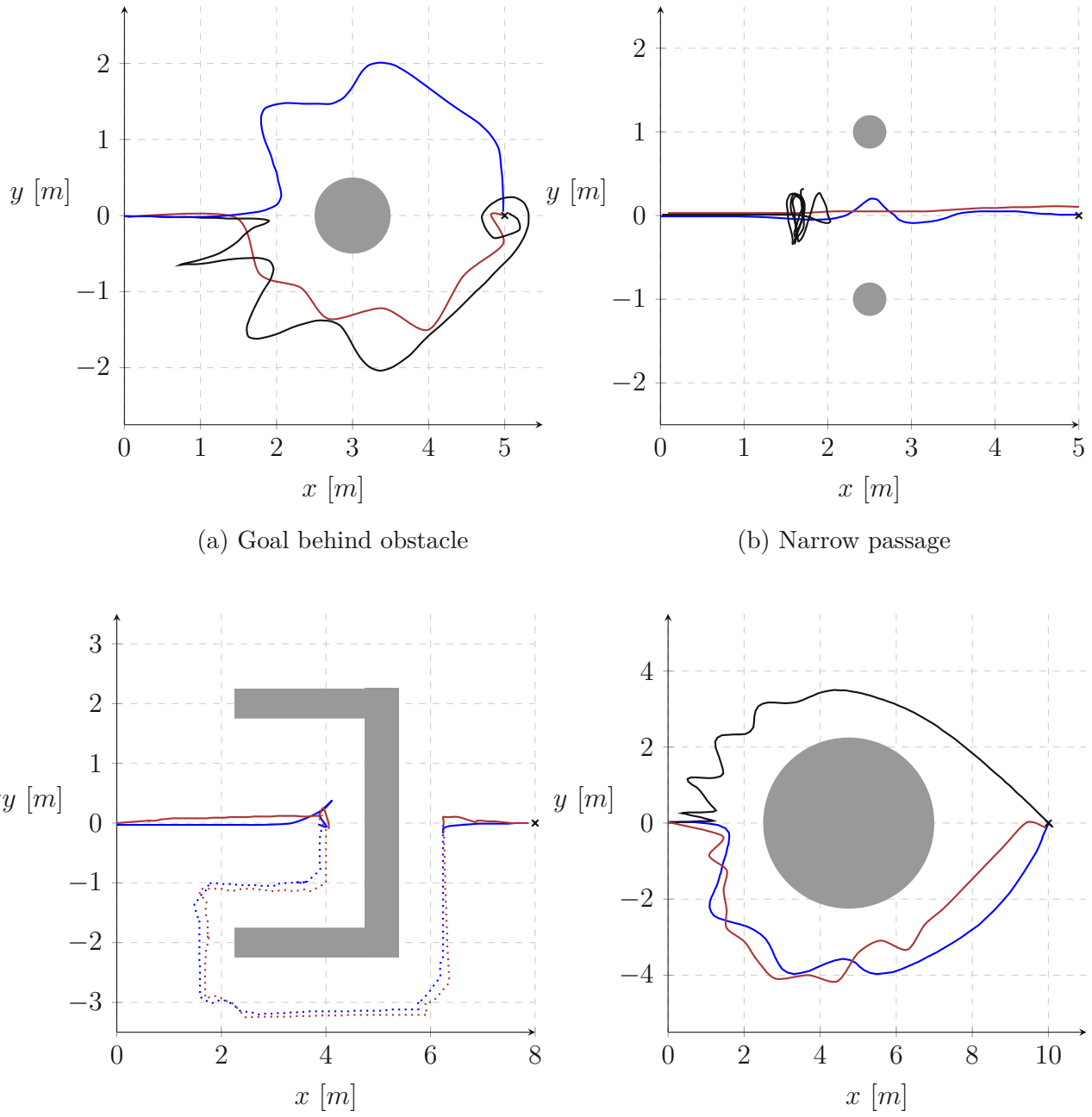
Table 6.2 displays mean values calculated from sets of ≥ 20 measurements for each path. The table includes various performance metrics: the minimum obstacle distance $\delta_{obs_{min}}$ encountered along the entire path, the average obstacle distance $\overline{\delta}_{obs}$, the total length of the avoidance path d_{path}, and the direct or beeline distance d_{direct} between the start $\mathbf{p}_{start}$ and goal $\mathbf{p}_{goal}$ positions. Additionally, the table shows the flight time t, the maximum speed v_{max} achieved along the trajectory, and the average speed $\overline{v}$. It also includes the average execution time $\overline{t}_{exe}$ of the path planning algorithm. For assessing the smoothness of the trajectory, the table provides the Root Mean Square (RMS) Value of Jerk J_{RMS} and the curvature κ_{RMS}, as detailed in section A.1.

As previously discussed, the APF+ method outperforms the DWA in several aspects. The APF+ method creates smoother transitions when switching to the Bug2 algorithm, and it also more effectively reduces overshooting and better maintains a constant altitude. According to Table 6.2, the APF method's execution time $\overline{t}_{exe}$ is nearly ten times faster than that of the DWA. It is important to note that the lower average values observed in paths $3 \leftrightarrow 3$ result from transitioning to the Bug2 algorithm. While the maximum speed achieved using the APF+ method is higher, the average speeds of both methods are comparable. Compared to the plain APF implementation, our implementation of the APF+ method shows speed improvements and smoothness. The acceleration profiles, depicted in Figure 6.9, further illustrate that our APF+ implementation results in the smoothest trajectory, implying reduced energy consumption. Still, each method maintains a sufficient minimal distance from obstacles $\delta_{obs_{min}}$. The lower minimum values observed in path $3 \leftrightarrow 3$ are primarily due to navigating through narrow passages, a situation where the plain APF method often resulted in extended or indirect routes, as illustrated in Figure 6.5a 2.

The distinct behaviors of the algorithms in special configurations are illustrated in Figure 6.8. Specifically, Figure 6.8a demonstrates how the plain APF method sometimes tends to overshoot the target, while the other two implementations - DWA and our enhanced APF+ - reach the goal more directly. However, the DWA encounters slight challenges. Figure 6.8b depicts navigating through a narrow passage. Here, the plain APF method fails to pass, getting caught in a local minimum, whereas the DWA method and our APF+ implementation successfully navigate through. Figure 6.8c showcases the challenge of confronting a U-shaped obstacle, which often poses a trap for most reactive algorithms. This type of local minimum is effectively addressed by transitioning to the Bug2 algorithm, which is activated once specific conditions are met.

Section A.2 presents plots of obstacle distance profiles and avoidance path trajectories within the Destroyed Garage scenario from the *Destruction Scenario Dataset*. Performance metrics related to these plots are detailed in Table A.1 and Table A.2. These results verify the behaviors previously analyzed.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.



(a) Goal behind obstacle

(b) Narrow passage

(c) U-shaped obstacle

(d) Large obstacle

Figure 6.8.: Generated trajectories in special obstacle configurations.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Configuration	$\delta_{obs_{min}}$ m	$\overline{\delta}_{obs}$ m	d_{path} m	d_{direct} m	t s	v_{max} ms^{-1}	$\overline{v}$ ms^{-1}	$\overline{t}_{exe}$ s	J_{RMS} ms^{-3}	κ_{RMS} m^{-1}	
APF+											
a)	0.48	1.33	9.32	5.00	12.50	1.32	0.74	3.85	1.24	0.64	
b)	0.52	1.43	5.97	5.00	9.08	1.62	0.66	2.99	1.16	0.67	
c)	0.42	1.05	24.79	8.00	51.75	1.34	0.48	2.31	1.01	0.61	[1]
d)	0.41	1.36	15.14	10.00	21.50	1.38	0.71	4.48	1.66	0.84	
Plain APF											
a)	0.52	1.29	14.23	5.00	21.50	1.55	0.66	3.57	1.86	0.88	
b)	0.67	–	–	–	–	1.58	–	3.49	–	–	[2]
c)	0.59	–	–	–	–	1.48	–	3.61	–	–	[2]
d)	0.56	1.63	18.35	10.00	28.25	0.66	0.65	3.84	1.83	0.91	
DWA											
a)	0.88	1.41	10.64	5.00	9.75	1.07	0.64	27.47	2.03	0.83	
b)	0.64	1.38	5.53	5.00	6.00	1.09	0.67	18.86	1.81	0.73	
c)	0.48	1.06	24.82	8.00	52.25	0.56	0.47	2.51	0.70	0.63	[1]
d)	0.49	1.16	16.03	10.00	21.75	0.96	0.73	28.79	1.56	0.98	

Table 6.1.: Special configurations trajectory performance metrics

Path	$\delta_{obs_{min}}$ m	$\overline{\delta}_{obs}$ m	d_{path} m	d_{direct} m	t s	v_{max} ms^{-1}	$\overline{v}$ ms^{-1}	$\overline{t}_{exe}$ s	J_{RMS} ms^{-3}	κ_{RMS} m^{-1}	
APF+											
$1 \rightarrow 1$	0.51	1.47	27.03	23.0	33.50	1.51	0.82	3.53	1.12	0.65	
$1 \leftarrow 1$	0.59	1.44	27.01	22.9	35.10	1.41	0.79	3.24	1.09	0.83	
$2 \rightarrow 2$	0.37	1.32	28.60	23.0	43.23	1.43	0.67	3.37	1.93	0.79	
$2 \leftarrow 2$	0.32	1.20	28.37	23.1	46.23	1.50	0.61	3.80	1.63	0.77	
$3 \rightarrow 3$	0.63	1.31	34.6	23.0	55.00	1.44	0.49	3.40	0.96	0.58	[1]
$3 \leftarrow 3$	0.52	1.01	41.5	22.9	52.31	1.39	0.53	3.01	1.02	0.71	[1]
Plain APF											
$1 \rightarrow 1$	0.59	1.40	27.20	23.0	54.00	0.52	0.47	3.30	1.68	0.96	
$1 \leftarrow 1$	0.69	1.44	25.61	22.9	55.00	0.53	0.48	2.58	1.47	0.95	
$2 \rightarrow 2$	0.57	1.65	32.32	23.0	57.50	0.78	0.58	3.14	1.66	0.87	
$2 \leftarrow 2$	0.51	1.88	28.10	22.9	56.50	0.65	0.49	2.31	1.06	0.84	
$3 \rightarrow 3$	0.81	–	–	–	–	0.57	–	3.22	–	–	[2]
$3 \leftarrow 3$	0.61	–	–	–	–	0.62	–	3.31	–	–	[2]
DWA											
$1 \rightarrow 1$	0.68	1.32	29.89	23.0	35.50	1.05	0.66	22.97	1.38	0.87	
$1 \leftarrow 1$	0.62	1.36	33.65	22.9	33.99	1.14	0.69	24.82	1.43	0.84	
$2 \rightarrow 2$	0.32	1.13	32.22	23.0	41.80	1.20	0.54	26.01	1.95	0.82	
$2 \leftarrow 2$	0.34	1.15	39.10	23.0	37.37	1.15	0.71	26.67	1.66	0.86	
$3 \rightarrow 3$	0.41	1.13	41.71	22.9	51.25	1.09	0.44	12.90	1.00	0.73	[1]
$3 \leftarrow 3$	0.39	1.05	44.95	22.9	48.75	1.16	0.43	13.01	1.21	0.79	[1]

Table 6.2.: Test environment trajectory performance metrics (mean values).

[1] Switch to Bug2 performed
[2] Algorithm trapped in local minima

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

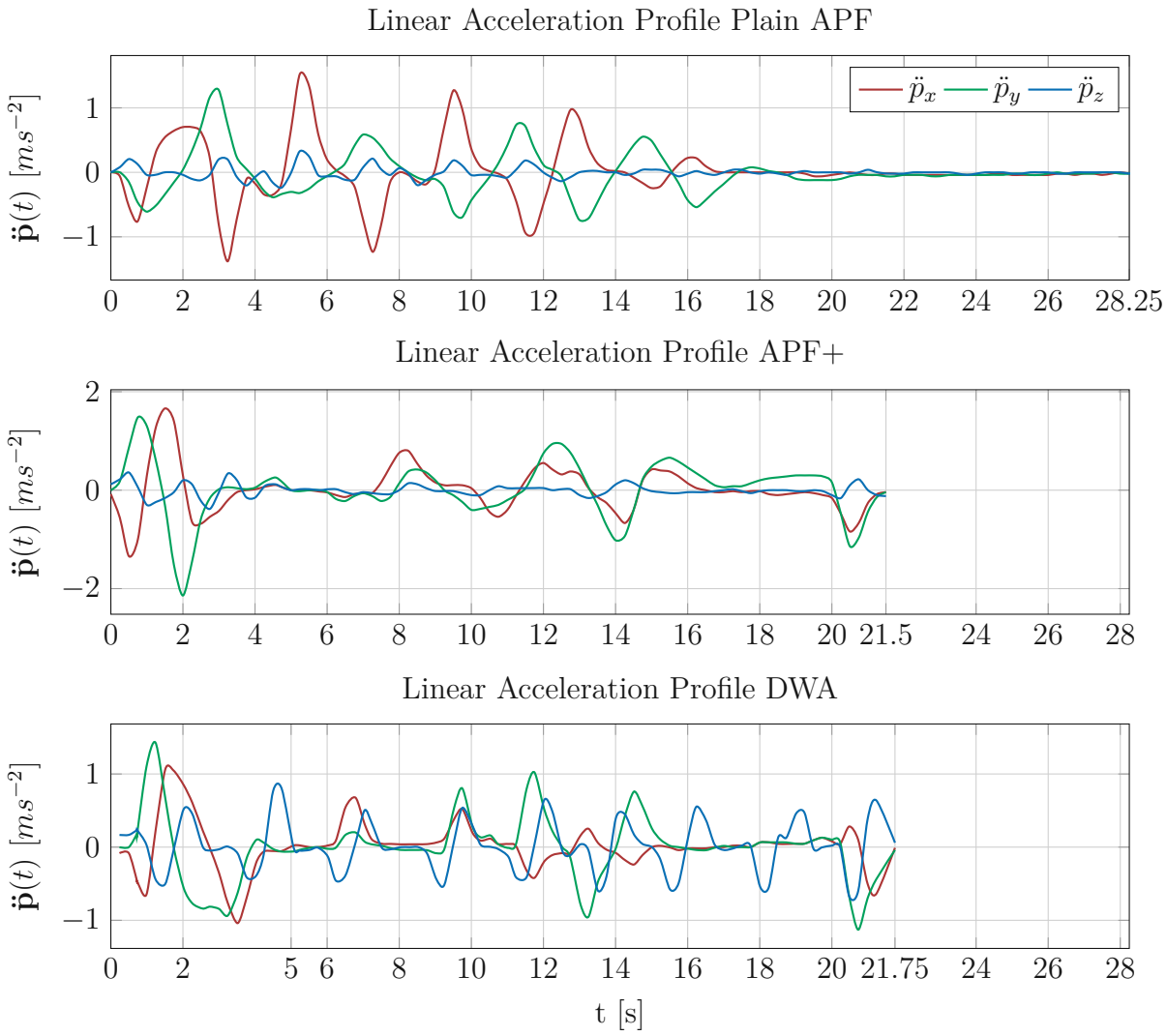


Figure 6.9.: Acceleration profiles for large obstacle configuration (Figure 6.8d).

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

6.3. Hardware in the Loop Tests

HITL tests are conducted to validate the accuracy of the simulated sensor data and the resulting avoidance movements. These tests are conducted on a Raspberry Pi 4 Model B, featuring an ARMv8 1.5GHz processor and 8GB of RAM, serving as the companion computer. The primary objective is to compare actual sensor data with simulated results and test for reliable obstacle detection.

To achieve this, different sized and shaped obstacles are placed around the hovering UAV. The simulation replicates the same arrangement, as depicted in Figure 6.11a. The simulation output is presented in Figure 6.11b, where red points represent the 2D laser scan data, black points represent the generated point cloud, and blue points show the 3D obstacle data extracted from the point cloud.

Given the computational overhead in processing the entire point cloud for avoidance algorithms, a down-sampled representation that still maintains the spatial integrity of the objects is chosen. Various methods are tested, including using only edge and center points. However, Voxel-down sampling is the most effective method of extracting practical 3D obstacle information for various object sizes and shapes. The optimal Voxel size, determined using actual sensor data, is set to 0.75, dramatically reducing the points in the point cloud but still offering good coverage. For the 2D data, point-skipping is implemented, where only every fifth point is used to reduce the overall number of points. Figure 6.11b displays the effectively utilized points. It is important to note that not all these points are used in the avoidance algorithm - only those within the influence radius are considered.

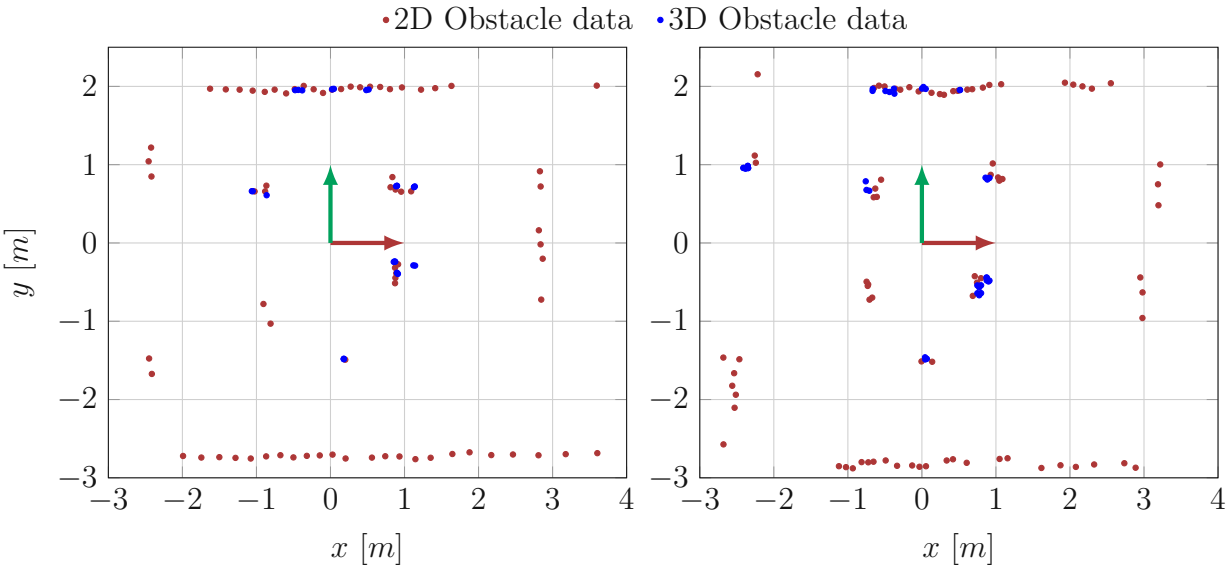
The comparison of actual and simulated sensor outputs in the xy plane for easier visualization is shown in Figure 6.10. A noteworthy observation is that the actual sensor also detects the window corner, which is actually beyond the 3D range specified in the datasheet. The simulated data incorporates Gaussian noise with zero mean and a standard deviation of 0.025 to mimic the actual sensor more accurately. Despite some differences in position, which could be attributed to slight variations in obstacle placement in the real and simulation settings, the outputs closely match. The average time taken for obstacle extraction in the given setting, computed directly on the Raspberry Pi, is $\bar{t}_{extract} = 5.9$ ms.

To assess the performance of both algorithms on the Raspberry Pi, their execution times and CPU usage are measured. The average base load of the companion computer running ROS, with all the launch files for the sensor and communication, is approximately 30%. This includes obstacle extraction, data merging, and MAVLink communication. The APF+ method consumes an average of 33% CPU power (63% in total), with a total average execution time of $\bar{t}_{exe} \approx 50$ ms. In contrast, the DWA requires 64% CPU power (94% in total) and has an average execution time of $\bar{t}_{exe} \approx 250$ ms. Given that the algorithm operates at an update frequency $f = 5$ Hz, this allows only 200 ms per iteration for computation. Therefore, the algorithm must execute significantly below this time frame to ensure sufficient margin for efficient processing.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Based on these findings, the DWA is unsuitable for implementation on our companion computer, making the APF+ method the preferred choice. Nevertheless, this decision aligns with the overall superior performance of the APF+ method compared to the DWA.

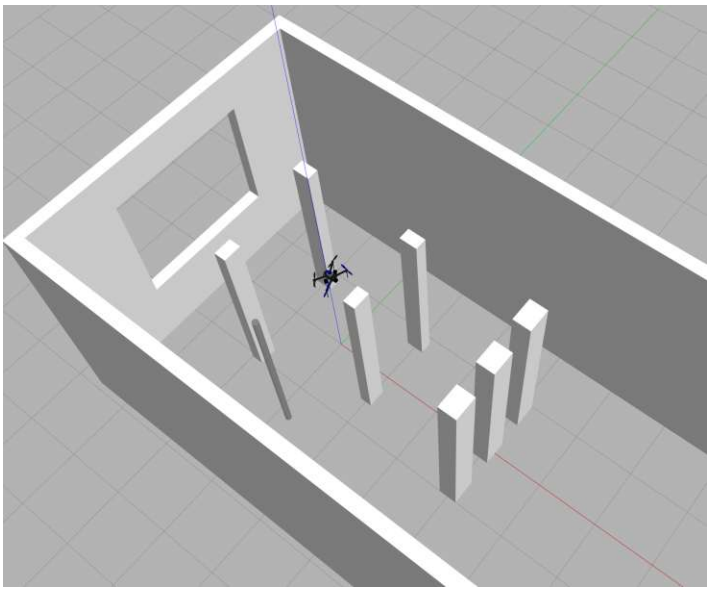
Finally, obstacle avoidance using the actual sensor data is performed. Since it is impossible to test an entire flight path with the UAV model in a simulation environment while using the actual sensor data on the physical UAV, only plausibility tests are performed to validate the avoidance maneuvers in simulation. Different shaped and sized obstacles are approximated to the real UAV platform, and the resulting avoidance movements are observed in the Gazebo simulation. The tests show that all obstacles are reliably detected, and the resulting avoidance algorithms generate smooth avoidance movements as expected.



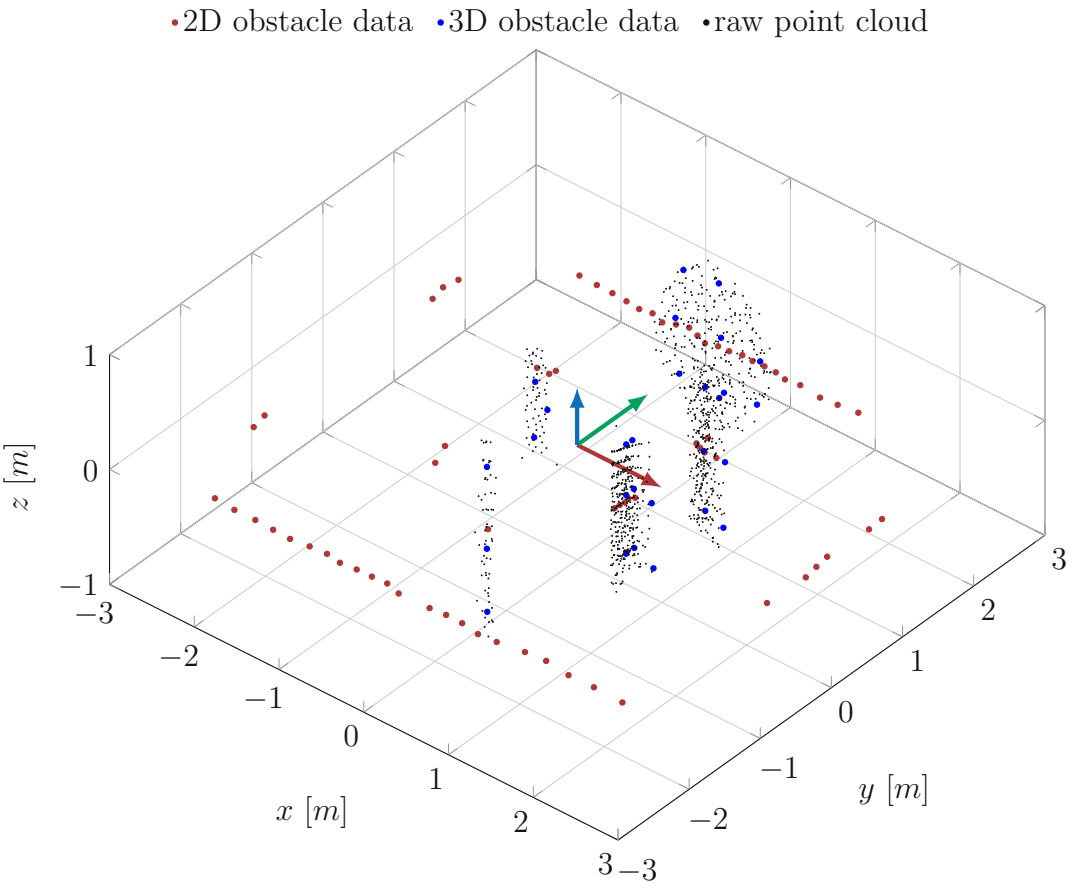
(a) Simulation data (b) Real sensor data

Figure 6.10.: Extracted obstacle points from simulated and real sensors in xy-plane. Red points represent 2D obstacle data points from the laser scanner,nd blue points are effective 3D obstacle data points extracted from the point cloud.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.



(a) Gazebo simulation of the obstacle setting in the flight laboratory.



(b) Extracted data points for obstacle information.

Figure 6.11.: Obstacle extraction in simulation. The black points illustrate the raw point cloud data, the blue points are the extracted 3D feature points, and the red points are the effective 2D laser scan points.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

6.4. Experimental Validation

Test flights with simple obstacle configurations are carried out to validate the simulation results in the AITs flight laboratory. The HITL tests show that the DWA cannot operate in real-time on the UAV hardware. Consequently, only the APF method is tested in actual flights. In Figure 6.12 the experiment setups are pictured. Note that in these experiments, positioning is based solely on data from the IMU and optical flow and does not utilize any SLAM data. Moreover, the safety considerations in the test flights are more stringent than in the simulations. Table 6.4 displays the parameter values of the APF method used in simulations and test flights. A reduction in the step size parameter α decreases the speed of the UAV and the switching conditions were disabled to prevent a transition to the Bug2 algorithm.



(a) Setup for only repelling movement.

(b) Complete obstacle avoidance setup.

Figure 6.12.: Experimental obstacle settings in the flight lab for the test flights using the APF method.

Experiment 1

In the first experimental test (setup Figure 6.12a), only the repulsive force generates an avoidance movement, as no target position with a corresponding attractive force is active. The UAV is manually controlled close to an obstacle, and upon activation of the collision avoidance system, the UAV is automatically repositioned to a safer distance. Figure Figure 6.13 illustrates the process of repelling avoidance: The minimum obstacle distance decreases as the UAV flies towards the obstacle. At time t_{active}, the collision avoidance is activated, initiating the avoidance movement and guiding the UAV to a position with a safe distance $\delta_{save} = 1.0$ m from the obstacle reaching the safe position at t_{safe}.

$\delta_{obs_{min}}$ m	$\overline{\delta}_{obs}$ m	d_{path} m	d_{direct} m	t s	v_{max} ms^{-1}	$\overline{v}$ ms^{-1}	$\overline{t}_{exe}$ s	J_{RMS} ms^{-3}	κ_{RMS} m^{-1}
0.43	–	0.72	–	4.60	0.24	0.15	52.76	0.42	0.69

Table 6.3.: Performance metrics from real flight experiment 1.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

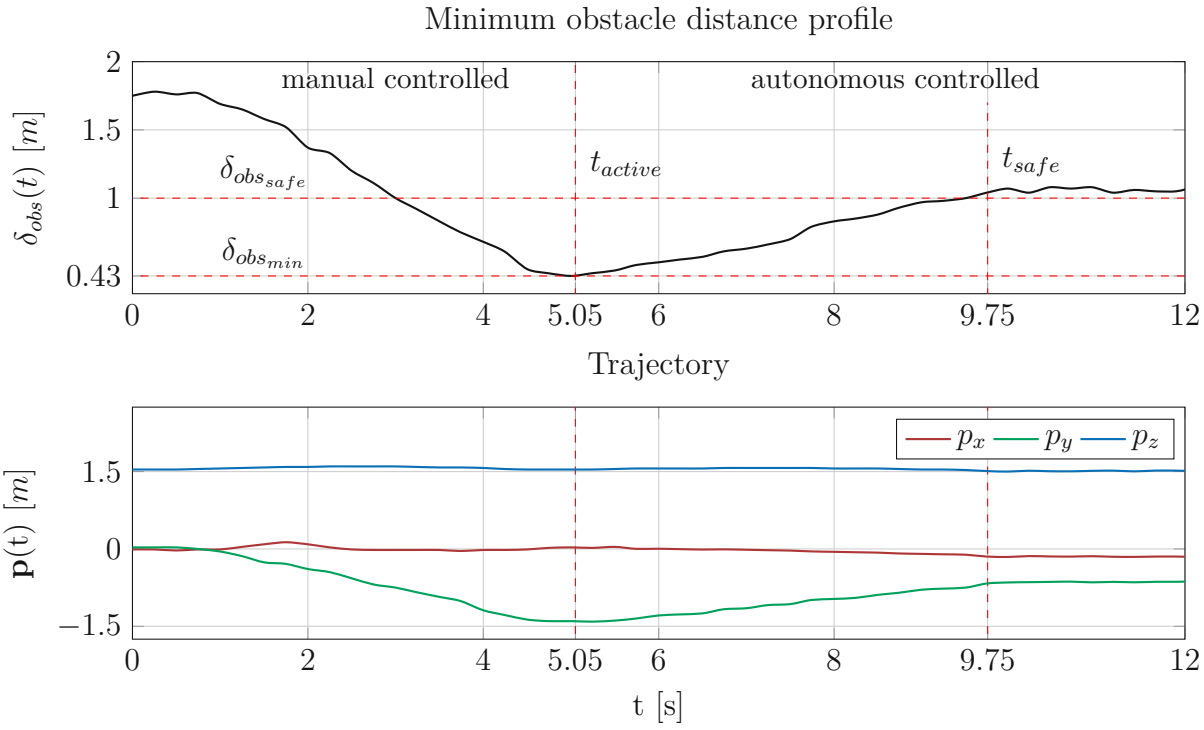


Figure 6.13.: Experiment 1: Minimum distance profile and trajectory of APF avoidance movement with only repelling force. The time t_{active} indicates where the switch to the avoidance algorithm is performed, and t_{safe} is the time where the UAV maintained a safe obstacle distance $\delta_{obs_{safe}}$. In the plot $\mathbf{p}_{start} = [0, 0, z_{start}]^T$ is defined for easier visualization.

Parameter	Simulation	Experiment	Unit
Attractive force scaling constant $\mathbf{k_a}$	$[850, 850, 850]^T$	$[850, 850, 850]^T$	
Repulsive force scaling constant $\mathbf{k_r}$	$[195, 195, 175]^T$	$[195, 195, 150]^T$	
Repulsive z force scaling constant $k_{g,c}$	215	200	
Boundary coefficient attractive force ρ	0.5	0.5	m
Max./min. step size $\alpha_{max}/\alpha_{min}$	1.5/0.5	0.25/0.25	
Max./min. 2D influence radius $d_{0_{max}}$, $d_{0_{min}}$	3.0/0.75	3.0/1.0	m
Max./min. 3D influence radius $d_{0_{max}}$, $d_{0_{min}}$	2.0/0.75	1.5/1.0	m
Threshold influence z distance z_{th}	1.0	0.75	m
Max./min. obstacle distance δ_{th_L}, δ_{th_H}	2.5/0.75	2.5/1.0	m
Max. rotational factor $\tilde{k}_{max}$	0.5	0.25	
Force threshold (switching condition) F_{th}	150	∞	N
Considered positions k (switching condition)	10	∞	
Threshold distance δ_{th} (switching condition)	0.3	0.3	m

Table 6.4.: Used APF parameter values in simulation and for test flight experiments.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Experiment 2

In the second experiment, an obstacle blocks the path from the start position to the goal position, as illustrated in Figure 6.12b. The UAV is manually controlled to the start position $\mathbf{p}_{start}$ and autonomously controlled to the goal position $\mathbf{p}_{goal}$ where $\mathbf{p}_{goal} = [x_{start} + 3m, y_{start}, z_{start}]^T$. An obstacle is placed in the line of sight from the start to the goal position. In Figure 6.14, the resulting avoidance path is pictured, and in Figure 6.15, the corresponding minimal obstacle distance and the trajectories of the test flight are plotted. The second test flight successfully generates a comprehensive avoidance path. In Figure A.6, the speed and acceleration profile is added as an additional plot. Note that in the plots $\mathbf{p}_{start} = [0, 0, z_{start}]^T$ is defined for easier visualization.

The UAV initiates its flight by moving directly from the start position $\mathbf{p}_{start}$ to the target position $\mathbf{p}_{goal}$, guided primarily by the attractive force. This direct course continues until the repulsive force (from the obstacle) becomes strong enough to alter its trajectory. Subsequently, the UAV is steered along the edge of the obstacle, maintaining a safe minimum distance while still progressing towards. Upon reaching $\mathbf{p}_{goal}$, there is noticeable fluctuation around (refer to Figure 6.15 at $t > 27.00$ s), which occurs because the algorithm remains active, awaiting a switch back to manual mode, rather than shutting off immediately upon reaching the destination.



Figure 6.14.: Experiment 2: Complete APF avoidance path in real flight experiment from start position $\mathbf{p}_{start}$ to goal position $\mathbf{p}_{goal}$, with $\mathbf{p}_{goal}$ behind obstacle.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

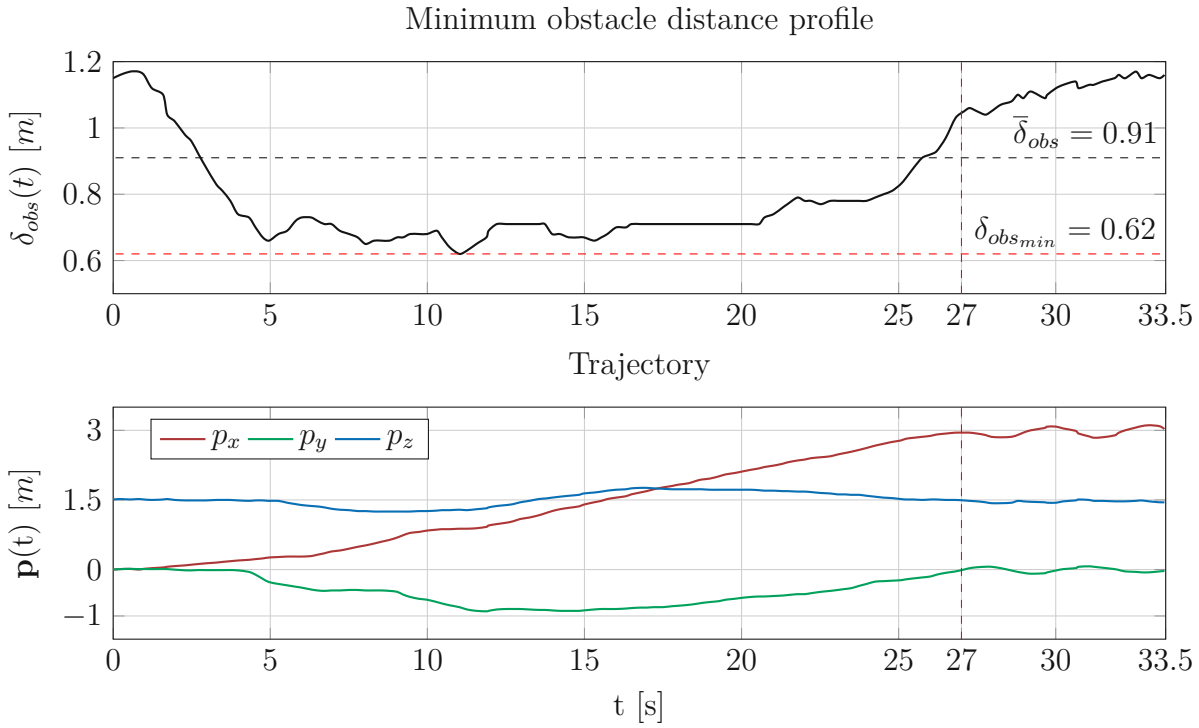


Figure 6.15.: Experiment 2: Minimum obstacle distance profile and avoidance trajectory of the APF avoidance movement from the start to the goal position. The red line at $t = 27.00$ s indicates where the target position $\mathbf{p}_{goal}$ is reached.

$\delta_{obs_{min}}$ m	$\overline{\delta}_{obs}$ m	d_{path} m	d_{direct} m	t s	v_{max} ms^{-1}	$\overline{v}$ ms^{-1}	$\overline{t}_{exe}$ s	J_{RMS} ms^{-3}	κ_{RMS} m^{-1}
0.62	0.91	5.63	3.00	27.00	0.24	0.19	55.56	0.54	0.86

Table 6.5.: Performance metrics real flight experiment 2.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

7. Conclusions and Future Work

In this work, an autonomous indoor hardware platform for a crisis and disaster management project is developed, and two different reactive collision avoidance systems are explored. The constructed quadcopter UAV, equipped with ToF and LiDAR sensors, is capable of autonomous exploration flights, providing a 3D map of the environment and enabling 3D collision avoidance.

The collision avoidance algorithms implemented, namely the APF+ and DWA, effectively navigate around obstacles of any shape and size in all three dimensions, creating smooth avoidance trajectories. Additionally, with the integration and transition to the Bug2 algorithm, the local minima problem is successfully addressed. The APF method and DWA have been extended to a three-dimensional implementation, allowing for effective obstacle avoidance in the x-y plane and the z-direction.

In the APF method, an extra potential function is implemented to consider data from the upward and downward-facing ToF sensors, and the virtual forces are three-dimensional. This method is particularly adept at operating in confined spaces, such as navigating through doors, by dynamically adjusting the influence radius based on the flight speed. To ensure the generation of smooth trajectories, the method uses additional rotational forces, which are dynamically scaled, along with a variable step size. Compared to the traditional APF approach, this enhanced method creates smoother trajectories, reduces the time to reach obstacles, and maintains a safe minimum distance from obstacles.

The DWA has also been successfully adapted to a 3D implementation by incorporating velocities in the z-direction and treating the UAV as holonomic. This adaptation significantly expands the velocity search space, making a restriction on the number of velocity samples to be simulated necessary. Unlike some implementations that only consider the nearest obstacle, our version of DWA considers all detected obstacles, enabling the generation of safe trajectories even in complex configurations. However, the trajectories produced by this 3D DWA are not as smooth as those generated by the proposed APF+ method, and the execution time per timestep is noticeably higher in the DWA than in the APF method. Based on these findings, the APF+ method has been chosen as the final collision avoidance algorithm to be implemented on the UAV platform.

Comprehensive testing of the collision avoidance system was primarily conducted within a simulated environment. While sensor readings for obstacle detection and execution times were assessed on the actual hardware, and some HITL tests were carried out, further testing in real-world environments is necessary for thorough validation.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.
TU Bibliothek Your knowledge hub WIEN

Test flights were conducted exclusively using the APF+ method because the DWA could not operate on our companion computer in real time.

Currently, the algorithms are implemented in Python, and transitioning them to C++ might lead to improved performance speeds. The avoidance of getting trapped in local minima is resolved by switching to the Bug2 algorithm, which is only capable of movements in the 2D plane in our implementation. In particular configurations, the combination of the algorithm still produces sub-optimal paths or gets trapped in endless loops. As an improvement, the global scope could be considered to avoid obstacles in advance more effectively, or memory-based approaches checking for already visited positions could be added to avoid endless loops. In the final project, the global planner effectively resolves situations involving local minima, eliminating the need to switch to the Bug2 algorithm.

In our setup, the collision avoidance algorithm operates independently on the UAV, communicating with the global exploration planner via ROS topics and Wi-Fi. A potential enhancement involves directly integrating the avoidance algorithm into the planner framework, including the path planner on the companion computer. To achieve this, replacing the existing companion computer with a more powerful one would be necessary. Additionally, incorporating small ultrasound sensors could improve the ability of the system to detect transparent objects like windows, which currently pose a challenge to the overall system.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

A. Appendix

A.1. Smoothness Factors

To measure the smoothness of a trajectory in 3D space, two commonly used metrics are the RMS Jerk and the RMS Curvature [115, 116].

RMS Jerk

The Jerk $\mathbf{J}(t)$ is defined as the third time derivative of the position vector $\mathbf{p}(t)$:

$$\mathbf{J}(t) = \frac{d^3\mathbf{p}(t)}{dt^3} = \dddot{\mathbf{p}}(t) \tag{A.1}$$

The RMS Jerk J_{RMS} is given by:

$$J_{RMS} = \sqrt{\frac{1}{T}\int_0^T |\mathbf{J}(t)|^2 \, dt} \tag{A.2}$$

RMS Curvature

The curvature $\kappa(t)$ of a curve $\mathbf{p}(t)$ in 3D is defined as:

$$\kappa(t) = \frac{\|\dot{\mathbf{p}}(t) \times \ddot{\mathbf{p}}(t)\|}{\|\dot{\mathbf{p}}(t)\|^3} \tag{A.3}$$

where the RMS Curvature κ_{RMS} is then:

$$\kappa_{RMS} = \sqrt{\frac{1}{T}\int_0^T \kappa^2(t) \, dt} \tag{A.4}$$

These two metrics are utilized to evaluate the smoothness of the generated avoidance trajectories in the present thesis.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

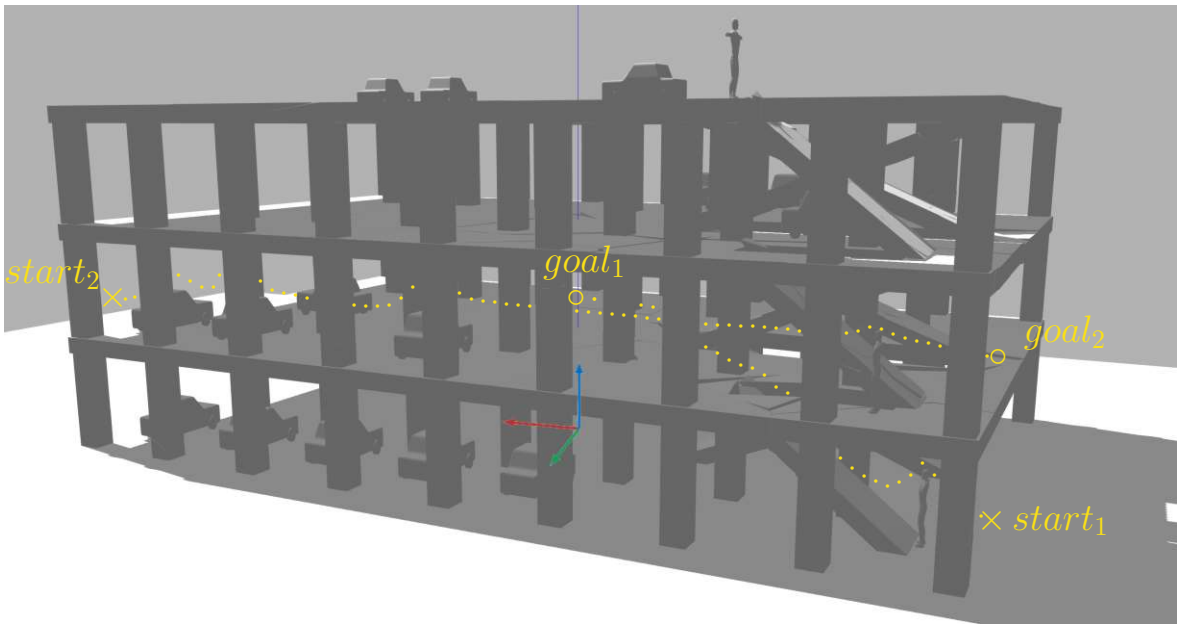


Figure A.1.: Simulation environment Destroyed Garage with goal and start positions and illustrative trajectories.

A.2. Simulation Results Destroyed Garage

A.2.1. APF+ and DWA Performance Metrics

Path	$\delta_{obs_{min}}$ m	$\overline{\delta}_{obs}$ m	d_{path} m	d_{direct} m	t s	v_{max} ms^{-1}	$\overline{v}$ ms^{-1}	$\overline{t}_{exe}$ s	J_{RMS} ms^{-3}	κ_{RMS} m^{-1}
$1 \rightarrow 1$	0.52	1.74	28.27	22.23	40.00	1.28	0.72	4.12	1.70	0.79
$2 \rightarrow 2$	0.61	1.56	41.01	38.70	53.25	1.40	0.76	3.46	1.24	0.71

Table A.1.: APF+ performance metrics simulation results Destroyed Garage.

Path	$\delta_{obs_{min}}$ m	$\overline{\delta}_{obs}$ m	d_{path} m	d_{direct} m	t s	v_{max} ms^{-1}	$\overline{v}$ ms^{-1}	$\overline{t}_{exe}$ s	J_{RMS} ms^{-3}	κ_{RMS} m^{-1}
$1 \rightarrow 1$	0.18	0.97	41.55	22.30	38.50	1.10	0.60	40.79	1.75	0.88
$2 \rightarrow 2$	0.56	1.35	43.68	38.75	60.50	0.98	0.68	26.47	1.55	0.85

Table A.2.: DWA performance metrics simulation results Destroyed Garage.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

A.2.2. APF+ Trajectories

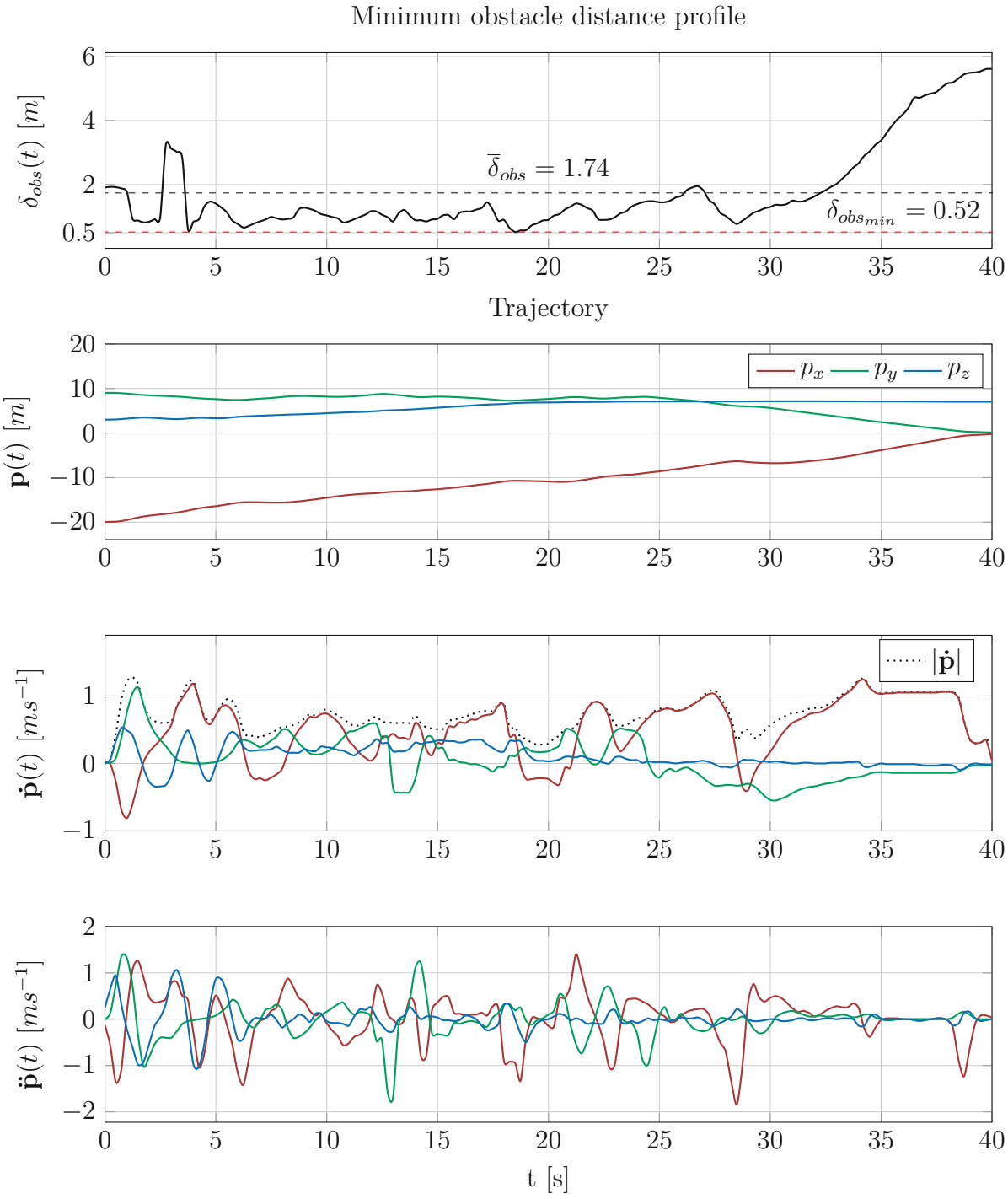


Figure A.2.: APF+ minimum obstacle distance profile and trajectories position $1 \rightarrow 1$.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

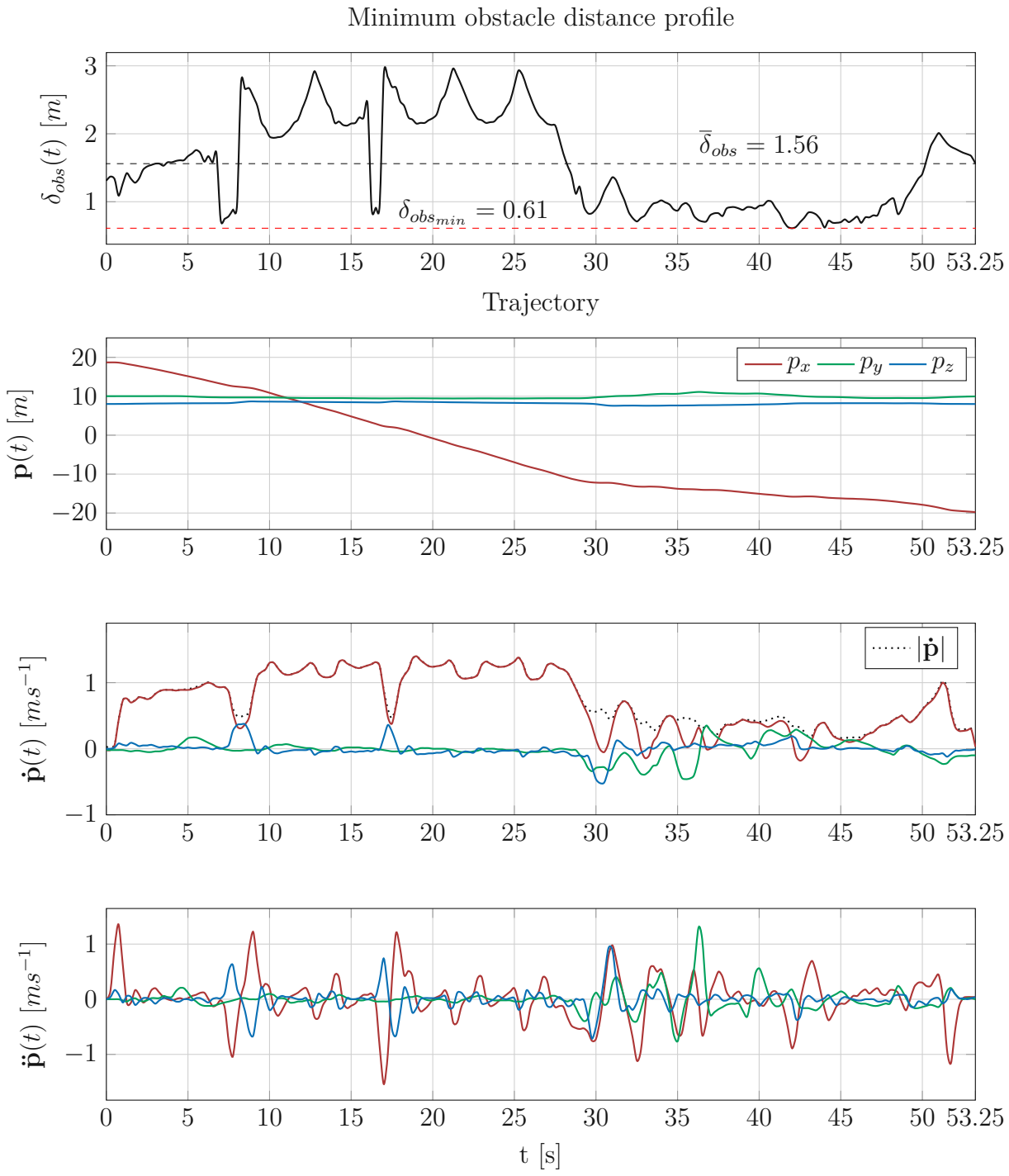


Figure A.3.: APF+ minimum obstacle distance profile and trajectories position 2 → 2.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

A.2.3. DWA Trajectories

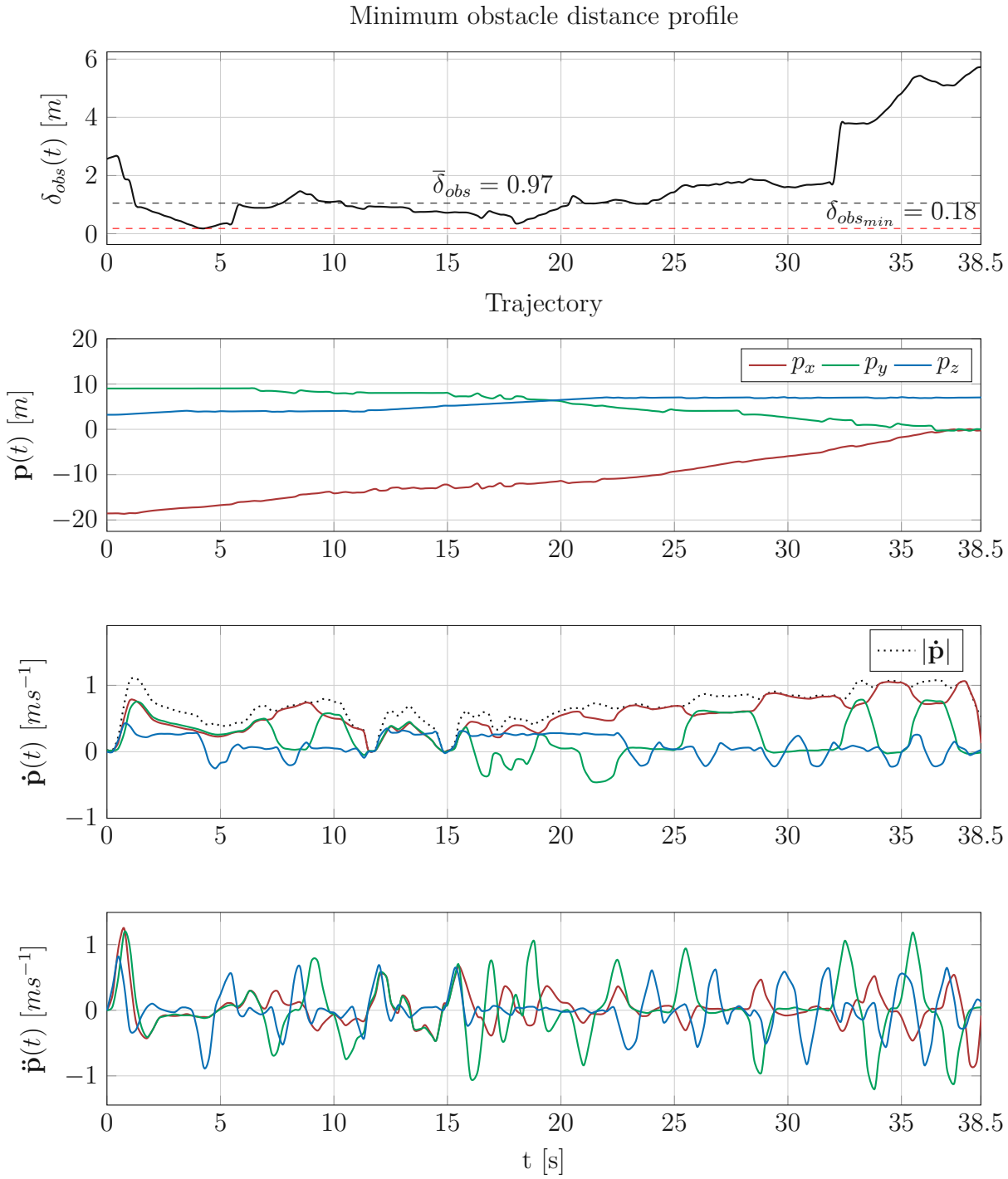


Figure A.4.: DWA minimum obstacle distance profile and trajectories 1 position → 1.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

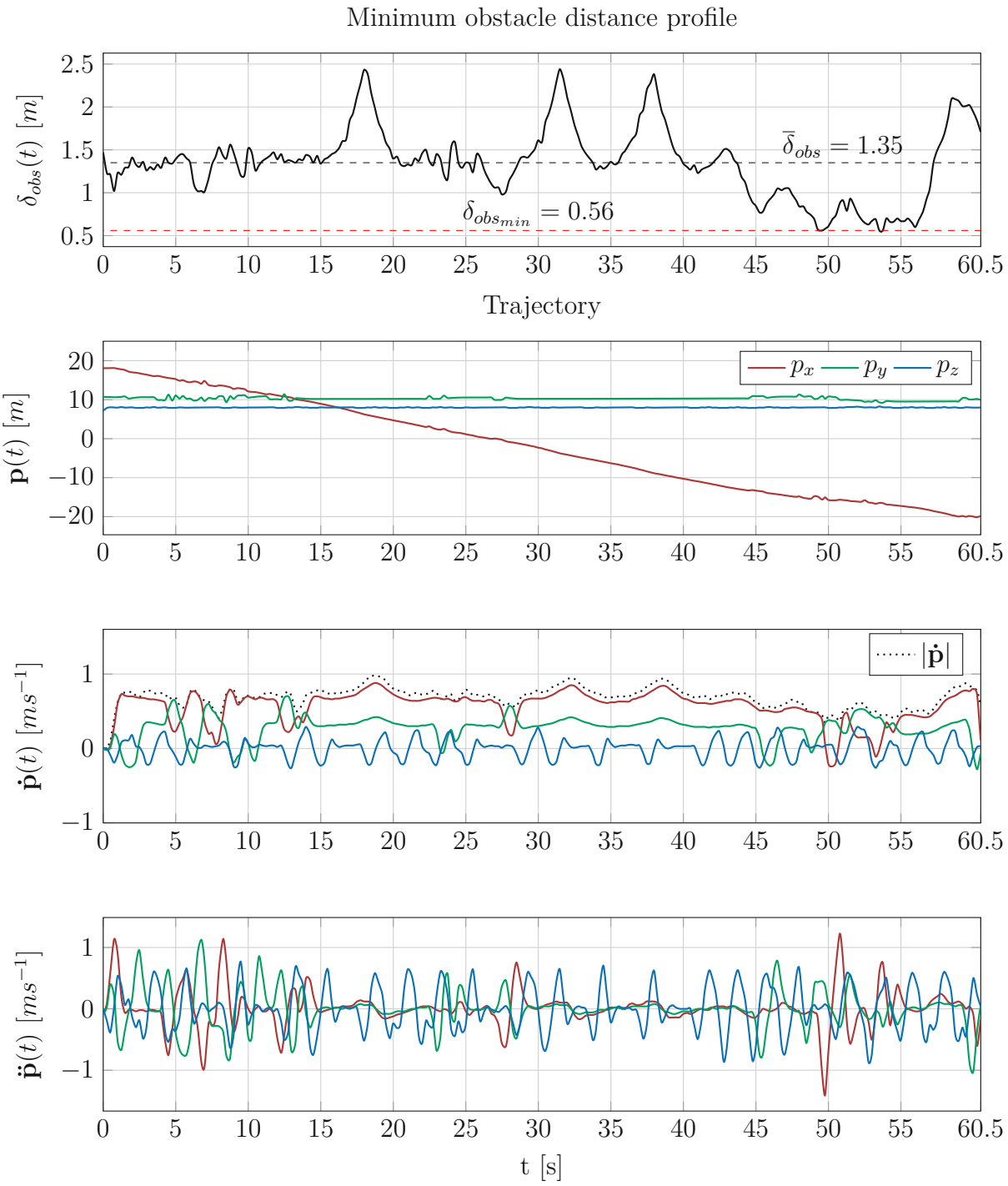


Figure A.5.: DWA minimum obstacle distance profile and trajectories position 2 → 2.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

A.3. APF Test Flight Trajectories

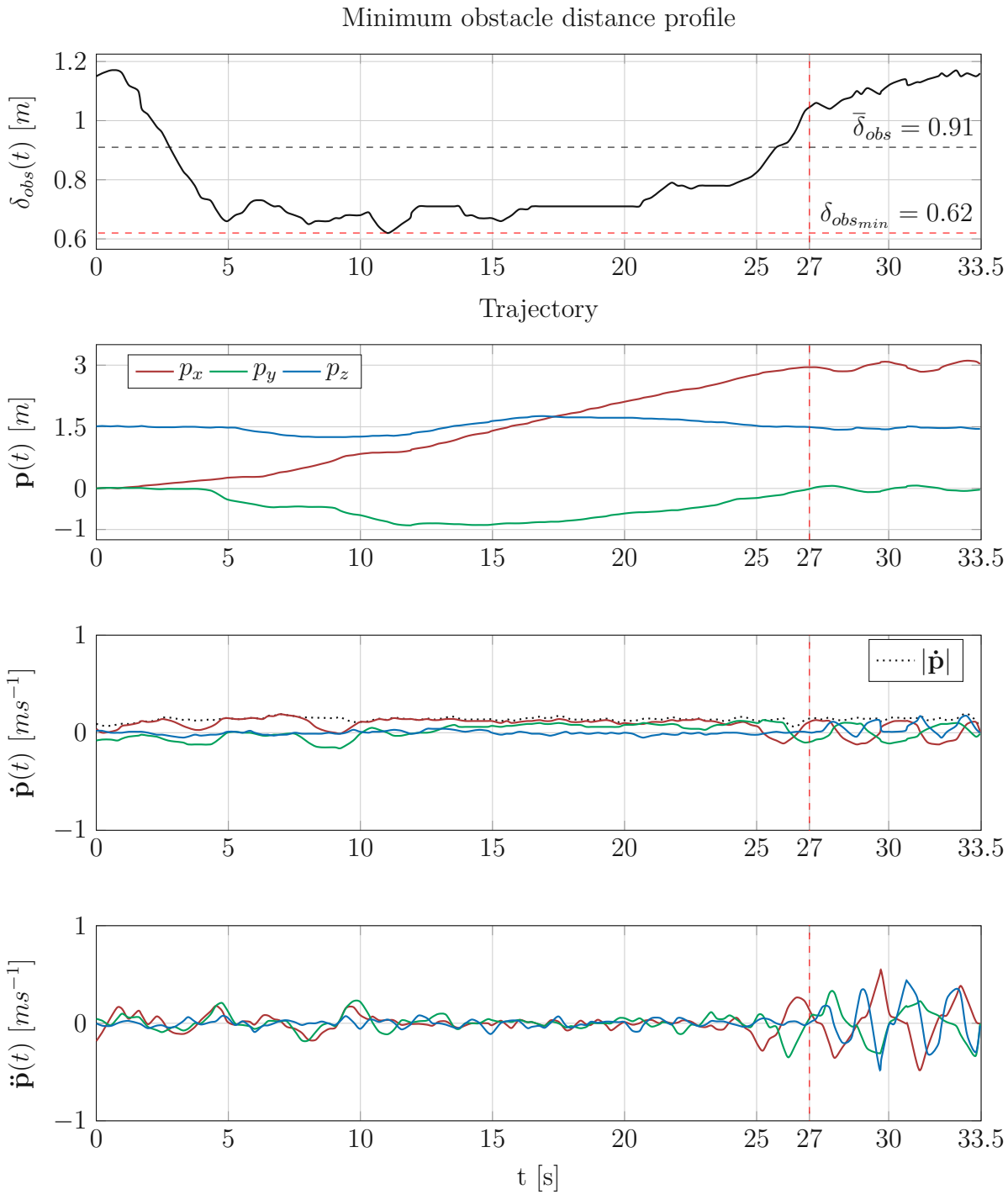


Figure A.6.: APF test flight minimum distance profile and trajectories.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Bibliography

[1] R. Austin. *Unmanned Aircraft Systems: UAVs Design, Development and Deployment*. John Wiley & Sons, 2010.

[2] H. Chao, Y. Cao, and Y. Chen. Autopilots for small unmanned aerial vehicles: A survey. *International Journal of Control, Automation and Systems*, 8(1):36–44, 2010.

[3] Hartmut Surmann, Dominik Slomma, Stefan Grobelny, and Robert Grafe. Deployment of aerial robots after a major fire of an industrial hall with hazardous substances, a report. In *2021 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, pages 40–47, 2021.

[4] Junlin Li, Xingzhong Xiong, Yuehao Yan, and Yunhong Yang. A survey of indoor uav obstacle avoidance research. *IEEE Access*, 11:51861–51891, 2023.

[5] Yuefan Xu, Minling Zhu, Yue Xu, and Mingjie Liu. Design and implementation of uav obstacle avoidance system. In *2019 2nd International Conference on Safety Produce Informatization (IICSPI)*, pages 275–278. IEEE, 2019.

[6] Yang Yu, Wang Tingting, Chen Long, and Zhang Weiwei. Stereo vision based obstacle avoidance strategy for quadcopter uav. In *2018 Chinese Control And Decision Conference (CCDC)*, pages 490–494. IEEE, 2018.

[7] Nils Gageik, Thilo Müller, and Sergio Montenegro. Obstacle detection and collision avoidance using ultrasonic distance sensors for an autonomous quadrocopter. *University of Wurzburg, Aerospace information Technologhy (germany) Wurzburg*, pages 3–23, 2012.

[8] Marc Steven Krämer and Klaus-Dieter Kuhnert. Multi-sensor fusion for uav collision avoidance. In *Proceedings of the 2018 2nd International Conference on Mechatronics Systems and Control Engineering*, pages 5–12, 2018.

[9] Young K Kwag and Chul H Chung. Uav based collision avoidance radar sensor. In *2007 IEEE International geoscience and remote sensing symposium*, pages 639–642. IEEE, 2007.

[10] Xu Huang, Joseph K. P. Tsoi, and Nitish Patel. mmwave radar sensors fusion for indoor object detection and tracking. *Electronics*, 11(14), 2022.

[11] Xin Huang, Xiaoya Dong, Jing Ma, Kuan Liu, Shibbir Ahmed, Jinlong Lin, and Baijing Qiu. The improved a* obstacle avoidance algorithm for the plant protection uav with millimeter wave radar and monocular camera data fusion. *Remote Sensing*, 13(17):3364, 2021.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

[12] Xiyue Wang, Xinsheng Wang, Zhiquan Zhou, and Junjie Li. Research on uav obstacle detection based on data fusion of millimeter wave radar and monocular camera. In *2021 2nd International Conference on Computing, Networks and Internet of Things*, pages 1–6, 2021.

[13] Zhanjun Hao, Hao Yan, Xiaochao Dang, Zhongyu Ma, Peng Jin, and Wenze Ke. Millimeter-wave radar localization using indoor multipath effect. *Sensors*, 22:5671, 07 2022.

[14] Allen Ferrick, Jesse Fish, Edward Venator, and Gregory S Lee. Uav obstacle avoidance using image processing techniques. In *2012 IEEE international conference on technologies for practical robot applications (TePRA)*, pages 73–78. IEEE, 2012.

[15] Lanxiang Zheng, Ping Zhang, Jia Tan, and Fang Li. The obstacle detection method of uav based on 2d lidar. *IEEE Access*, 7:163437–163448, 2019.

[16] Faying Li, Juan Xiao, Wenyi Huang, and Sijie Cai. Research on the intelligent obstacle avoidance and path planning strategy of uav based on multi-sensor fusion. In *2022 IEEE International Conference on Advances in Electrical Engineering and Computer Applications (AEECA)*, pages 628–632. IEEE, 2022.

[17] Yuncheng Lu, Zhucun Xue, Gui-Song Xia, and Liangpei Zhang. A survey on vision-based uav navigation. *Geo-spatial information science*, 21(1):21–32, 2018.

[18] Abdulla Al-Kaff, Qinggang Meng, David Martín, Arturo de la Escalera, and José María Armingol. Monocular vision-based obstacle detection/avoidance for unmanned aerial vehicles. In *2016 IEEE intelligent vehicles symposium (IV)*, pages 92–97. IEEE, 2016.

[19] Yiming Mao, Ming Chen, Xianglin Wei, and Bing Chen. Obstacle recognition and avoidance for uavs under resource-constrained environments. *IEEE Access*, 8:169408–169422, 2020.

[20] Zhouyu Zhang, Yunfeng Cao, Meng Ding, Likui Zhuang, and Jiang Tao. Monocular vision based obstacle avoidance trajectory planning for unmanned aerial vehicle. *Aerospace Science and Technology*, 106:106199, 2020.

[21] Zhiyong Chen, Xiaoyuan Luo, and Bicheng Dai. Design of obstacle avoidance system for micro-uav based on binocular vision. In *2017 International Conference on Industrial Informatics-Computing Technology, Intelligent Technology, Industrial Information Integration (ICIICII)*, pages 67–70. IEEE, 2017.

[22] Siyu Chen, Lei Wang, and Wei Chen. Research on optimization of visual navigation algorithm for real-time obstacle avoidance of uav moving target tracking. In *2019 IEEE 1st International Conference on Civil Aviation Safety and Information Technology (ICCASIT)*, pages 315–320. IEEE, 2019.

[23] Zhefan Xu, Xiaoyang Zhan, Baihan Chen, Yumeng Xiu, Chenhao Yang, and Kenji Shimada. A real-time dynamic obstacle tracking and mapping system for

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

uav navigation and collision avoidance with an rgb-d camera. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10645–10651. IEEE, 2023.

[24] Milton CP Santos, Lucas V Santana, Alexandre S Brandao, and Mário Sarcinelli-Filho. Uav obstacle avoidance using rgb-d system. In *2015 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 312–319. IEEE, 2015.

[25] L. Wang et al. Applications and prospects of agricultural unmanned aerial vehicle obstacle avoidance technology in china. *Sensors (Basel)*, 19, 2019.

[26] Edsger W Dijkstra. A note on two problems in connexion with graphs. In *Edsger Wybe Dijkstra: His Life, Work, and Legacy*, pages 287–290. 2022.

[27] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[28] Anthony Stentz. Optimal and efficient path planning for partially-known environments. In *Proceedings of the 1994 IEEE international conference on robotics and automation*, pages 3310–3317. IEEE, 1994.

[29] Steven LaValle. Rapidly-exploring random trees: A new tool for path planning. *Research Report 9811*, 1998.

[30] Ningyi Cheng, Zhiqian Liu, and Yuqi Li. Path planning algorithm of dijkstra-based intelligent aircraft under multiple constraints. *Xibei Gongye Daxue Xuebao/Journal of Northwestern Polytechnical University*, 38(6):1284–1290, 2020.

[31] Xiong Bai, Haikun Jiang, Junjie Cui, Kuan Lu, Pengyun Chen, and Ming Zhang. Uav path planning based on improved a and dwa algorithms. *International journal of aerospace engineering*, 2021:1–12, 2021.

[32] Fan Yang, Xi Fang, Fei Gao, Xianjin Zhou, Hao Li, Hongbin Jin, and Yu Song. Obstacle avoidance path planning for uav based on improved rrt algorithm. *Discrete Dynamics in Nature and Society*, 2022:1–9, 2022.

[33] Karthik Karur, Nitin Sharma, Chinmay Dharmatti, and Joshua E Siegel. A survey of path planning algorithms for mobile robots. *Vehicles*, 3(3):448–468, 2021.

[34] Chad Goerzen, Zhaodan Kong, and Bernard Mettler. A survey of motion planning algorithms from the perspective of autonomous uav guidance. *Journal of Intelligent and Robotic Systems*, 57:65–100, 2010.

[35] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. *The international journal of robotics research*, 5(1):90–98, 1986.

[36] Shuzhi Sam Ge and Yong-Jin Cui. Dynamic motion planning for mobile robots using potential field method. *Autonomous Robots*, 13(3):207–222, 2002.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

[37] Yuzhen Xu, Jang Hwan Choi, and Sangmin Oh. Motion planning of aerial robots via hierarchical reinforcement learning. In *2015 IEEE 7th International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE Conference on Robotics, Automation and Mechatronics (RAM)*, pages 144–149. IEEE, 2015.

[38] Johann Borenstein and Yoram Koren. The vector field histogram-fast obstacle avoidance for mobile robots. *IEEE transactions on robotics and automation*, 7(3):278–288, 1991.

[39] Elon Rimon and Daniel E Koditschek. Exact robot navigation using artificial potential functions. *IEEE Transactions on Robotics and Automation*, 8(5):501–518, 1992.

[40] Han Wang, Muqing Cao, Hao Jiang, and Lihua Xie. Feasible computationally efficient path planning for uav collision avoidance. In *2018 IEEE 14th International Conference on Control and Automation (ICCA)*, pages 576–581. IEEE, 2018.

[41] Xi Lin, Yuan-Jin Yu, Shi-Zhuang Chen, and Yang-Yang Shi. An improved apf method for complex and dynamic obstacles' avoidance. *Unmanned Systems*, 11(02):175–189, 2023.

[42] Taifei Zhao, Jiatong Yao, Chunjie Gong, and Yiqiong Wang. Wireless ultraviolet light mimo assisted uav direction perception and collision avoidance method. *Physical Communication*, 54:101815, 2022.

[43] Wenjing Li, Feng Gao, and Zhongjing Wang. A new artificial potential field method based on harmonic potential field and virtual water flow. *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, 226(1):74–84, 2012.

[44] Thi Thoa Mac, Cosmin Copot, Andres Hernandez, and Robin De Keyser. Improved potential field method for unknown obstacle avoidance using uav in indoor environment. In *2016 IEEE 14th International Symposium on Applied Machine Intelligence and Informatics (SAMI)*, pages 345–350, 2016.

[45] Björn Lindqvist, Jakub Haluska, Christoforos Kanellakis, and George Nikolakopoulos. An adaptive 3d artificial potential field for fail-safe uav navigation. In *2022 30th Mediterranean Conference on Control and Automation (MED)*, pages 362–367, 2022.

[46] Feng Tang, Shiquan Jiang, Chuanxi Li, and Xiaofeng Tang. Path planning based on ant colony algorithm and distributed local navigation for multi-agent systems. *Journal of Control Theory and Applications*, 7(3):265–271, 2009.

[47] Jinwen Hu, Changwei Cheng, Ce Wang, Chunhui Zhao, Quan Pan, and Zhenbo Liu. An improved artificial potential field method based on dwa and path optimization. In *2019 IEEE International Conference on Unmanned Systems (ICUS)*, pages 809–814, 2019.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

[48] Lei Tai, Giuseppe Paolo, and Ming Liu. Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 31–36. IEEE, 2017.

[49] Lingyu Li, Defeng Wu, Youqiang Huang, and Zhi-Ming Yuan. A path planning strategy unified with a colregs collision avoidance function based on deep reinforcement learning and artificial potential field. *Applied Ocean Research*, 113:102759, 2021.

[50] Johann Borenstein and Y Koren. High-speed obstacle avoidance for mobile robots. In *Proceedings 1988 IEEE International Symposium on Intelligent Control*, pages 382–383. IEEE Computer Society, 1988.

[51] Iwan Ulrich and Johann Borenstein. Vfh+: Reliable obstacle avoidance for fast mobile robots. In *Proceedings. 1998 IEEE international conference on robotics and automation (Cat. No. 98CH36146)*, volume 2, pages 1572–1577. IEEE, 1998.

[52] Iwan Ulrich and Johann Borenstein. Vfh/sup*: Local obstacle avoidance with look-ahead verification. In *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065)*, volume 3, pages 2505–2511. IEEE, 2000.

[53] Simon Vanneste, Ben Bellekens, and Maarten Weyn. 3dvfh+: Real-time three-dimensional obstacle avoidance using an octomap. In *MORSE 2014 Model-Driven Robot Software Engineering: proceedings of the 1st International Workshop on Model-Driven Robot Software Engineering co-located with International Conference on Software Technologies: Applications and Foundations (STAF 2014), York, UK, July 21, 2014/Assmann, Uwe [edit.]*, number 1319, pages 91–102, 2014.

[54] Tanja Baumann. Obstacle avoidance for drones using a 3dvfh* algorithm. *Spring Term 2018*, 67, 2018.

[55] Natallia Urieva, Jeffrey McDonald, Tatsiana Uryeva, April Sandy Rose Ramos, and Subodh Bhandari. Collision detection and avoidance using optical flow for multicopter uavs. In *2020 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 607–614, 2020.

[56] Giancarlo Allasia, Alessandro Rizzo, and Kimon Valavanis. Quadrotor uav 3d path planning with optical-flow-based obstacle avoidance. In *2021 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 1029–1036, 2021.

[57] Jean-Christophe Zufferey and Dario Floreano. Fly-inspired visual steering of an ultralight indoor aircraft. *IEEE Transactions on Robotics*, 22(1):137–146, 2006.

[58] Pinghai Gao, Daibing Zhang, Qiang Fang, and Shaogang Jin. Obstacle avoidance for micro quadrotor based on optical flow. In *2017 29th Chinese Control And Decision Conference (CCDC)*, pages 4033–4037, 2017.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

[59] Deqing Sun, Stefan Roth, and Michael J Black. Secrets of optical flow estimation and their principles. In *2010 IEEE computer society conference on computer vision and pattern recognition*, pages 2432–2439. IEEE, 2010.

[60] Qifeng Chen and Vladlen Koltun. Full flow: Optical flow estimation by global optimization over regular grids. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4706–4714, 2016.

[61] Bruce D Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. In *IJCAI'81: 7th international joint conference on Artificial intelligence*, volume 2, pages 674–679, 1981.

[62] Berthold KP Horn and Brian G Schunck. Determining optical flow. *Artificial intelligence*, 17(1-3):185–203, 1981.

[63] Philipp Fischer, Alexey Dosovitskiy, Eddy Ilg, Philip Häusser, Caner Hazırbaş, Vladimir Golkov, Patrick Van der Smagt, Daniel Cremers, and Thomas Brox. Flownet: Learning optical flow with convolutional networks. *arXiv preprint arXiv:1504.06852*, 2015.

[64] Deqing Sun, Xiaodong Yang, Ming-Yu Liu, and Jan Kautz. Pwc-net: Cnns for optical flow using pyramid, warping, and cost volume. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8934–8943, 2018.

[65] Haiyang Chao, Yu Gu, Jason Gross, Guodong Guo, mario Fravolini, and Marcello Napolitano. A comparative study of optical flow and traditional sensors in uav navigation. pages 3858–3863, 06 2013.

[66] Vladimir Lumelsky and Alexander Stepanov. Dynamic path planning for a mobile automaton with limited information on the environment. *IEEE transactions on Automatic control*, 31(11):1058–1063, 1986.

[67] K Taylor and SM LaValle. I-bug: An intensity-based bug algorithm. In *2009 IEEE International Conference on Robotics and Automation*, pages 3981–3986. IEEE, 2009.

[68] N Buniyamin, WW Ngah, N Sariff, and Z Mohamad. A simple local path planning algorithm for autonomous mobile robots. *International journal of systems applications, Engineering & development*, 5(2):151–159, 2011.

[69] Q-L Xu and G-Y Tang. Vectorization path planning for autonomous mobile agent in unknown environment. *Neural Computing and Applications*, 23(7-8):2129–2135, 2013.

[70] V.J. Lumelsky and T. Skewis. Incorporating range sensing in the robot navigation function. *IEEE Transactions on Systems, Man, and Cybernetics*, 20(5):1058–1069, 1990.

[71] I Kamon and E Rivlin. Sensory-based motion planning with global proofs. *IEEE transactions on Robotics and Automation*, 13(6):814–822, 1997.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

[72] X Yun and K-C Tan. A wall-following method for escaping local minima in potential field based motion planning. In *1997 8th International Conference on Advanced Robotics. Proceedings. ICAR'97*, pages 421–426. IEEE, 1997.

[73] M Finkelstein and B Hunte. Wall following with collision avoidance and mapping using a laser range finder. Technical report, 2008.

[74] I Gavrilut, V Tiponut, A Gacsadi, and L Tepelea. Wall-following method for an autonomous mobile robot using two ir sensors. In *WSEAS International Conference. Proceedings. Mathematics and Computers in Science and Engineering*, number 12. WSEAS, 2008.

[75] Dieter Fox, Wolfram Burgard, and Sebastian Thrun. The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, 4(1):23–33, 1997.

[76] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, Andrew Y Ng, et al. Ros: an open-source robot operating system. In *ICRA workshop on open source software*, volume 3, page 5. Kobe, Japan, 2009.

[77] Petter Ogren and Naomi Ehrich Leonard. A convergent dynamic window approach to obstacle avoidance. *IEEE Transactions on Robotics*, 21(2):188–195, 2005.

[78] Cyrill Stachniss and Wolfram Burgard. An integrated approach to goal-directed obstacle avoidance under dynamic constraints for dynamic environments. In *IEEE/RSJ international conference on intelligent robots and systems*, volume 1, pages 508–513. IEEE, 2002.

[79] Oliver Brock and Oussama Khatib. High-speed navigation using the global dynamic window approach. In *Proceedings 1999 ieee international conference on robotics and automation (Cat. No. 99CH36288C)*, volume 1, pages 341–346. IEEE, 1999.

[80] Jungho Moon, Byung-Yoon Lee, and Min-Jea Tahk. A hybrid dynamic window approach for collision avoidance of vtol uavs. *International Journal of Aeronautical and Space Sciences*, 19:889–903, 2018.

[81] Desheng Feng, Lixia Deng, Tao Sun, Haiying Liu, Hui Zhang, and Yang Zhao. Local path planning based on an improved dynamic window approach in ros. In *The 10th International Conference on Computer Engineering and Networks*, pages 1164–1171. Springer, 2021.

[82] Inara Tusseyeva, Seong-Gon Kim, and Yong-Gi Kim. 3d global dynamic window approach for navigation of autonomous underwater vehicles. *International Journal of Fuzzy Logic and Intelligent Systems*, 13, 06 2013.

[83] Ce Xu, Zhenbang Xu, and Mingyi Xia. Obstacle avoidance in a three-dimensional dynamic environment based on fuzzy dynamic windows. *Applied Sciences*, 11(2):504, 2021.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

[84] Jawad N Yasin, Sherif AS Mohamed, Mohammad-Hashem Haghbayan, Jukka Heikkonen, Hannu Tenhunen, and Juha Plosila. Unmanned aerial vehicles (uavs): Collision avoidance systems and approaches. *IEEE access*, 8:105139–105155, 2020.

[85] A. Imdoukh et al. Semi-autonomous indoor firefighting uav. In *International Conference on Advanced Robotics (ICAR)*, 2017.

[86] Norhafizan Ahmad, Raja Ariffin Raja Ghazilla, Nazirah M Khairi, and Vijayabaskar Kasi. Reviews on various inertial measurement unit (imu) sensor applications. *International Journal of Signal Processing Systems*, 1(2):256–262, 2013.

[87] Lorenz Meier, Petri Tanskanen, Friedrich Fraundorfer, and Marc Pollefeys. Pixhawk: A system for autonomous flight using onboard computer vision. In *2011 IEEE International Conference on Robotics and Automation*, pages 2992–2997, 2011.

[88] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6235–6240, 2015.

[89] Yuntian Li, Matteo Scanavino, Elisa Capello, Fabrizio Dabbene, Giorgio Guglieri, and Andrea Vilardi. A novel distributed architecture for uav indoor navigation. *Transportation Research Procedia*, 35:13–22, 2018.

[90] Cindy Leung, Shoudong Huang, and Gamini Dissanayake. Active slam using model predictive control and attractor based exploration. pages 5026 – 5031, 11 2006.

[91] Muhammad Farhan Ahmed, Khayyam Masood, Vincent Fremont, and Isabelle Fantoni. Active slam: A review on last decade. *Sensors*, 23(19):8097, 2023.

[92] F Vultaggio, F d'Apolito, C Sulzbachner, and P Fanta-Jende. Simulation of low-cost mems-lidar and analysis of its effect on the performances of state-of-the-art slams. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 48:539–545, 2023.

[93] MAVLink Developer Team. Mavlink: A micro air vehicle communication protocol, 2023. Available online: `https://mavlink.io/`.

[94] Stanford Artificial Intelligence Laboratory et al. Robotic operating system.

[95] Mark G Sobell. *A practical guide to Ubuntu Linux*. Pearson Education, 2015.

[96] Dma technical report tr8350.2-b. Technical report, Defense Mapping Agency, 12 1987. Second Printing, Supplement to DoD WGS 84 Technical Report Part 2 - Parameters, Formulas, and Graphics.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

[97] Teppo Luukkonen. Modelling and control of quadcopter. *Independent research project in applied mathematics, Espoo*, 22(22), 2011.

[98] James Diebel et al. Representing attitude: Euler angles, unit quaternions, and rotation vectors. *Matrix*, 58(15-16):1–35, 2006.

[99] Ademola Abdulkareem, Victoria Oguntosin, Olawale M Popoola, Ademola A Idowu, et al. Modeling and nonlinear control of a quadcopter for stabilization and trajectory tracking. *Journal of Engineering*, 2022, 2022.

[100] Matthias Faessler, Antonio Franchi, and Davide Scaramuzza. Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories. *IEEE Robotics and Automation Letters*, 3(2):620–626, 2017.

[101] Koushil Sreenath, Taeyoung Lee, and Vijay Kumar. Geometric control and differential flatness of a quadrotor uav with a cable-suspended load. In *52nd IEEE Conference on Decision and Control*, pages 2269–2274, 2013.

[102] Marco Cella, Francesco d'Apolito, Phillipp Fanta-Jende, and Christoph Sulzbachner. Fueling glocal: Optimization-based path planning for indoor uavs in an autonomous exploration framework. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 48:85–91, 2023.

[103] Lukas Schmid, Michael Pantic, Raghav Khanna, Lionel Ott, Roland Siegwart, and Juan Nieto. An efficient sampling-based method for online informative path planning in unknown environments. *IEEE Robotics and Automation Letters*, 5(2):1500–1507, 2020.

[104] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[105] Martin Ester, Hans-Peter Kriegel, Jorg Sander, Xiaowei Xu, et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, volume 96, pages 226–231, 1996.

[106] Praveen Selvam, Gunasekaran Raja, Vasantharaj Rajagopal, Kapal Dev, and Sebastian Knorr. Collision-free path planning for uavs using efficient artificial potential field algorithm. pages 1–5, 04 2021.

[107] Praveen Kumar Selvam, Gunasekaran Raja, Vasantharaj Rajagopal, Kapal Dev, and Sebastian Knorr. Collision-free path planning for uavs using efficient artificial potential field algorithm. In *2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)*, pages 1–5. IEEE, 2021.

[108] Claude Lemaréchal. Cauchy and the gradient method. *Doc Math Extra*, 251(254):10, 2012.

[109] Analysis of potential fields for formation of multi-agent robots with collision avoidance. pages 115–120, 2017.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

[110] Jiayi Sun, Jun Tang, and Songyang Lao. Collision avoidance for cooperative uavs with optimized artificial potential field algorithm. *IEEE Access*, 5:18382–18390, 2017.

[111] M Wang, Z Su, D Tu, and X Lu. A hybrid algorithm based on artificial potential field and bug for path planning of mobile robot. In *Measurement, Information and Control (ICMIC), 2013 International Conference on*, volume 2, pages 1393–1398. IEEE, 2013.

[112] Y Zhu, T Zhang, and J Song. An improved wall following method for escaping from local minimum in artificial potential field based path planning. In *2009 48th IEEE Conference on Decision and Control (CDC)*, pages 6017–6022. IEEE, 2009.

[113] N. Koenig and A. Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, volume 3, pages 2149–2154 vol.3, 2004.

[114] Blender Online Community. *Blender - a 3D modelling and rendering package*. Blender Foundation, Stichting Blender Foundation, Amsterdam, 2018.

[115] Sivakumar Balasubramanian, Alejandro Melendez-Calderon, and Etienne Burdet. A robust and sensitive metric for quantifying movement smoothness. *IEEE transactions on biomedical engineering*, 59(8):2126–2136, 2011.

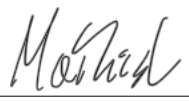
[116] ND Muñoz, JA Valencia, and N Londono. Evaluation of navigation of an autonomous mobile robot. In *Proceedings of the 2007 Workshop on Performance Metrics for Intelligent Systems*, pages 15–21, 2007.

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.

Eidesstattliche Erklärung

Hiermit erkläre ich, dass die vorliegende Arbeit gemäß dem Code of Conduct - Regeln zur Sicherung guter wissenschaftlicher Praxis (in der aktuellen Fassung des jeweiligen Mitteilungsblattes der TU Wien), insbesondere ohne unzulässige Hilfe Dritter und ohne Benutzung anderer als der angegebenen Hilfsmittel, angefertigt wurde. Die aus anderen Quellen direkt oder indirekt übernommenen Daten und Konzepte sind unter Angabe der Quelle gekennzeichnet. Die Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder in ähnlicher Form in anderen Prüfungsverfahren vorgelegt.

Vienna, January 2024



Noah Maikisch

Die approbierte gedruckte Originalversion dieser Diplomarbeit ist an der TU Wien Bibliothek verfügbar
The approved original version of this thesis is available in print at TU Wien Bibliothek.